AIF-C01 Master Glossary

AWS AI Cloud Practitioner Certification Prep Program

Reference Document - Task 9 of 15 Companion to the Master Project Blueprint and all five Domain Study Modules

---

How to Use This Glossary

This glossary is organized into two versions:

- **Version A: Alphabetical Glossary** - the primary, full-detail version. Use this for deep lookup of any single term. Every entry includes a simple definition, a detailed explanation, an everyday analogy, exam guidance, a worked example, a common misunderstanding to avoid, and the related AWS service (where applicable).
- **Version B: Domain-Organized Glossary** - a condensed version that groups every term under its primary exam domain, showing only the simple definition and exam relevance. Use this for fast domain-by-domain review once you have already studied the full entries at least once.

Every term that appears in the Master Project Blueprint's term list and the five Domain Study Modules is included here. If a term applies to multiple domains, it is filed under its most heavily tested domain in Version B, and that is noted in its Version A entry.

VERSION A: ALPHABETICAL GLOSSARY

A

Accuracy

Domain(s): 1, 3 **Priority:** High **Category:** AI/ML Concept

Simple Definition: Accuracy is the percentage of predictions a model got right out of all the predictions it made.

Detailed Explanation: Accuracy is calculated as the number of correct predictions divided by the total number of predictions made. It is one of the most intuitive model performance metrics, but it can be misleading when a dataset is imbalanced. For example, if 95% of transactions in a dataset are legitimate, a model that simply predicts "legitimate" every time would be 95% accurate while being completely useless at catching fraud. Because of this, accuracy is usually evaluated alongside precision, recall, and F1 score rather than on its own.

Everyday Analogy: Think of a weather forecaster who predicts "sunny" every single day in a desert town. They would be right almost all the time, giving them high accuracy, but they would never warn you about the rare rainy day, which is the day you actually needed the forecast to be useful.

Why It Matters for the Exam: The exam expects you to recognize accuracy as one of four core model performance metrics (with precision, recall, and F1 score) and to know that accuracy

alone can be a misleading metric for imbalanced datasets, such as fraud detection.

Example: A spam filter that correctly sorts 980 out of 1,000 emails has 98% accuracy.

Common Misunderstanding: Learners often assume high accuracy always means a good model. In reality, a model can have high accuracy and still perform very poorly at detecting the rare but important cases (like fraud or disease), which is why precision and recall matter.

Related AWS Service (if applicable): Amazon SageMaker AI (provides accuracy and other metrics during model training and evaluation).

Activation Function

Domain(s): 1 **Priority:** Low **Category:** AI/ML Concept

Simple Definition: An activation function is a small mathematical rule inside a neural network that decides whether and how strongly a piece of information should be passed forward to the next layer.

Detailed Explanation: In a neural network, each artificial neuron receives inputs, combines them, and then passes the result through an activation function before sending it to the next layer. The activation function introduces non-linearity, which allows the network to learn complex patterns rather than just simple straight-line relationships. Without activation functions, a deep neural network would behave like a single simple linear calculation no matter how many layers it had. Common activation functions include ReLU and sigmoid, though the exam does not require you to know their formulas.

Everyday Analogy: Think of a bouncer at a club entrance who decides, based on a set of rules, exactly how much of the crowd outside should be let through to the next room - not just yes or no, but how much.

Why It Matters for the Exam: This is a low-priority, conceptual-only term. The exam may reference it briefly as part of explaining how neural networks process information, but you will not need to calculate or select specific activation functions.

Example: In an image recognition neural network, an activation function helps the network decide whether a detected edge or shape is significant enough to influence the next layer's decision about what object is in the image.

Common Misunderstanding: Learners sometimes think activation functions are optional add-ons. In reality, they are a core structural component of every neural network layer, not an optional feature.

Related AWS Service (if applicable): Amazon SageMaker AI (used when building and training custom neural network models).

Adversarial Input

Domain(s): 4, 5 **Priority:** Medium **Category:** Security/Governance

Simple Definition: An adversarial input is data that has been deliberately crafted to trick an AI model into making a wrong decision.

Detailed Explanation: Adversarial inputs exploit weaknesses in how a model processes data, often through small, carefully designed changes that are imperceptible or nonsensical to a human but cause the model to fail. In generative AI specifically, an adversarial input is often a crafted prompt designed to bypass safety controls, which overlaps with concepts like jailbreaking and prompt injection. This is a security and robustness concern because it shows that a model's behavior can be manipulated by motivated bad actors rather than only by accident. Detecting and resisting adversarial inputs is part of building a robust, secure AI system.

Everyday Analogy: It is like a forged signature that looks close enough to the real one to fool a quick glance, but was deliberately designed to deceive the person checking it.

Why It Matters for the Exam: Expect this term in the context of Domain 4 (robustness) and Domain 5 (security threats). The exam wants you to recognize that AI systems can be deliberately targeted, not just accidentally wrong.

Example: An attacker slightly alters a stop sign image with stickers so that a computer vision model misclassifies it as a speed limit sign.

Common Misunderstanding: Learners sometimes confuse adversarial inputs with simple bad data or noise. The key distinguishing feature of an adversarial input is intent: it is deliberately engineered to cause a specific failure.

Related AWS Service (if applicable): Amazon Bedrock Guardrails (helps filter and reduce the effect of malicious or manipulative inputs on FM applications).

Agent (AI Agent)

Domain(s): 2, 3 **Priority:** High **Category:** GenAI Concept

Simple Definition: An AI agent is a system that can take a goal, break it into steps, and carry out those steps on its own - including using tools and making decisions along the way - without a human guiding every single action.

Detailed Explanation: Unlike a simple chatbot that only responds to one prompt at a time, an AI agent can plan a sequence of actions, call external tools or APIs, retrieve information, and adjust its plan based on what it learns at each step. Agents are a core part of agentic AI, the broader concept of AI systems acting autonomously toward a goal. Multiple agents can also work together in a multi-agent system, each handling part of a larger task. On AWS, agents are commonly built using frameworks like Strands Agents and deployed and managed using Amazon Bedrock AgentCore.

Everyday Analogy: Think of the difference between a vending machine (which only responds to one button press at a time) and a personal assistant who you ask to "plan and book my entire trip," who then independently checks flights, compares hotel prices, and confirms a rental car without you supervising every click.

Why It Matters for the Exam: The exam tests your ability to define the role of AI agents and describe their business applications (Domain 3) as well as foundational agentic AI concepts

like multi-agent systems and tool usage (Domain 2). Know the distinction between Strands Agents (the open-source framework) and Amazon Bedrock AgentCore (the managed runtime).

Example: A customer service agent that can look up an order in a database, check the return policy, and automatically issue a refund - all from a single customer request - is functioning as an AI agent.

Common Misunderstanding: Learners often think any chatbot is an "agent." A true AI agent is distinguished by autonomous multi-step action and tool use, not just generating a text response to a single prompt.

Related AWS Service (if applicable): Amazon Bedrock AgentCore and Strands Agents.

AI Bias

Domain(s): 4 **Priority:** High **Category:** Responsible AI

Simple Definition: AI bias happens when a model's outputs unfairly favor or disadvantage certain people or groups, usually because of patterns in the data it was trained on.

Detailed Explanation: AI bias is not the same thing as a person's intentional prejudice; it typically emerges unintentionally from skewed, incomplete, or historically unfair training data. If a dataset under-represents certain groups, or reflects past discriminatory practices, a model trained on that data can learn and repeat those patterns at scale. AI bias can show up in many forms, including demographic bias (favoring certain age, gender, or ethnic groups) and dataset bias (the training data itself is not representative). AWS provides tools like Amazon SageMaker Clarify specifically to help detect this kind of bias before and after deployment.

Everyday Analogy: Imagine training a new hiring manager by only showing them resumes from one company's past hires, who all happened to come from a handful of the same schools. The new manager would unintentionally start preferring those schools, even though that was never an explicit rule - they simply learned a biased pattern from limited examples.

Why It Matters for the Exam: Domain 4 directly tests your ability to identify bias as a core responsible AI feature, describe its effects on demographic groups, and name the AWS tools (especially SageMaker Clarify) used to detect it.

Example: A resume-screening model trained mostly on resumes from one demographic group may unfairly score equally qualified candidates from other groups lower.

Common Misunderstanding: Learners often think bias only comes from "bad" or malicious data. In reality, even well-intentioned, historically accurate data can produce biased outcomes if it reflects past inequities.

Related AWS Service (if applicable): Amazon SageMaker Clarify.

AI Governance

Domain(s): 5 **Priority:** High **Category:** Security/Governance

Simple Definition: AI governance is the set of policies, processes, and oversight an organization puts in place to make sure its AI systems are used responsibly, safely, and in compliance with regulations.

Detailed Explanation: AI governance covers decisions about who can approve a new AI use case, how model behavior is reviewed and audited, how often systems are re-evaluated, and how an organization demonstrates compliance to regulators or customers. It often relies on recognized frameworks, such as the Generative AI Security Scoping Matrix, to structure decision-making consistently. Good AI governance also requires team training so that employees understand their responsibilities when working with AI systems, and clear transparency standards so stakeholders understand how and where AI is being used. Governance is what turns responsible AI principles into repeatable organizational practice.

Everyday Analogy: Think of AI governance like a school's set of policies for field trips: rules about who can approve a trip, who supervises students, how risks are assessed beforehand, and how incidents get reported afterward. Without those policies, every trip would be handled inconsistently and some risks would go unmanaged.

Why It Matters for the Exam: Domain 5 (Task Statement 5.2) tests your ability to describe governance processes: policies, review cadence, governance frameworks, transparency standards, and training requirements.

Example: A company requires every new generative AI feature to pass a documented risk review and receive sign-off from a governance committee before it is released to customers.

Common Misunderstanding: Learners sometimes confuse AI governance (organizational policy and process) with AI security (technical protective measures). Governance is the "who decides and how" layer; security is the "how do we technically protect it" layer. They work together but are not the same thing.

Related AWS Service (if applicable): AWS Audit Manager, AWS Config, AWS CloudTrail.

AI Safety

Domain(s): 4, 5 **Priority:** High **Category:** Responsible AI

Simple Definition: AI safety means designing and operating AI systems so they do not cause physical, financial, emotional, or other harm to people.

Detailed Explanation: Safety is one of the core features of responsible AI alongside bias, fairness, inclusivity, robustness, and veracity. A safe AI system avoids generating harmful, dangerous, or offensive content, and it includes safeguards to prevent the system from being misused or from taking harmful autonomous actions. On AWS, safety controls are commonly implemented using tools like Amazon Bedrock Guardrails, which can filter content, restrict topics, and check for grounding before a response reaches a user. Safety also overlaps with security topics in Domain 5, such as toxicity detection and output filtering.

Everyday Analogy: Think of safety guardrails on a mountain road: they do not stop the car from driving, but they prevent it from going off a cliff if something goes wrong.

Why It Matters for the Exam: The exam includes safety as one of the named responsible

AI features in Task Statement 4.1, and expects you to connect it to tools like Amazon Bedrock Guardrails.

Example: A customer-facing chatbot is configured with guardrails so that it refuses to provide instructions for dangerous activities, even if a user repeatedly asks.

Common Misunderstanding: Learners sometimes treat "safety" and "security" as identical. Safety is about preventing harmful outputs and actions; security is about protecting the system itself from unauthorized access or attack. They are related but distinct.

Related AWS Service (if applicable): Amazon Bedrock Guardrails.

Algorithm

Domain(s): 1 **Priority:** High **Category:** AI/ML Concept

Simple Definition: An algorithm is a step-by-step set of rules or instructions that a computer follows to solve a problem or complete a task.

Detailed Explanation: In machine learning, an algorithm is the specific method used to find patterns in data during training, such as a decision tree algorithm or a linear regression algorithm. The algorithm is the "recipe," while the model is the "finished dish" that results from applying that recipe to a specific dataset. The same algorithm can produce very different models depending on the data it is trained on. Understanding this distinction is foundational vocabulary for the entire exam.

Everyday Analogy: A recipe for chocolate chip cookies is the algorithm; the actual batch of cookies you bake using your specific ingredients is the model.

Why It Matters for the Exam: This is one of the very first terms defined in Task Statement 1.1. The exam expects you to clearly distinguish "algorithm" from "model" and "training."

Example: A bank uses a classification algorithm to build a model that predicts whether a loan application is likely to default.

Common Misunderstanding: Learners frequently use "algorithm" and "model" interchangeably. The algorithm is the general method; the model is the specific, trained result of applying that method to data.

Related AWS Service (if applicable): Amazon SageMaker AI (provides access to many built-in algorithms for training custom models).

Algorithmic Bias

Domain(s): 4 **Priority:** Medium **Category:** Responsible AI

Simple Definition: Algorithmic bias is unfair or skewed behavior that comes from the design or logic of the algorithm itself, rather than only from the training data.

Detailed Explanation: While AI bias broadly covers any unfair outcome, algorithmic bias specifically points to how an algorithm's design choices, such as which features it weighs most heavily or

how it optimizes for accuracy, can introduce or amplify unfairness even with reasonably balanced data. For example, an algorithm optimized purely for overall accuracy may systematically perform worse for a smaller subgroup within the data because correctly handling that subgroup contributes less to the overall accuracy score. This distinguishes algorithmic bias from purely data-driven bias, though in practice the two often combine and compound each other.

Everyday Analogy: Imagine a grading rule that gives bonus points for speed on a test. Even if every student receives the exact same questions, the rule itself disadvantages students who think carefully rather than quickly, regardless of whether their answers were correct.

Why It Matters for the Exam: Expect this as a supporting concept under Domain 4's broader bias and fairness objectives, distinguishing the source of unfairness (algorithm design choices) from dataset-driven bias.

Example: A model designed to maximize overall prediction accuracy might consistently under-predict outcomes for a minority subgroup because that subgroup represents a smaller share of the overall optimization target.

Common Misunderstanding: Learners often assume all bias problems can be fixed simply by adding more or better data. Algorithmic bias requires also examining how the algorithm itself defines and optimizes for "correct" outcomes.

Related AWS Service (if applicable): Amazon SageMaker Clarify.

Amazon Bedrock Agents

Domain(s): 3 **Priority:** High **Category:** AWS Service

Simple Definition: Amazon Bedrock Agents is a feature of Amazon Bedrock that lets you build AI agents that can complete multi-step tasks by reasoning, calling APIs, and retrieving information automatically.

Detailed Explanation: Amazon Bedrock Agents allows an organization to define a task, connect it to relevant data sources and APIs, and let a foundation model orchestrate the steps needed to complete that task with minimal custom orchestration code. It is closely related to, but distinct from, Amazon Bedrock AgentCore: Bedrock Agents is the agent-building capability within the Bedrock console and API, while AgentCore is the broader managed service for deploying, securing, and scaling production-grade agents, including those built with open-source frameworks like Strands Agents. Bedrock Agents can integrate with Amazon Bedrock Knowledge Bases to ground its actions in company data.

Everyday Analogy: It is like hiring a project coordinator who already knows which departments to call, which forms to fill out, and which approvals to chase down, so you only need to describe the end goal rather than every individual step.

Why It Matters for the Exam: Domain 3 expects you to know that AI agents have practical business applications and that Amazon Bedrock is the primary AWS service supporting agent-building and orchestration for FM-based applications.

Example: An HR team uses Amazon Bedrock Agents to build an assistant that can look up an employee's PTO balance, check policy rules, and submit a leave request automatically.

Common Misunderstanding: Learners often blur Bedrock Agents and Bedrock AgentCore together. Keep in mind: Bedrock Agents helps you build the agent's logic; AgentCore helps you deploy, secure, and scale agents (including ones built elsewhere) in production.

Related AWS Service (if applicable): Amazon Bedrock; Amazon Bedrock AgentCore; Amazon Bedrock Knowledge Bases.

Amazon Bedrock Guardrails

Domain(s): 4, 5 **Priority:** High **Category:** AWS Service

Simple Definition: Amazon Bedrock Guardrails is a feature that lets organizations set safety limits on what a foundation model is allowed to say or do, such as blocking harmful content or restricting certain topics.

Detailed Explanation: Guardrails can filter both the input a user sends to a model and the output the model generates, screening for things like harmful content, personally identifiable information (PII), specific denied topics, and factual grounding against source documents. This makes Guardrails a key tool for enforcing responsible AI principles (Domain 4) and for meeting security and content-safety requirements (Domain 5). Guardrails can be applied consistently across multiple foundation models and applications, which helps an organization maintain a single safety standard rather than configuring safety separately for every individual application.

Everyday Analogy: Think of Guardrails as the bumper lanes at a bowling alley: the model can still move and respond freely within the lane, but it cannot go where it should not go.

Why It Matters for the Exam: This is one of the most heavily tested AWS service names across both Domain 4 (responsible AI tools) and Domain 5 (AI system security features). Expect direct questions asking you to match "filtering harmful content" or "restricting topics" to this service.

Example: A financial services chatbot uses Bedrock Guardrails to block any response that resembles personalized investment advice, since that could create regulatory risk.

Common Misunderstanding: Learners sometimes think Guardrails only blocks profanity or obviously offensive language. In reality, it can also enforce topic restrictions, detect PII, and check whether a response is properly grounded in approved source material.

Related AWS Service (if applicable): Amazon Bedrock; Amazon Bedrock AgentCore Policy.

Amazon Bedrock Knowledge Bases

Domain(s): 3 **Priority:** High **Category:** AWS Service

Simple Definition: Amazon Bedrock Knowledge Bases is a managed feature that connects a foundation model to an organization's own documents so the model can retrieve relevant information before answering a question.

Detailed Explanation: Knowledge Bases is the primary AWS implementation of Retrieval Augmented Generation (RAG). It automatically handles the underlying steps of RAG: ingesting documents, chunking them into manageable pieces, converting those chunks into vector embeddings,

storing them in a vector database (such as Amazon OpenSearch Service or Amazon Aurora), and retrieving the most relevant chunks at query time to ground the model's response. This reduces hallucinations and allows organizations to use proprietary data without retraining or fine-tuning the underlying foundation model.

Everyday Analogy: It is like giving an extremely knowledgeable new employee instant access to your company's internal wiki the moment they start, rather than expecting them to memorize the entire wiki before they can answer a single question.

Why It Matters for the Exam: Domain 3, Task Statement 3.1 explicitly names Amazon Bedrock Knowledge Bases as the example service for RAG business applications. This is a near-certain exam topic.

Example: A company uses Bedrock Knowledge Bases to let an internal chatbot answer employee questions by pulling accurate, current information from the company's HR policy documents.

Common Misunderstanding: Learners sometimes think Knowledge Bases changes or retrains the underlying foundation model. It does not; it retrieves and supplies relevant context to the model at the moment of the request, leaving the model itself unchanged.

Related AWS Service (if applicable): Amazon Bedrock; Amazon OpenSearch Service; Amazon Aurora; Amazon Neptune; Amazon RDS for PostgreSQL (pgvector).

Anomaly Detection

Domain(s): 1 **Priority:** Medium **Category:** AI/ML Concept

Simple Definition: Anomaly detection is the use of AI/ML to automatically spot data points or events that are unusual compared to the normal pattern.

Detailed Explanation: Anomaly detection models learn what "normal" looks like from historical data and then flag anything that deviates significantly from that pattern. It is widely used for fraud detection, equipment failure prediction, network security monitoring, and quality control. Anomaly detection can use supervised learning (if labeled examples of anomalies exist) or unsupervised learning (if the model must discover unusual patterns without being told what an anomaly looks like in advance).

Everyday Analogy: It is like a parent who knows their child's normal routine so well that they instantly notice when something is off, like an unusually quiet evening or an unexpected change in behavior, without anyone telling them what to look for.

Why It Matters for the Exam: Task Statement 1.2 lists fraud detection as a key real-world AI application category, and anomaly detection is the underlying technique most associated with it.

Example: A credit card company uses anomaly detection to flag a transaction made in a foreign country minutes after a transaction was made locally, since that pattern deviates from the customer's normal spending behavior.

Common Misunderstanding: Learners sometimes assume anomaly detection always requires labeled "fraud" or "not fraud" examples. Unsupervised anomaly detection can identify never-before-seen patterns without any prior labeling.

Related AWS Service (if applicable): Amazon SageMaker AI.

Artificial Intelligence (AI)

Domain(s): 1 **Priority:** High **Category:** AI/ML Concept

Simple Definition: Artificial intelligence is the broad field of building computer systems that can perform tasks normally requiring human-like thinking, such as recognizing speech, making decisions, or solving problems.

Detailed Explanation: AI is the umbrella term that contains machine learning, deep learning, and generative AI as more specific subfields and techniques. AI does not always require "learning" from data; some early AI systems were simply pre-programmed rule-based systems. Today, most practical AI capability comes from machine learning approaches, where systems learn patterns from data rather than following only hand-written rules. Understanding AI as the broadest category, with ML and GenAI nested inside it, is essential vocabulary for the entire exam.

Everyday Analogy: Think of AI as the entire category of "vehicles." Machine learning is like "cars" - a major type within that category - and generative AI is like a specific brand or model of car within that subtype.

Why It Matters for the Exam: Task Statement 1.1 explicitly requires you to define AI and describe how it relates to ML, GenAI, deep learning, and agentic AI. Expect questions that test whether you understand the nested relationship between these terms.

Example: A rules-based chess program that follows pre-written logic to choose moves is a form of AI, even though it does not "learn" from new data the way a modern machine learning model does.

Common Misunderstanding: Many learners assume AI and machine learning are the same thing. ML is one major approach to building AI; AI is the overall goal or field, achievable through multiple approaches.

Related AWS Service (if applicable): Amazon Bedrock; Amazon SageMaker AI (the two primary AWS platforms spanning the AI/ML ecosystem).

Audit Trail

Domain(s): 5 **Priority:** High **Category:** Security/Governance

Simple Definition: An audit trail is a recorded history of actions and events that shows who did what, and when, within a system.

Detailed Explanation: For AI systems specifically, an audit trail might capture which user submitted a prompt, what response a model generated, which data sources were retrieved during a RAG query, or which configuration changes were made to a deployed model. Audit trails are essential for security investigations, regulatory compliance, and accountability, since they let an organization reconstruct exactly what happened if something goes wrong or needs to be reviewed. On AWS, audit trails for account and resource activity are primarily generated by AWS CloudTrail.

Everyday Analogy: It is like a building's security camera footage and sign-in log combined: if something goes missing or something unusual happens, you can look back and see exactly who entered which room and when.

Why It Matters for the Exam: Domain 5, Task Statement 5.1 explicitly lists "audit trail and logging requirements for AI interactions" as a security and privacy consideration you must be able to describe.

Example: A healthcare AI application logs every query made to a patient knowledge base, along with the user's identity and timestamp, to support compliance audits.

Common Misunderstanding: Learners sometimes confuse an audit trail (a record of what happened) with a compliance report (an assessment of whether what happened meets a standard). The audit trail is the raw evidence; the compliance assessment is built using that evidence.

Related AWS Service (if applicable): AWS CloudTrail.

AWS KMS (AWS Key Management Service)

Domain(s): 5 **Priority:** Medium **Category:** AWS Service

Simple Definition: AWS KMS is a managed service for creating and controlling the encryption keys used to protect data at rest and in transit.

Detailed Explanation: AWS KMS lets organizations create, manage, and control access to cryptographic keys, including customer managed keys (CMKs), which give the organization more granular control than AWS-managed default keys. For AI workloads, KMS is commonly used to encrypt training data stored in Amazon S3, model artifacts, and any sensitive information processed by AI applications. KMS integrates with many other AWS services so that encryption can be applied consistently without each service needing to manage its own separate key infrastructure.

Everyday Analogy: Think of KMS as a highly secure central key-cutting and key-management office for an entire office building: rather than every department managing its own locks and spare keys informally, one trusted office issues, tracks, and revokes every key used anywhere in the building.

Why It Matters for the Exam: KMS appears in Domain 5 as a core encryption-related service. Know that it manages keys (not application secrets like passwords, which is the role of AWS Secrets Manager).

Example: A company encrypts the documents stored in its Amazon S3-based RAG knowledge base using a customer managed key in AWS KMS, ensuring only specifically authorized roles can decrypt the content.

Common Misunderstanding: Learners often confuse AWS KMS with AWS Secrets Manager. KMS manages the cryptographic keys used to lock and unlock data; Secrets Manager stores and rotates application credentials like passwords and API tokens.

Related AWS Service (if applicable): AWS KMS itself; closely related to AWS Secrets Manager.

AWS PrivateLink

Domain(s): 5 **Priority:** Medium **Category:** AWS Service

Simple Definition: AWS PrivateLink lets you privately connect your AWS resources to AWS services without your traffic ever traveling across the public internet.

Detailed Explanation: For AI workloads that process sensitive data, PrivateLink allows an application running inside a customer's private network (a VPC) to call services like Amazon Bedrock or Amazon SageMaker AI without exposing that traffic to the public internet. This reduces the attack surface and helps meet stricter data residency and security requirements. PrivateLink is a specific feature that works alongside a VPC; the VPC is the overall private network environment, while PrivateLink is the private connectivity mechanism to AWS services from within it.

Everyday Analogy: Imagine a private, direct tunnel between two specific buildings rather than requiring everyone to walk out onto a public street to get from one building to the other.

Why It Matters for the Exam: Domain 5, Task Statement 5.1 explicitly names AWS PrivateLink as a service used to secure AI systems. Expect it as a service-matching answer choice for "private connectivity" scenarios.

Example: A healthcare company uses AWS PrivateLink so that its application can call Amazon Bedrock to process patient-related prompts without that traffic ever crossing the public internet.

Common Misunderstanding: Learners sometimes think PrivateLink and VPC are the same thing. A VPC is the broader isolated network environment; PrivateLink is a specific feature that provides private connections from that VPC to AWS services.

Related AWS Service (if applicable): Amazon VPC.

B

Batch Size

Domain(s): 1 **Priority:** Low **Category:** AI/ML Concept

Simple Definition: Batch size is the number of training examples a model looks at together before it updates what it has learned.

Detailed Explanation: During training, data is rarely fed into a model one example at a time or all at once; instead it is split into batches. The batch size affects training speed, memory usage, and how smoothly the model's learning progresses. A larger batch size generally means more stable but slower-to-compute updates, while a smaller batch size means faster but noisier updates. This is a hyperparameter that data scientists configure, and on the AIF-C01 exam it is only tested at a conceptual recognition level, not a calculation level.

Everyday Analogy: It is like grading a stack of 100 essays. You could grade and adjust your grading scale after every single essay (small batch), or wait and review 20 essays at a time before adjusting your approach (larger batch).

Why It Matters for the Exam: Batch size is a low-priority, definitional-only term. You should be able to recognize it as a hyperparameter related to model training, but you will not be asked to calculate or tune it.

Example: A data science team trains an image classification model using a batch size of 32, meaning the model processes 32 images before each internal update.

Common Misunderstanding: Learners sometimes confuse batch size (a training configuration) with batch inferencing (a way of running predictions on large groups of data after a model is already trained). These are related concepts but apply to different stages of the ML lifecycle.

Related AWS Service (if applicable): Amazon SageMaker AI.

BERTScore

Domain(s): 3 **Priority:** Medium **Category:** Foundation Model Application

Simple Definition: BERTScore is an automated metric that measures how similar in meaning a model's generated text is to a reference text, even if the exact words are different.

Detailed Explanation: Unlike older metrics such as ROUGE and BLEU, which mostly compare exact word overlap, BERTScore uses a language model's understanding of meaning to compare generated text against reference text at a semantic level. This makes it more forgiving of paraphrasing and more sensitive to genuine differences in meaning. It is commonly used to evaluate summarization, translation, and other generative tasks where multiple correct phrasings of the same idea are possible. BERTScore sits alongside ROUGE, BLEU, and LLM-as-a-judge as one of the automated evaluation metrics covered in Domain 3.

Everyday Analogy: It is like a teacher who grades an essay answer based on whether the student captured the right idea, rather than only checking whether the student used the exact same words as the textbook.

Why It Matters for the Exam: Task Statement 3.4 explicitly lists BERTScore alongside ROUGE, BLEU, and LLM-as-a-judge as a relevant FM performance metric. Know that it measures semantic similarity rather than exact word overlap.

Example: A company comparing two different summarization models uses BERTScore to confirm that a more concise summary still captured the same core meaning as a longer reference summary.

Common Misunderstanding: Learners sometimes think BERTScore, ROUGE, and BLEU are interchangeable. ROUGE and BLEU mostly reward matching exact words and phrases; BERTScore can recognize correct meaning even with different wording.

Related AWS Service (if applicable): Amazon Bedrock Model Evaluation.

BLEU Score

Domain(s): 3 **Priority:** High **Category:** Foundation Model Application

Simple Definition: BLEU is an automated score that measures how closely a model's translated or generated text matches a human reference translation, primarily by comparing matching words and phrases.

Detailed Explanation: BLEU (Bilingual Evaluation Understudy) was originally designed to evaluate machine translation quality. It compares short sequences of words (called n-grams) in the

generated output against one or more human-written reference translations and produces a score reflecting the degree of overlap. A higher BLEU score generally indicates closer alignment with the reference text, although it can penalize correct translations that simply use different but equally valid wording. On the exam, BLEU is most strongly associated with translation quality, while ROUGE is most strongly associated with summarization quality.

Everyday Analogy: Imagine grading a translated sentence by counting how many of the exact same word groupings appear in both the student's translation and the answer key, rather than judging whether the overall meaning came through correctly.

Why It Matters for the Exam: Task Statement 3.4 names BLEU specifically, and the exam frequently tests the pairing: ROUGE = summarization, BLEU = translation. Memorize this pairing.

Example: A company evaluating an AWS Translate-style translation feature uses BLEU scores to compare output quality across several candidate models before choosing one for production.

Common Misunderstanding: Learners often mix up ROUGE and BLEU. The simplest memory aid: BLEU sounds like it relates to "bilingual," matching its translation use case, while ROUGE is associated with summarization (Gisting Evaluation).

Related AWS Service (if applicable): Amazon Bedrock Model Evaluation.

C

Chain-of-Thought Prompting

Domain(s): 3 **Priority:** High **Category:** Foundation Model Application

Simple Definition: Chain-of-thought prompting is a technique where you ask a model to reason through a problem step by step before giving its final answer.

Detailed Explanation: Rather than asking a model to jump directly to a conclusion, chain-of-thought prompting explicitly instructs the model to "think out loud" through intermediate reasoning steps, such as breaking a math problem into smaller calculations or listing the logical steps behind a recommendation. This technique often improves accuracy on tasks that require multi-step logic, such as math problems, planning tasks, or complex business reasoning, because it gives the model the opportunity to catch its own errors along the way. It is one of several named prompt engineering techniques on the exam, alongside zero-shot, single-shot, and few-shot prompting.

Everyday Analogy: It is like asking a student to "show their work" on a math test rather than just writing down a final answer; showing the steps makes errors easier to catch and often improves the final result.

Why It Matters for the Exam: Task Statement 3.2 explicitly names chain-of-thought prompting as a core prompt engineering technique you must be able to define and differentiate from zero-shot, single-shot, and few-shot prompting.

Example: A prompt that says "Think through this step by step before giving your final recommendation" to help a model correctly solve a multi-part business calculation is using chain-of-thought prompting.

Common Misunderstanding: Learners sometimes think chain-of-thought prompting requires providing examples. It does not require examples at all; it simply instructs the model to reason through steps, and can be combined with zero-shot or few-shot approaches.

Related AWS Service (if applicable): Amazon Bedrock; Amazon Bedrock Prompt Management.

Chunking

Domain(s): 2, 3 **Priority:** High **Category:** GenAI Concept

Simple Definition: Chunking is the process of breaking a large document into smaller pieces so that a model or retrieval system can process and search them more effectively.

Detailed Explanation: In a RAG system, documents are too large to feed entirely into a model's context window, and storing an entire document as a single unit in a vector database would make retrieval imprecise. Chunking solves this by splitting documents into smaller, more manageable sections, each of which is converted into its own embedding for storage in a vector database. The way chunks are sized and split matters: chunks that are too large can include irrelevant information, while chunks that are too small can lose important context. Effective chunking strategy is a key design consideration when building a Bedrock Knowledge Bases-powered RAG application.

Everyday Analogy: It is like breaking a long reference book into individually labeled index cards by topic, instead of forcing someone to read the entire book every time they need one fact.

Why It Matters for the Exam: Chunking is named directly in Task Statement 2.1 as a foundational GenAI concept and underpins how RAG and vector databases work in Domain 3.

Example: A company building a RAG application splits its 200-page employee handbook into approximately 500-word sections before generating embeddings for its knowledge base.

Common Misunderstanding: Learners sometimes think chunking happens at query time. In a typical RAG pipeline, chunking happens when documents are first ingested into the knowledge base, not when a user submits a question.

Related AWS Service (if applicable): Amazon Bedrock Knowledge Bases.

Classification

Domain(s): 1 **Priority:** High **Category:** AI/ML Concept

Simple Definition: Classification is a machine learning technique that assigns an input to one of a set of predefined categories.

Detailed Explanation: Classification models are trained using labeled data so they learn to recognize the patterns associated with each category. Classification can be binary (two possible categories, such as "fraud" or "not fraud") or multi-class (more than two categories, such as classifying a support ticket as "billing," "technical," or "general"). It is one of the three core technique types tested in Task Statement 1.2, alongside regression and clustering, and learners must be able to recognize which business scenarios call for classification versus the other two.

Everyday Analogy: It is like a mail sorter who places each piece of mail into one of several labeled bins - local, regional, or international - based on patterns they have learned to recognize from the address.

Why It Matters for the Exam: The exam frequently presents a business scenario and asks you to select the correct technique. Recognize classification scenarios: anything where the output is a category label (fraud/not fraud, spam/not spam, type of customer complaint).

Example: A bank uses a classification model to label each loan application as "approve," "deny," or "manual review."

Common Misunderstanding: Learners sometimes confuse classification (assigning a category) with regression (predicting a continuous number). If the answer to a business question is "which group?" it is classification; if it is "how much?" it is regression.

Related AWS Service (if applicable): Amazon SageMaker AI; Amazon Comprehend (for text classification tasks like sentiment).

Clustering

Domain(s): 1 **Priority:** High **Category:** AI/ML Concept

Simple Definition: Clustering is a machine learning technique that groups similar items together based on patterns in the data, without being told in advance what the groups should be.

Detailed Explanation: Clustering is a form of unsupervised learning because it does not require labeled data. Instead, the algorithm examines the characteristics of each data point and groups together those that are most similar to each other, while keeping dissimilar items in separate groups. Clustering is often used for customer segmentation, anomaly detection, and discovering hidden structure in large unlabeled datasets. It is one of the three named technique types in Task Statement 1.2, alongside classification and regression.

Everyday Analogy: It is like sorting a giant pile of mixed buttons by similarity (size, color, shape) without anyone telling you the categories in advance; you simply group the ones that look alike together.

Why It Matters for the Exam: Recognize clustering scenarios: anything where the goal is "discover natural groupings" without predefined categories, such as customer segmentation.

Example: A retailer uses clustering to discover that its customers naturally fall into a few distinct shopping behavior groups, without having predefined those groups in advance.

Common Misunderstanding: Learners sometimes confuse clustering with classification. Classification assigns items to categories that are already known and labeled in advance; clustering discovers groupings that were not predefined.

Related AWS Service (if applicable): Amazon SageMaker AI.

Compliance

Domain(s): 5 **Priority:** Medium **Category:** Security/Governance

Simple Definition: Compliance means meeting the legal, regulatory, or industry standards that apply to how an organization collects, uses, and protects data and AI systems.

Detailed Explanation: Compliance requirements for AI systems can come from general data protection laws (such as GDPR or HIPAA), industry-specific regulations, or internal corporate policy. Demonstrating compliance typically requires documented evidence, audit trails, and governance processes, which is why AWS provides services like AWS Artifact (for accessing AWS's own compliance reports) and AWS Audit Manager (for building an organization's own compliance evidence). Compliance is closely tied to the shared responsibility model, since AWS is responsible for the compliance of the underlying cloud infrastructure, while the customer is responsible for how they configure and use that infrastructure.

Everyday Analogy: It is like a restaurant passing a health inspection: the building owner ensures the facility itself meets code, but the restaurant operator is still responsible for how they handle food and serve customers day to day.

Why It Matters for the Exam: Domain 5, Task Statement 5.2 tests your ability to recognize the AWS services that support governance and compliance, and how regulatory frameworks like GDPR and HIPAA apply to AI workloads.

Example: A healthcare company deploying an AI-powered patient chatbot must ensure that the way it stores and processes patient data complies with HIPAA requirements.

Common Misunderstanding: Learners sometimes think AWS compliance certifications automatically make any application built on AWS compliant. Compliance for the application itself still depends on how the customer configures, secures, and governs their specific use of AWS services.

Related AWS Service (if applicable): AWS Artifact; AWS Audit Manager.

Computer Vision

Domain(s): 1 **Priority:** High **Category:** AI/ML Concept

Simple Definition: Computer vision is the field of AI that enables computers to interpret and understand images and video, such as identifying objects, faces, or text within a picture.

Detailed Explanation: Computer vision systems are trained on large sets of labeled images to recognize patterns such as shapes, colors, textures, and spatial relationships that correspond to specific objects or categories. Common computer vision tasks include object detection, facial analysis, scene recognition, and optical character recognition (extracting text from images). On AWS, computer vision capability is primarily delivered through Amazon Rekognition (general visual analysis) and Amazon Textract (extracting text and structured data from documents).

Everyday Analogy: It is like teaching someone who has only ever heard descriptions of fruit to instead recognize an apple just by looking at thousands of pictures of apples until they can identify one on sight.

Why It Matters for the Exam: Computer vision is named directly in Task Statement 1.1 as a basic term to define, and it appears again in Task Statement 1.2 as a real-world AI application category. Know the difference between Rekognition (visual content analysis) and Textract (text extraction from documents).

Example: A retailer uses computer vision to automatically detect when shelves are empty in security camera footage.

Common Misunderstanding: Learners often confuse Amazon Rekognition with Amazon Textract. Rekognition analyzes what is depicted in an image (objects, scenes, faces); Textract extracts written or printed text and structured data from document images.

Related AWS Service (if applicable): Amazon Rekognition; Amazon Textract.

Confusion Matrix

Domain(s): 1 **Priority:** Medium **Category:** AI/ML Concept

Simple Definition: A confusion matrix is a table that shows how many predictions a classification model got right and wrong, broken down by category.

Detailed Explanation: A confusion matrix organizes predictions into four categories for a binary classification problem: true positives (correctly predicted positive), true negatives (correctly predicted negative), false positives (incorrectly predicted positive), and false negatives (incorrectly predicted negative). Precision, recall, accuracy, and F1 score are all calculated from the values in a confusion matrix. Reviewing a confusion matrix helps a learner see exactly what kind of mistakes a model is making, which a single accuracy number cannot reveal.

Everyday Analogy: It is like a scoreboard that does not just show the final score of a game but breaks down exactly which plays succeeded and which specific plays failed, and how.

Why It Matters for the Exam: The confusion matrix is a supporting concept behind accuracy, precision, recall, and F1 score. You are unlikely to be asked to build one, but you should recognize it as the source of those metrics.

Example: A fraud detection team reviews a confusion matrix and discovers their model has a high number of false negatives, meaning it is missing a concerning number of actual fraud cases.

Common Misunderstanding: Learners sometimes think a confusion matrix is itself a single performance metric. It is actually a table of raw counts from which other metrics, like precision and recall, are calculated.

Related AWS Service (if applicable): Amazon SageMaker AI.

Content Filtering

Domain(s): 5 **Priority:** High **Category:** Security/Governance

Simple Definition: Content filtering is the practice of automatically screening AI inputs or outputs to block harmful, inappropriate, or unwanted material.

Detailed Explanation: Content filtering can be applied to both directions of an AI interaction: filtering what a user submits to a model (to block malicious prompts) and filtering what the model generates in response (to block harmful, biased, or off-topic content before it reaches the user). On AWS, content filtering for foundation model applications is primarily implemented through Amazon Bedrock Guardrails, which can detect categories like hate speech, violence, and sensitive

topics, and either block or flag them. Content filtering is closely related to output filtering and toxicity detection, all of which fall under Domain 5's security and privacy considerations.

Everyday Analogy: It is like a spam filter for email, but instead of just filtering unwanted messages, it screens both the incoming requests and the outgoing replies for anything inappropriate.

Why It Matters for the Exam: Domain 5, Task Statement 5.1 names output filtering and validation, and Domain 4 names Guardrails' content filters as a responsible AI tool. Expect overlap between these two domains on this topic.

Example: An AI customer service tool uses content filtering to automatically block any response containing offensive language before it reaches a customer.

Common Misunderstanding: Learners sometimes assume content filtering only screens what the model produces. Effective content filtering also screens what users submit, helping prevent prompt injection or jailbreaking attempts.

Related AWS Service (if applicable): Amazon Bedrock Guardrails.

Context Window

Domain(s): 2, 3 **Priority:** High **Category:** GenAI Concept

Simple Definition: The context window is the maximum amount of text, measured in tokens, that a model can consider at one time when generating a response.

Detailed Explanation: Everything a model "knows" about the current conversation, including the system prompt, conversation history, retrieved RAG content, and the user's latest question, must fit within the context window. If the combined input exceeds the model's context window limit, older or less relevant information must be dropped or summarized. Context window size is one of the selection criteria for choosing a foundation model (Task Statement 3.1), since applications that require analyzing long documents need a model with a large context window. Context engineering is the deliberate practice of managing what information is placed into this limited space.

Everyday Analogy: Think of the context window as the size of a desk: you can only have so many documents spread out and visible at once before you have to put some away to make room for new ones.

Why It Matters for the Exam: This term appears in both Domain 2 (foundational vocabulary) and Domain 3 (FM selection criteria). Know that context window size is measured in tokens and directly limits how much information a model can use at once.

Example: A legal team analyzing a 300-page contract needs to select a foundation model with a sufficiently large context window to process the entire document in a single request.

Common Misunderstanding: Learners sometimes think the context window is the model's permanent memory across all conversations. It is actually limited to the current request or session; once the conversation ends or the limit is exceeded, earlier content is not automatically retained unless the application specifically manages and resupplies it.

Related AWS Service (if applicable): Amazon Bedrock.

Continued Pre-Training

Domain(s): 2, 3 **Priority:** Medium **Category:** Foundation Model Application

Simple Definition: Continued pre-training is the process of updating an already-trained foundation model with new, additional general data, without starting the training process completely over.

Detailed Explanation: Continued pre-training (also called continuous pre-training) keeps a foundation model up to date with more recent or domain-relevant general knowledge, which differs from fine-tuning in that it focuses on broad knowledge updates rather than teaching the model a specific task or response format. It is a more cost-effective alternative to training an entirely new foundation model from scratch and helps address the fact that an FM's "knowledge" is otherwise frozen at the time its original training data was collected. This is one of the customization approaches named in Domain 3's discussion of FM training stages.

Everyday Analogy: It is like sending an experienced employee back for periodic refresher training on new industry developments, rather than firing them and hiring and training someone completely new from day one.

Why It Matters for the Exam: Task Statement 3.3 explicitly lists continuous pre-training as one of the key stages and methods of FM training and fine-tuning. Know that it updates general knowledge rather than teaching a specific task.

Example: A foundation model provider periodically applies continued pre-training using newly published documents so the model's general knowledge does not become outdated.

Common Misunderstanding: Learners often confuse continued pre-training with fine-tuning. Continued pre-training expands general knowledge; fine-tuning teaches the model to perform a specific task or follow a specific style or format.

Related AWS Service (if applicable): Amazon Bedrock; Amazon SageMaker AI.

Controllability

Domain(s): 4 **Priority:** Medium **Category:** Responsible AI

Simple Definition: Controllability is the degree to which humans can monitor, adjust, override, or stop an AI system's behavior.

Detailed Explanation: A controllable AI system gives operators the ability to intervene when something goes wrong, such as halting an autonomous agent's actions, adjusting guardrail settings, or escalating a decision to a human reviewer. Controllability becomes especially important for agentic AI systems, since an autonomous agent that takes multi-step actions without sufficient human oversight could cause cascading problems if it misunderstands a goal. Controllability connects closely to human-in-the-loop and human oversight practices, which provide concrete mechanisms for maintaining control over AI decisions.

Everyday Analogy: It is like a car with both a self-driving feature and a steering wheel and brake pedal that the driver can use to take back control at any moment.

Why It Matters for the Exam: While not always given top billing in the official task statements, controllability supports the broader responsible AI conversation in Domain 4 around safety and human oversight of AI systems, especially agentic ones.

Example: A company deploying an autonomous AI agent that processes refunds builds in a control mechanism that lets a human supervisor pause the agent's actions at any time.

Common Misunderstanding: Learners sometimes assume controllability only matters for traditional ML models. It is especially critical for agentic AI systems, which can take real-world actions rather than just generate text.

Related AWS Service (if applicable): Amazon Bedrock AgentCore Policy; Amazon Augmented AI (A2I).

Cross-Validation

Domain(s): 1 **Priority:** Low **Category:** AI/ML Concept

Simple Definition: Cross-validation is a technique for testing how well a model will likely perform on new data by repeatedly splitting the training data into different training and testing portions.

Detailed Explanation: Instead of testing a model only once on a single held-out portion of data, cross-validation repeats the training and testing process multiple times using different splits of the same dataset, then averages the results. This produces a more reliable estimate of how the model will perform on data it has never seen, and helps detect problems like overfitting. Cross-validation is a data science technique used during model development, and the AIF-C01 exam tests it only at a conceptual, recognition level.

Everyday Analogy: It is like practicing for a test using several different practice exams instead of just one, to get a more reliable sense of how ready you actually are, rather than getting lucky or unlucky on a single practice round.

Why It Matters for the Exam: This is a low-priority, definitional term. You should recognize what cross-validation is for, but you will not be asked to perform a cross-validation calculation.

Example: A data science team uses 5-fold cross-validation to confirm that their model's accuracy is consistent across different subsets of their training data, not just a lucky result from one split.

Common Misunderstanding: Learners sometimes think cross-validation eliminates the need for a separate, final test dataset. Cross-validation helps during model development, but a final, untouched test set is still valuable for confirming real-world performance.

Related AWS Service (if applicable): Amazon SageMaker AI.

Customer Managed Key (CMK)

Domain(s): 5 **Priority:** Low **Category:** Security/Governance

Simple Definition: A customer managed key is an encryption key in AWS KMS that the customer creates and controls directly, rather than relying on an AWS-managed default key.

Detailed Explanation: A CMK gives organizations more granular control over key policies, rotation schedules, and access permissions than an AWS-managed key. This matters for AI workloads handling sensitive data, since organizations may need to demonstrate to auditors or regulators that they specifically control who can decrypt that data and under what conditions. Using a CMK is a common best practice for meeting stricter compliance requirements around data at rest, particularly for training data, model artifacts, or knowledge base content stored in Amazon S3.

Everyday Analogy: It is the difference between using a master key provided by your apartment building (a default AWS-managed key) versus installing and controlling your own personal lock and key for your unit (a CMK).

Why It Matters for the Exam: This is a supporting, low-priority detail under the broader AWS KMS and encryption topics in Domain 5. You are unlikely to see deep technical questions on CMKs specifically.

Example: A financial services company uses a customer managed key in AWS KMS to encrypt sensitive training data, giving its own security team direct control over key rotation and access policies.

Common Misunderstanding: Learners sometimes think a CMK is a different encryption technology altogether. It is simply a type of key within AWS KMS that the customer, rather than AWS, directly manages.

Related AWS Service (if applicable): AWS KMS.

D

Data Classification

Domain(s): 5 **Priority:** Medium **Category:** Security/Governance

Simple Definition: Data classification is the practice of labeling data according to its sensitivity level, such as public, internal, confidential, or restricted.

Detailed Explanation: Classifying data helps an organization apply the right level of protection to the right data: highly sensitive data (such as health records or financial information) needs stronger encryption, stricter access controls, and more careful handling than public marketing material. For AI systems, data classification matters both for training data and for data retrieved during RAG, since a knowledge base might contain a mix of public and highly sensitive documents that require different access permissions. Data classification supports broader data governance efforts and is closely tied to tools like Amazon Macie, which automatically discovers and classifies sensitive data stored in Amazon S3.

Everyday Analogy: It is like color-coding folders in a filing cabinet: green for anyone to read, yellow for internal staff only, and red for only a few authorized people, so everyone instantly knows how carefully to handle each folder.

Why It Matters for the Exam: Data classification supports the data governance objectives in Domain 5, Task Statement 5.2, and connects directly to Amazon Macie's automated sensitive data discovery capability.

Example: A company classifies its customer support transcripts as "confidential" and restricts which AI applications and personnel can access them.

Common Misunderstanding: Learners sometimes think data classification is a one-time, manual process. In practice, AWS services like Amazon Macie can continuously and automatically scan and classify data as it changes.

Related AWS Service (if applicable): Amazon Macie.

Data Drift

Domain(s): 1, 4 **Priority:** Medium **Category:** AI/ML Concept

Simple Definition: Data drift happens when the real-world data a deployed model sees in production starts to look statistically different from the data it was originally trained on.

Detailed Explanation: Over time, customer behavior, market conditions, or external factors can change, causing the characteristics of incoming data to shift away from what the model learned during training. Even if the model itself has not changed, its performance can degrade simply because the world it is being applied to has changed. Detecting data drift early is a core part of MLOps and ongoing model monitoring, since it signals that a model may need retraining with newer data to stay accurate and fair.

Everyday Analogy: It is like a tour guide who memorized a city's street layout years ago, but the city has since added new roads and closed old ones; the guide's old knowledge is no longer an accurate reflection of the current streets.

Why It Matters for the Exam: Data drift supports the MLOps concepts in Task Statement 1.3 (model monitoring, model re-training) and the responsible AI monitoring tools in Domain 4.

Example: A retail demand-forecasting model trained on pre-pandemic shopping patterns experiences significant data drift once customer purchasing habits shift dramatically.

Common Misunderstanding: Learners sometimes confuse data drift with model drift. Data drift refers to changes in the incoming data itself; model drift refers to the resulting decline in the model's predictive performance, which is often caused by data drift.

Related AWS Service (if applicable): Amazon SageMaker Model Monitor.

Data Lineage

Domain(s): 5 **Priority:** High **Category:** Security/Governance

Simple Definition: Data lineage is the documented history of where a piece of data came from and how it has been transformed or moved over time.

Detailed Explanation: For AI systems, data lineage tracks the full journey of training data, from its original source through any cleaning, transformation, or combination steps, all the way to its use in training or fine-tuning a model. This is essential for compliance, since organizations often need to prove that the data used to build an AI system was legally obtained and properly licensed, and for troubleshooting, since lineage helps identify where a data quality issue may have originated.

Amazon SageMaker Model Cards can help document data lineage alongside other model provenance information.

Everyday Analogy: It is like a complete shipping manifest for a piece of furniture: where the wood was harvested, which factory assembled it, which warehouse stored it, and which truck delivered it to your door.

Why It Matters for the Exam: Domain 5, Task Statement 5.1 explicitly names data lineage as part of source citation and documenting data origins, alongside data cataloging and Model Cards.

Example: A company maintains data lineage records showing that the training data for their customer service model came only from properly licensed, internally collected support transcripts.

Common Misunderstanding: Learners sometimes think data lineage and data cataloging are the same thing. Data lineage tracks the history and transformation of data over time; data cataloging maintains a structured inventory of what datasets exist and where they are.

Related AWS Service (if applicable): Amazon SageMaker Model Cards.

Data Masking

Domain(s): 5 **Priority:** Medium **Category:** Security/Governance

Simple Definition: Data masking is a technique that hides or replaces sensitive information in a dataset so it can be used or viewed without exposing the real underlying values.

Detailed Explanation: Data masking might replace real names, account numbers, or other personally identifiable information with realistic-looking but fake substitute values. This allows developers, testers, or even AI training pipelines to work with data that retains the structure and statistical properties of real data, without exposing actual sensitive details. Data masking is one of the privacy-enhancing technologies named in Domain 5, alongside techniques like differential privacy, and it supports the broader goal of secure data engineering for AI systems.

Everyday Analogy: It is like redacting a sensitive legal document with black marker strips over personal details before sharing it with someone who does not need to see the original information.

Why It Matters for the Exam: Task Statement 5.1 names privacy-enhancing technologies (including data masking) as part of secure data engineering best practices.

Example: Before letting a development team test a new AI chatbot, a company replaces all real customer names and account numbers in the test dataset with fictional placeholder values.

Common Misunderstanding: Learners sometimes think data masking and encryption are the same protective measure. Encryption makes data unreadable without a key but preserves the original value once decrypted; masking permanently replaces the real value with a different one for non-production use.

Related AWS Service (if applicable): Amazon Macie (for identifying sensitive data that may need masking).

Data Minimization

Domain(s): 4 **Priority:** Medium **Category:** Responsible AI

Simple Definition: Data minimization is the principle of collecting and retaining only the data that is actually necessary for a specific purpose, and nothing more.

Detailed Explanation: Applying data minimization to AI systems means deliberately limiting what personal or sensitive information is gathered for training or operating a model, reducing the risk of privacy violations, legal liability, and data breaches. This principle is central to many privacy regulations, including GDPR, which legally requires organizations to justify why they are collecting specific categories of personal data. For responsible AI, data minimization supports both privacy protection and risk reduction, since data that was never collected cannot later be leaked or misused.

Everyday Analogy: It is like packing only what you actually need for a weekend trip rather than bringing your entire closet; less to carry means less that can be lost, damaged, or stolen along the way.

Why It Matters for the Exam: Data minimization supports the responsible AI dataset characteristics discussed in Domain 4 and connects to privacy regulations referenced in Domain 5.

Example: A company building a customer service AI assistant collects only the information needed to resolve a support ticket, rather than also gathering unrelated personal details.

Common Misunderstanding: Learners sometimes think data minimization only applies to AI training data. It also applies to ongoing data collected from users during real-time interactions with deployed AI applications.

Related AWS Service (if applicable): Amazon Macie.

Data Preprocessing

Domain(s): 1 **Priority:** Medium **Category:** AI/ML Concept

Simple Definition: Data preprocessing is the work of cleaning and preparing raw data so it is in a usable, consistent format before it is used to train a model.

Detailed Explanation: Raw data collected from the real world is often messy: it can contain missing values, duplicate entries, inconsistent formatting, or irrelevant information. Data preprocessing addresses these issues through steps like removing duplicates, filling in or removing missing values, standardizing formats, and normalizing numerical ranges. This stage typically happens early in the AI/ML pipeline, before feature engineering and model training, and the quality of preprocessing directly affects how well a model can learn meaningful patterns.

Everyday Analogy: It is like washing, peeling, and chopping vegetables before cooking; the meal can only turn out as good as the preparation that went into the raw ingredients.

Why It Matters for the Exam: Data preprocessing is named as a stage in the AI/ML pipeline in Task Statement 1.3. Know its position in the pipeline: after data collection, before feature engineering and model training.

Example: A data science team removes duplicate customer records and standardizes inconsistent date formats before training a churn prediction model.

Common Misunderstanding: Learners sometimes use "data preprocessing" and "feature engineering" interchangeably. Preprocessing cleans and standardizes the data; feature engineering goes a step further by creating new, more informative variables from that cleaned data.

Related AWS Service (if applicable): AWS Glue; AWS Glue DataBrew.

Dataset

Domain(s): 1 **Priority:** High **Category:** AI/ML Concept

Simple Definition: A dataset is the collection of data used to train, validate, or test a machine learning model.

Detailed Explanation: A dataset can take many forms depending on the problem: tabular data (rows and columns, like a spreadsheet), text data, image data, audio data, or time-series data. Datasets are typically split into separate portions for training (teaching the model), validation (tuning the model during development), and testing (evaluating final performance on unseen data). The quality, size, and representativeness of a dataset has a major impact on how well a resulting model will perform and how fair its outputs will be.

Everyday Analogy: A dataset is like the full set of practice problems and answer keys a student studies from before taking a final exam, including the example problems used to learn the material and the separate ones used to test what they actually retained.

Why It Matters for the Exam: Datasets and their characteristics (labeled vs. unlabeled, structured vs. unstructured, etc.) are foundational vocabulary tested throughout Task Statement 1.1.

Example: A company building a fraud detection model assembles a dataset of millions of past transactions, each labeled as either "fraudulent" or "legitimate."

Common Misunderstanding: Learners sometimes use "dataset" and "data source" interchangeably. A dataset is typically a specific, organized collection prepared for a machine learning task, while a data source can refer more broadly to any origin of data, such as a database or application.

Related AWS Service (if applicable): Amazon S3 (the most common storage location for AI/ML datasets).

Deep Learning

Domain(s): 1 **Priority:** High **Category:** AI/ML Concept

Simple Definition: Deep learning is a type of machine learning that uses neural networks with many layers to automatically learn complex patterns directly from large amounts of data.

Detailed Explanation: Deep learning models, sometimes called deep neural networks, can automatically discover relevant features in raw data (such as images, audio, or text) without a human manually defining what to look for, which differs from many traditional machine learning approaches that require manual feature engineering. The "deep" in deep learning refers to having multiple stacked layers of artificial neurons, which allows the network to learn increasingly abstract

representations of the data at each layer. Deep learning is the technology underlying most modern computer vision, natural language processing, and generative AI breakthroughs, including the foundation models that power Amazon Bedrock.

Everyday Analogy: Think of deep learning as a student who, instead of being explicitly taught individual rules for recognizing handwriting, learns to recognize letters automatically just by looking at thousands and thousands of examples.

Why It Matters for the Exam: Task Statement 1.1 requires you to define deep learning and explain its relationship to AI, ML, and GenAI. Know that deep learning is a subset of machine learning, and that it underlies the foundation models used in generative AI.

Example: A deep learning model with many layers is trained to recognize tumors in medical images by learning patterns directly from thousands of labeled scans.

Common Misunderstanding: Learners sometimes treat "deep learning" and "machine learning" as synonyms. Deep learning is one specific, powerful approach within the broader field of machine learning, not a replacement term for it.

Related AWS Service (if applicable): Amazon SageMaker AI; Amazon Bedrock (which hosts foundation models built using deep learning).

Diffusion Model

Domain(s): 2 **Priority:** Medium **Category:** GenAI Concept

Simple Definition: A diffusion model is a type of generative AI model that creates new images by starting with random noise and gradually refining it into a coherent picture.

Detailed Explanation: Diffusion models are trained by learning to reverse a process of gradually adding noise to real images until they become unrecognizable; once trained, the model can run that process backward, starting from pure random noise and progressively removing it in a way that produces a realistic, coherent image. This architecture differs from the transformer-based architecture used in most text-generating large language models, although some modern multimodal systems combine both approaches. Diffusion models are the technology behind most modern AI image generation tools.

Everyday Analogy: It is like a sculptor starting with a rough, shapeless block of stone and gradually refining it, removing a little bit at a time, until a clear and recognizable figure emerges.

Why It Matters for the Exam: Task Statement 2.1 names diffusion models as part of the foundational GenAI vocabulary you must define. Know that diffusion models are most associated with image generation, distinct from transformer-based LLMs used for text.

Example: A marketing team uses a diffusion model-based image generation tool to create custom promotional artwork from a text description.

Common Misunderstanding: Learners sometimes assume all generative AI models use the same underlying architecture. Diffusion models (commonly used for images) and transformer-based models (commonly used for text) are distinct architectural approaches, even though both fall under the umbrella of generative AI.

Related AWS Service (if applicable): Amazon Bedrock (provides access to image-generating foundation models).

Disparate Impact

Domain(s): 4 **Priority:** Medium **Category:** Responsible AI

Simple Definition: Disparate impact occurs when a policy or AI system that appears neutral on its face still ends up disproportionately harming a particular group.

Detailed Explanation: Disparate impact is a legal and ethical fairness concept that focuses on outcomes rather than intent: even if an AI model was never designed with any group in mind, if its real-world results consistently and significantly disadvantage a particular demographic group, that is a disparate impact problem. This differs from disparate treatment, which involves intentionally treating groups differently. Organizations assess disparate impact by examining outcome differences across subgroups, which is exactly what subgroup analysis and tools like Amazon SageMaker Clarify are designed to support.

Everyday Analogy: Imagine a height requirement for a job that was never intended to exclude any particular group, but which, because of average height differences between groups, ends up disproportionately disqualifying one group far more than others.

Why It Matters for the Exam: Disparate impact supports the fairness and bias objectives in Domain 4, Task Statement 4.1, particularly the discussion of bias effects on demographic groups.

Example: A hiring algorithm that does not directly use gender as an input still produces disparate impact if it consistently scores resumes from one gender lower due to correlated patterns in the training data.

Common Misunderstanding: Learners often confuse disparate impact with disparate treatment. Disparate impact is about unequal outcomes regardless of intent; disparate treatment is about intentionally different treatment based on a protected characteristic.

Related AWS Service (if applicable): Amazon SageMaker Clarify.

Disparate Treatment

Domain(s): 4 **Priority:** Medium **Category:** Responsible AI

Simple Definition: Disparate treatment occurs when an AI system or organization intentionally treats people differently based on a protected characteristic, such as race, gender, or age.

Detailed Explanation: Unlike disparate impact, which focuses on unequal outcomes regardless of intent, disparate treatment specifically involves direct, intentional differential treatment. In an AI context, this could happen if a model explicitly used a protected characteristic as an input feature in a way that disadvantaged a particular group, even if that was not the deliberate goal of the system's designers. Both concepts matter for responsible AI because they represent two different legal and ethical paths to an unfair outcome, and recognizing the difference helps an organization correctly diagnose and fix a fairness problem.

Everyday Analogy: It is like a store that has an official, posted policy of charging different prices to different groups of customers based on their appearance, rather than a neutral policy that happens to produce unequal results.

Why It Matters for the Exam: This term pairs directly with disparate impact in the fairness discussion under Domain 4. The exam may test your ability to distinguish the two.

Example: A loan approval system that explicitly uses an applicant's zip code as a stand-in for race, intentionally adjusting approval odds based on that proxy, would be an example of disparate treatment.

Common Misunderstanding: Learners sometimes think disparate treatment only happens through obvious, explicit rules. It can also occur through proxy variables that are correlated with a protected characteristic, even if that characteristic is not directly used.

Related AWS Service (if applicable): Amazon SageMaker Clarify.

Dropout

Domain(s): 1 **Priority:** Low **Category:** AI/ML Concept

Simple Definition: Dropout is a training technique where a neural network randomly ignores some of its internal connections during training to help it generalize better and avoid overfitting.

Detailed Explanation: By randomly "dropping out" a portion of neurons during each training pass, the network is forced to learn more robust, redundant patterns rather than relying too heavily on any single pathway. This makes the model less likely to memorize the exact quirks of the training data (overfitting) and more likely to perform well on new, unseen data. Dropout is a technical regularization technique that data scientists configure during model development; the AIF-C01 exam tests it only at a definitional, recognition level.

Everyday Analogy: It is like a sports team that practices with a different player randomly sitting out each session, forcing the rest of the team to learn to perform well without depending too heavily on any single star player.

Why It Matters for the Exam: Dropout is a low-priority, supporting term under the broader topic of overfitting and regularization. You only need to recognize what it is, not implement it.

Example: A data science team adds a dropout rate of 20% to their neural network to help reduce overfitting on a relatively small training dataset.

Common Misunderstanding: Learners sometimes think dropout permanently removes connections from a network. Dropout only temporarily ignores certain connections during each individual training step; it does not permanently delete them.

Related AWS Service (if applicable): Amazon SageMaker AI.

E

Embedding

Domain(s): 2, 3 **Priority:** High **Category:** GenAI Concept

Simple Definition: An embedding is a way of converting text, images, or other data into a list of numbers that captures its meaning, so that a computer can compare how similar different pieces of content are.

Detailed Explanation: Embeddings represent data as a vector, a point in a mathematical space, where items with similar meaning end up positioned close together and items with different meaning end up farther apart. This numeric representation is what allows a computer to perform semantic search, finding content that means something similar even if it uses completely different words. Embeddings are a core building block of RAG systems: documents are converted into embeddings and stored in a vector database, and a user's query is also converted into an embedding so the system can find the closest, most relevant matches.

Everyday Analogy: Think of embeddings like plotting every book in a library on a giant map based on its themes, so that mystery novels naturally cluster in one area, cookbooks in another, and a book that blends both genres sits somewhere in between.

Why It Matters for the Exam: Embeddings are named directly in Task Statement 2.1 as foundational GenAI vocabulary, and they underpin the vector database and RAG concepts tested heavily in Domain 3.

Example: A customer support search tool converts both a user's question and every help article into embeddings so it can find the most semantically relevant article, even if the user's wording is completely different from the article's title.

Common Misunderstanding: Learners sometimes think embeddings store the original text itself. An embedding is a numeric representation of meaning, not a copy of the original text; the original text is typically stored separately and retrieved alongside its embedding.

Related AWS Service (if applicable): Amazon Bedrock (generates embeddings); Amazon OpenSearch Service, Amazon Aurora, Amazon Neptune, Amazon RDS for PostgreSQL (store embeddings).

Encryption at Rest

Domain(s): 5 **Priority:** High **Category:** Security/Governance

Simple Definition: Encryption at rest is the protection of stored data by scrambling it so it cannot be read without the correct decryption key, even if someone gains unauthorized access to the storage itself.

Detailed Explanation: For AI systems, encryption at rest protects training datasets, model artifacts, and knowledge base documents while they sit in storage services like Amazon S3 or in databases used for vector storage. On AWS, encryption at rest is commonly managed using AWS KMS, which controls the keys used to encrypt and decrypt the data. This is one half of a complete data protection strategy; the other half, encryption in transit, protects data as it moves between systems.

Everyday Analogy: It is like storing valuables in a safe rather than leaving them out in the open; even if someone breaks into the room, they still cannot access the contents without the combination.

Why It Matters for the Exam: Domain 5, Task Statement 5.1 explicitly lists encryption at rest and in transit as security and privacy considerations for AI systems. Expect the exam to test the distinction between the two.

Example: A company encrypts the Amazon S3 bucket containing its training data using server-side encryption, ensuring the data remains protected even if storage-level access controls were somehow bypassed.

Common Misunderstanding: Learners sometimes think encryption at rest alone is sufficient protection. Data must also be protected as it travels between systems, which requires encryption in transit as a separate, complementary control.

Related AWS Service (if applicable): AWS KMS; Amazon S3.

Encryption in Transit

Domain(s): 5 **Priority:** High **Category:** Security/Governance

Simple Definition: Encryption in transit is the protection of data as it moves between systems or across a network, preventing it from being read if intercepted along the way.

Detailed Explanation: For AI applications, encryption in transit protects prompts, model responses, and retrieved documents as they travel between a user's device, the application backend, the foundation model API, and any connected data stores. This is commonly implemented using protocols like TLS (Transport Layer Security). AWS PrivateLink further strengthens this protection by ensuring that traffic to AWS services never needs to traverse the public internet at all. Together, encryption in transit and encryption at rest provide complete coverage of data protection across its full lifecycle.

Everyday Analogy: It is like sending a sealed, tamper-proof envelope through the mail instead of a postcard that anyone handling it along the way could read.

Why It Matters for the Exam: This term is named alongside encryption at rest in Task Statement 5.1. Know that "at rest" means stored data and "in transit" means moving data.

Example: An organization ensures all API calls to Amazon Bedrock are made over an encrypted connection so prompts and responses cannot be intercepted and read while traveling across the network.

Common Misunderstanding: Learners sometimes assume encryption in transit is automatically guaranteed just because a service is hosted on AWS. Applications must be properly configured to use secure protocols; this is part of the customer's responsibility under the shared responsibility model.

Related AWS Service (if applicable): AWS PrivateLink; AWS KMS.

Epoch

Domain(s): 1 **Priority:** Low **Category:** AI/ML Concept

Simple Definition: An epoch is one complete pass through the entire training dataset during the model training process.

Detailed Explanation: Models are typically trained over many epochs, meaning the algorithm looks at the full training dataset multiple times, refining its internal parameters a little more with each pass. Too few epochs can leave a model underfit, having not learned enough from the data, while too many epochs can cause overfitting, where the model starts memorizing quirks of the training data rather than learning generalizable patterns. Epoch is a hyperparameter that data scientists configure, and the AIF-C01 exam tests it only at a conceptual, vocabulary level.

Everyday Analogy: It is like reading through an entire textbook once (one epoch) versus rereading the entire book multiple times to better absorb the material, though rereading it too many times might cause you to start memorizing the exact wording instead of truly understanding the concepts.

Why It Matters for the Exam: This is a low-priority, definitional term related to overfitting and underfitting. Recognize the basic definition; calculation is not required.

Example: A model is trained for 50 epochs, meaning the training algorithm passes through the entire dataset 50 separate times.

Common Misunderstanding: Learners sometimes confuse an epoch with a batch. A batch is a small subset of data processed together; an epoch is one full pass through the entire dataset, which is typically made up of many batches.

Related AWS Service (if applicable): Amazon SageMaker AI.

Equity

Domain(s): 4 **Priority:** Medium **Category:** Responsible AI

Simple Definition: Equity is the principle of ensuring AI systems provide fair outcomes that account for differences in people's starting circumstances, rather than treating everyone exactly identically.

Detailed Explanation: Equity is closely related to, but subtly different from, fairness and equality. Equality means treating everyone exactly the same; equity recognizes that identical treatment does not always produce a fair result if people start from different circumstances. In AI systems, equity-focused design might involve actively correcting for known disadvantages in the data or adjusting how a model is evaluated for different subgroups, rather than assuming a single, uniform approach will automatically be fair to everyone.

Everyday Analogy: Equality is giving every student in a classroom the exact same size step stool to see over a fence; equity is giving each student whatever size step stool they specifically need so that everyone can see over the fence equally well.

Why It Matters for the Exam: Equity is a supporting concept under the broader fairness and inclusivity objectives in Domain 4. Expect it to appear as a nuance within fairness-related

questions rather than as its own standalone heavily tested term.

Example: A company designing a hiring AI tool intentionally evaluates outcomes separately across different demographic subgroups to ensure equitable results, rather than assuming uniform treatment is automatically fair.

Common Misunderstanding: Learners often use "equity" and "equality" interchangeably. Equality is identical treatment; equity is fair treatment that may require different approaches to reach a fair outcome.

Related AWS Service (if applicable): Amazon SageMaker Clarify.

Explainability (XAI)

Domain(s): 4 **Priority:** High **Category:** Responsible AI

Simple Definition: Explainability is the ability to describe, in understandable terms, why an AI model produced a particular output or decision.

Detailed Explanation: Explainability matters because stakeholders, regulators, and affected individuals often need to understand the reasoning behind an AI decision, not just the decision itself, especially in high-stakes areas like lending, hiring, or healthcare. Some models, like simple decision trees, are naturally easy to explain, while others, like deep neural networks and large language models, are much harder to interpret because their internal reasoning involves millions or billions of interconnected parameters. There is often a tradeoff between explainability and performance: the most accurate models tend to be the least explainable, and vice versa. AWS supports explainability through tools like Amazon SageMaker Clarify, which can identify which features most influenced a prediction.

Everyday Analogy: It is the difference between a teacher who can clearly show their work and explain exactly why they arrived at a grade, versus a mysterious grading machine that just spits out a number with no explanation at all.

Why It Matters for the Exam: Task Statement 4.2 is built entirely around transparency and explainability. The exam will test your understanding of why explainability matters and the tradeoff between explainability and model performance.

Example: A bank's loan denial tool provides an explanation showing that a low credit score and high existing debt were the primary factors behind a rejection, allowing the applicant to understand and potentially address the issue.

Common Misunderstanding: Learners sometimes think explainability and transparency are identical concepts. Transparency is about openness regarding how a system works in general; explainability is specifically about being able to explain individual decisions or outputs.

Related AWS Service (if applicable): Amazon SageMaker Clarify.

F

F1 Score

Domain(s): 1, 3 **Priority:** High **Category:** AI/ML Concept

Simple Definition: The F1 score is a single metric that combines precision and recall into one number, giving a balanced view of a classification model's performance.

Detailed Explanation: F1 score is calculated as the harmonic mean of precision and recall, which means it only scores high when both precision and recall are reasonably high; a model that is excellent at one but very poor at the other will still receive a low F1 score. This makes F1 especially useful for imbalanced datasets, such as fraud detection or rare disease diagnosis, where accuracy alone can be misleading and where you specifically care about both catching true cases (recall) and avoiding false alarms (precision). F1 score is one of the four core model performance metrics named in Task Statement 1.3, alongside accuracy, precision, and recall.

Everyday Analogy: It is like a combined athletic score that only rewards an athlete who is well-rounded in both speed and accuracy, rather than someone who is excellent at one but weak in the other.

Why It Matters for the Exam: Memorize that F1 score balances precision and recall. The exam may present a scenario where accuracy is misleading and ask which metric better reflects performance; F1 score is frequently the better answer for imbalanced classification problems.

Example: A rare disease detection model with high precision but low recall (catching very few of the actual cases) would have a noticeably lower F1 score than its accuracy alone might suggest.

Common Misunderstanding: Learners sometimes think a high F1 score requires perfect precision and recall individually. It actually requires a good balance between the two; a very lopsided result in either direction will pull the F1 score down.

Related AWS Service (if applicable): Amazon SageMaker AI; Amazon Bedrock Model Evaluation.

Fairness

Domain(s): 4 **Priority:** High **Category:** Responsible AI

Simple Definition: Fairness means an AI system treats all individuals and groups justly, without producing systematically unfair advantages or disadvantages for any particular group.

Detailed Explanation: Fairness is one of the six core features of responsible AI named in Task Statement 4.1, alongside bias, inclusivity, robustness, safety, and veracity. Fairness is closely connected to bias: bias is often the cause of unfair outcomes, while fairness is the broader goal of avoiding those outcomes in the first place. There are multiple technical definitions of fairness used in the AI field (such as ensuring equal accuracy across groups, or ensuring equal false positive rates across groups), and different fairness definitions can sometimes conflict with each other, which is part of why fairness assessment is a genuinely difficult, judgment-based exercise rather than a simple checkbox.

Everyday Analogy: Think of a referee in a sports game who applies the exact same rules consistently to both teams, regardless of which team they personally favor or which team has more star players.

Why It Matters for the Exam: This is one of the most heavily tested terms in Domain 4. Expect direct definitional questions and scenario questions asking you to identify a fairness problem.

Example: A company evaluates whether its AI-powered loan approval tool produces similar approval rates for equally qualified applicants across different demographic groups.

Common Misunderstanding: Learners sometimes think fairness simply means "treating everyone identically." As the related concept of equity highlights, sometimes achieving a fair outcome requires accounting for different starting circumstances rather than applying purely uniform treatment.

Related AWS Service (if applicable): Amazon SageMaker Clarify; Amazon Bedrock Guardrails.

Feature

Domain(s): 1 **Priority:** Medium **Category:** AI/ML Concept

Simple Definition: A feature is an individual measurable property or characteristic of the data that a model uses to make predictions.

Detailed Explanation: In a tabular dataset, each column other than the target outcome typically represents a feature: for example, in a loan default prediction dataset, features might include income, credit score, and loan amount. The model learns to weigh these features in combination to make its prediction. Choosing, creating, and refining the right features (a process called feature engineering) has a major effect on how well a model performs, sometimes even more than the specific algorithm chosen.

Everyday Analogy: If you are trying to predict whether a piece of fruit is ripe, features might include its color, firmness, and smell - each individual clue you use to make your overall judgment.

Why It Matters for the Exam: Feature is foundational vocabulary supporting the broader AI/ML pipeline discussion in Task Statement 1.3, especially feature engineering.

Example: A house price prediction model uses features like square footage, number of bedrooms, and neighborhood as inputs.

Common Misunderstanding: Learners sometimes confuse "feature" (an input characteristic) with "label" (the outcome the model is trying to predict). Features are the inputs; the label is the target output during supervised learning.

Related AWS Service (if applicable): Amazon SageMaker AI.

Feature Engineering

Domain(s): 1 **Priority:** Medium **Category:** AI/ML Concept

Simple Definition: Feature engineering is the process of creating, selecting, or transforming the input variables (features) used to train a machine learning model, to help it learn more effectively.

Detailed Explanation: Feature engineering can involve combining existing features into new, more useful ones (such as calculating "debt-to-income ratio" from separate debt and income figures), removing irrelevant features that add noise, or transforming features into a more usable format (such as converting categories into numbers). Although the AIF-C01 exam explicitly states that learners are not expected to perform feature engineering themselves, you are expected to recognize it as a named stage of the AI/ML development pipeline.

Everyday Analogy: It is like a chef who, instead of just throwing raw ingredients into a pot, carefully combines and prepares them - dicing, marinating, blending - into a more useful and flavorful form before cooking.

Why It Matters for the Exam: Feature engineering is named directly as a pipeline stage in Task Statement 1.3. Remember it is explicitly listed as a job task that is out of scope to perform, but it is in scope to recognize and describe.

Example: A data science team creates a new "average purchase frequency" feature by combining a customer's total number of purchases with their account age.

Common Misunderstanding: Learners sometimes think feature engineering and data preprocessing are the same step. Preprocessing cleans and standardizes raw data; feature engineering goes further to create new, more predictive variables from that cleaned data.

Related AWS Service (if applicable): AWS Glue; AWS Glue DataBrew.

Few-Shot Learning

Domain(s): 2 **Priority:** Medium **Category:** GenAI Concept

Simple Definition: Few-shot learning refers to a model's ability to learn a new task from only a small number of examples, rather than needing a large training dataset.

Detailed Explanation: Foundation models pretrained on massive amounts of general data often already have broad enough underlying knowledge that they can adapt to a new specific task after seeing just a handful of relevant examples within the prompt itself. This is closely related to, but conceptually distinct from, few-shot prompting: few-shot learning is the broader capability of a model to generalize from limited examples, while few-shot prompting is the specific prompt engineering technique of providing those examples directly inside a prompt to demonstrate the desired pattern.

Everyday Analogy: It is like an experienced cook who, after seeing just two or three examples of an unfamiliar cuisine's dishes, can confidently start preparing similar dishes correctly, drawing on their broad existing culinary expertise.

Why It Matters for the Exam: Few-shot learning is foundational vocabulary under Domain 2's GenAI concepts. Be ready to distinguish it conceptually from the more specific, applied technique of few-shot prompting in Domain 3.

Example: A foundation model is able to correctly format addresses in a new, unusual format after being shown only three example address conversions.

Common Misunderstanding: Learners often treat "few-shot learning" and "few-shot prompting" as fully interchangeable terms. Few-shot learning describes the model's underlying capability; few-shot prompting describes the specific technique of supplying examples in a prompt to use that capability.

Related AWS Service (if applicable): Amazon Bedrock.

Few-Shot Prompting

Domain(s): 3 **Priority:** High **Category:** Foundation Model Application

Simple Definition: Few-shot prompting is a prompt engineering technique where you provide a model with several examples of the desired input-output pattern before asking it to handle a new, similar case.

Detailed Explanation: By including multiple examples directly within the prompt, few-shot prompting helps the model better understand the exact format, tone, or pattern you expect in its response, often improving accuracy and consistency compared to providing no examples at all. It sits on a spectrum with zero-shot prompting (no examples) and single-shot prompting (exactly one example), with few-shot prompting providing multiple examples for even stronger pattern guidance. The tradeoff is that more examples consume more of the model's limited context window and increase token-based cost.

Everyday Analogy: It is like training a new employee by showing them several completed sample reports before asking them to write their own, rather than describing the format only in words or showing just one example.

Why It Matters for the Exam: Task Statement 3.2 explicitly names few-shot prompting as one of the core prompt engineering techniques you must differentiate from zero-shot, single-shot, and chain-of-thought prompting.

Example: A prompt that includes five example customer emails along with their correct categorization before asking the model to categorize a sixth, new email is using few-shot prompting.

Common Misunderstanding: Learners sometimes think more examples are always better. In practice, there are diminishing returns, and including too many examples can use up valuable context window space and increase costs without meaningfully improving results.

Related AWS Service (if applicable): Amazon Bedrock; Amazon Bedrock Prompt Management.

Fine-Tuning

Domain(s): 2, 3 **Priority:** High **Category:** Foundation Model Application

Simple Definition: Fine-tuning is the process of further training an already-built foundation model on a smaller, specific dataset so it becomes better at a particular task or domain.

Detailed Explanation: Fine-tuning starts with a foundation model that already has broad general capabilities from pre-training, and then adjusts the model's internal parameters using a more

focused, often labeled dataset relevant to a specific use case, such as customer service for a particular industry. This produces a model that performs better on the targeted task than the general-purpose original model, without the enormous cost of pre-training a new model from scratch. Fine-tuning sits in the middle of the FM customization cost spectrum: more expensive and involved than prompt engineering or RAG, but far less expensive than full pre-training.

Everyday Analogy: It is like taking a general medical school graduate and putting them through a focused residency program in a specific specialty, such as cardiology, to sharpen their broad medical knowledge into specialized expertise.

Why It Matters for the Exam: Fine-tuning is one of the most heavily tested customization concepts across Domains 2 and 3. Know where it sits on the cost and complexity spectrum: more involved than RAG or in-context learning, less involved than full pre-training.

Example: A legal services company fine-tunes a foundation model using thousands of examples of correctly drafted contract clauses so it produces output in the firm's specific style and terminology.

Common Misunderstanding: Learners frequently confuse fine-tuning with RAG. Fine-tuning permanently changes the model's internal parameters through additional training; RAG leaves the model unchanged and instead supplies relevant external information at the moment of the request.

Related AWS Service (if applicable): Amazon Bedrock; Amazon SageMaker AI.

Foundation Model (FM)

Domain(s): 2, 3 **Priority:** High **Category:** GenAI Concept

Simple Definition: A foundation model is a very large, general-purpose AI model that has been pre-trained on massive amounts of data and can be adapted to perform many different tasks.

Detailed Explanation: Unlike traditional machine learning models, which are typically trained from scratch for one narrow task, a foundation model learns broad, general-purpose patterns from enormous and diverse datasets, allowing it to be applied to a wide range of downstream tasks such as summarization, translation, question answering, and code generation. Foundation models can be adapted through techniques like prompt engineering, RAG, or fine-tuning rather than requiring a brand-new model to be built for every use case. Amazon Bedrock is the primary AWS platform for accessing and customizing foundation models, including Amazon's own Nova models and models from third-party providers.

Everyday Analogy: A foundation model is like a brilliant, broadly educated generalist who has read an enormous amount about almost every subject, and who can then be quickly briefed to handle nearly any specific job you give them, rather than needing to be trained from infancy for one narrow profession.

Why It Matters for the Exam: Foundation model is central vocabulary across Domains 2 and 3. Know how it relates to LLMs (a foundation model focused on text), and understand the selection criteria and customization approaches that apply to FMs.

Example: A company uses a single foundation model accessed through Amazon Bedrock to power both its customer service chatbot and its internal document summarization tool.

Common Misunderstanding: Learners sometimes use "foundation model" and "large language model" interchangeably. An LLM is a type of foundation model focused specifically on language tasks; foundation models can also be multi-modal, covering images, audio, or combinations of data types beyond just text.

Related AWS Service (if applicable): Amazon Bedrock; Amazon Nova; Amazon SageMaker JumpStart.

G

GDPR (General Data Protection Regulation)

Domain(s): 5 **Priority:** Medium **Category:** Security/Governance

Simple Definition: GDPR is a European Union data privacy law that sets strict rules for how organizations collect, use, store, and protect personal data.

Detailed Explanation: GDPR grants individuals specific rights over their personal data, including the right to know what data is collected about them, the right to request its deletion, and the right to understand how automated decisions affecting them are made. For AI systems, GDPR has significant implications: organizations training models on data involving EU residents must ensure they have a lawful basis for using that data, and AI systems making automated decisions about individuals may need to provide meaningful explanations of those decisions, connecting GDPR compliance directly to the explainability concepts in Domain 4. GDPR is one of two named regulatory frameworks on the exam, alongside HIPAA.

Everyday Analogy: Think of GDPR like a strict set of house rules a guest must follow when they are given access to someone else's personal belongings: clear limits on what they can use, for how long, and a requirement to give everything back or destroy it if asked.

Why It Matters for the Exam: Domain 5, Task Statement 5.2 references compliance regulations broadly, and GDPR is the most commonly referenced privacy regulation alongside HIPAA. Know that GDPR is specifically about personal data privacy, primarily in the EU context.

Example: A global company training a foundation model on customer interaction data must ensure GDPR-compliant handling of any personal data belonging to its European customers, including providing a way for those customers to request data deletion.

Common Misunderstanding: Learners sometimes think GDPR only applies to companies physically located in the EU. GDPR can apply to any organization that processes the personal data of individuals located in the EU, regardless of where the organization itself is based.

Related AWS Service (if applicable): AWS Artifact; AWS Audit Manager.

Generative AI (GenAI)

Domain(s): 1, 2 **Priority:** High **Category:** GenAI Concept

Simple Definition: Generative AI is a type of AI that creates brand-new content, such as text, images, audio, or code, instead of just analyzing existing data to make a prediction or classification.

Detailed Explanation: Traditional machine learning typically predicts a single, specific output, such as "spam" or "not spam." Generative AI instead produces new, original content by learning the underlying patterns, structure, and style of its training data and then generating fresh material that follows those same patterns. GenAI is powered by foundation models, most commonly large language models for text and diffusion models for images. GenAI is a subset of the broader AI field, and it represents the technology behind chatbots, AI writing assistants, code generation tools, and image generators.

Everyday Analogy: Traditional AI is like a critic who can tell you whether a painting is good or bad; generative AI is like an artist who can paint an entirely new picture in a similar style.

Why It Matters for the Exam: This is foundational vocabulary tested across both Domain 1 and Domain 2. Be ready to explain how GenAI differs from traditional predictive ML, and how it relates to foundation models and LLMs.

Example: A marketing team uses generative AI to draft multiple original variations of ad copy from a single product description.

Common Misunderstanding: Learners sometimes think generative AI always produces perfectly accurate or factual content. Because GenAI generates new content based on learned patterns rather than retrieving verified facts, it can hallucinate, producing confident but incorrect information.

Related AWS Service (if applicable): Amazon Bedrock.

Grounding

Domain(s): 3, 5 **Priority:** High **Category:** Foundation Model Application

Simple Definition: Grounding is the practice of anchoring an AI model's response in verified, factual source material, rather than letting the model rely only on its own generated patterns.

Detailed Explanation: Grounding directly addresses the hallucination problem in generative AI by connecting a model's output to specific, checkable evidence, most commonly through Retrieval Augmented Generation, where retrieved documents from a knowledge base ground the response. Some systems also include grounding checks, where a tool evaluates whether a model's output is actually consistent with the retrieved source material before it is delivered to the user. Grounding is referenced in both Domain 3 (as a core benefit of RAG) and Domain 5 (as a hallucination detection and prevention technique).

Everyday Analogy: It is like requiring a journalist to cite verifiable sources for every claim in an article, rather than allowing them to simply write from memory or instinct without checking the facts.

Why It Matters for the Exam: Grounding connects RAG (Domain 3) to hallucination detection and prevention (Domain 5). Expect the exam to test your understanding of why grounding reduces hallucinations.

Example: A company's customer support chatbot uses RAG to ground every response in the company's actual return policy document, rather than letting the model guess at the policy from memory.

Common Misunderstanding: Learners sometimes think grounding eliminates hallucinations completely. Grounding significantly reduces the risk of hallucination by anchoring responses in real source material, but it does not guarantee a model will perfectly use or accurately represent that material every time.

Related AWS Service (if applicable): Amazon Bedrock Knowledge Bases; Amazon Bedrock Guardrails.

Ground Truth

Domain(s): 1 **Priority:** Medium **Category:** AI/ML Concept

Simple Definition: Ground truth is the verified, correct answer or label used to train and evaluate a machine learning model.

Detailed Explanation: Ground truth represents the real-world fact that a model is trying to learn to predict, such as the actual outcome of a loan (paid back or defaulted) or the actual category of an image (cat or dog). The quality of ground truth labels directly affects how well a model can learn, since a model trained on inaccurate or inconsistent ground truth will learn incorrect patterns. Ground truth quality is closely tied to the concept of label quality, which is one of the tools used to detect and monitor bias and trustworthiness in Domain 4.

Everyday Analogy: It is like the official answer key for a test: the model's predictions are compared against this trusted, verified answer key to see how well it is learning.

Why It Matters for the Exam: Ground truth supports the broader discussion of labeled data and supervised learning in Task Statement 1.1, and label quality assessment in Domain 4.

Example: A team building a medical imaging model relies on confirmed diagnoses from radiologists as the ground truth labels for training and evaluating the model.

Common Misunderstanding: Learners sometimes assume ground truth is always perfectly accurate simply because it is labeled as "ground truth." In reality, ground truth labels can themselves contain human error or bias, which is why label quality analysis matters.

Related AWS Service (if applicable): Amazon SageMaker AI.

H

Hallucination

Domain(s): 2, 4, 5 **Priority:** High **Category:** GenAI Concept

Simple Definition: A hallucination is when a generative AI model produces a confident-sounding response that is actually false, fabricated, or not supported by real evidence.

Detailed Explanation: Hallucinations occur because generative AI models generate text based on learned statistical patterns rather than by retrieving verified facts from a trusted source, which means a model can produce plausible-sounding but entirely incorrect information, fabricated citations, or invented facts. Hallucinations are one of the most significant disadvantages of GenAI for business use, since they can mislead users, create legal liability, or damage customer trust if not

properly managed. Techniques like RAG grounding, output validation, and confidence scoring are specifically used to detect and reduce the risk of hallucination.

Everyday Analogy: It is like a confident storyteller who fills in gaps in their memory with plausible-sounding details that never actually happened, telling the story with just as much conviction as the parts that are true.

Why It Matters for the Exam: Hallucination is tested across three domains: as a GenAI limitation (Domain 2), as a legal and trust risk (Domain 4), and as a security and accuracy concern requiring grounding techniques (Domain 5). This breadth makes it one of the highest-priority terms on the exam.

Example: A legal research assistant powered by a foundation model invents a plausible-sounding but completely fictitious court case citation when asked for supporting precedent.

Common Misunderstanding: Learners sometimes think hallucinations only happen with poorly built models. Even highly capable, well-built foundation models can hallucinate, because the underlying generation process is fundamentally probabilistic rather than fact-checking based, unless specifically grounded with external verified data.

Related AWS Service (if applicable): Amazon Bedrock Knowledge Bases (RAG grounding); Amazon Bedrock Guardrails.

HIPAA (Health Insurance Portability and Accountability Act)

Domain(s): 5 **Priority:** Medium **Category:** Security/Governance

Simple Definition: HIPAA is a United States law that sets strict rules for protecting the privacy and security of individuals' health information.

Detailed Explanation: HIPAA establishes requirements for how healthcare providers, insurers, and their business partners handle protected health information (PHI), including strict access controls, encryption requirements, and audit trail expectations. For AI systems specifically, any application that processes patient data, such as a clinical decision support tool or a healthcare chatbot, must be designed and operated in a way that satisfies HIPAA's privacy and security rules. HIPAA is the second of the two regulatory frameworks explicitly relevant to AI governance and compliance discussions on the exam, alongside GDPR.

Everyday Analogy: Think of HIPAA like a strict hospital policy requiring every staff member to follow specific procedures for handling, storing, and sharing patient charts, with serious consequences for mishandling that information.

Why It Matters for the Exam: Domain 5's governance and compliance objectives expect you to recognize HIPAA as a healthcare-specific compliance framework relevant to AI systems handling health data.

Example: A hospital deploying an AI-powered patient intake chatbot ensures all patient conversation data is encrypted and access-controlled in line with HIPAA requirements.

Common Misunderstanding: Learners sometimes think HIPAA applies to all personal data. HIPAA specifically governs protected health information within the healthcare context in the United States; broader personal data protection is covered by other frameworks like GDPR.

Related AWS Service (if applicable): AWS Artifact; AWS Audit Manager.

Human-in-the-Loop

Domain(s): 3, 4 **Priority:** High **Category:** Responsible AI

Simple Definition: Human-in-the-loop means a process where a person reviews, verifies, or approves an AI system's output before it is finalized or acted upon.

Detailed Explanation: Human-in-the-loop is commonly used when an AI model's confidence in a prediction is low, when the decision is especially high-stakes, or when regulatory requirements demand human oversight. It is both an evaluation method (having people rate model outputs to assess FM performance, as named in Domain 3) and a responsible AI safeguard (incorporating human review to catch errors before they cause harm, as named in Domain 4). On AWS, Amazon Augmented AI (A2I) is the dedicated service for building human review workflows into ML prediction pipelines.

Everyday Analogy: It is like a junior employee who can draft a contract on their own, but a senior partner must always review and sign off on it before it is sent to a client.

Why It Matters for the Exam: This term spans Domain 3 (as an FM evaluation method) and Domain 4 (as a responsible AI safeguard). Know Amazon A2I as the specific AWS service that operationalizes human-in-the-loop review.

Example: A document processing pipeline automatically routes any extracted form with low confidence scores to a human reviewer for manual verification before the data is used.

Common Misunderstanding: Learners sometimes think human-in-the-loop means a human reviews every single AI output. In most practical implementations, only lower-confidence or higher-risk outputs are routed to a human, while high-confidence outputs proceed automatically.

Related AWS Service (if applicable): Amazon Augmented AI (A2I).

Human Oversight

Domain(s): 4, 5 **Priority:** Medium **Category:** Responsible AI

Simple Definition: Human oversight is the broader practice of having people monitor, govern, and remain accountable for how an AI system behaves over time, beyond just reviewing individual outputs.

Detailed Explanation: While human-in-the-loop typically refers to a specific review step within a workflow, human oversight is a broader governance concept covering ongoing monitoring, periodic audits, team training requirements, and the establishment of clear lines of accountability for AI system behavior. Human oversight ensures that even highly autonomous systems, like AI agents, remain ultimately answerable to human decision-makers who can intervene, adjust policies, or shut down a system if needed. This concept connects governance (Domain 5) with responsible AI principles (Domain 4).

Everyday Analogy: Human-in-the-loop is like a manager approving an individual employee's specific decision; human oversight is the broader practice of that manager regularly reviewing the entire department's performance, policies, and training to ensure things stay on track over time.

Why It Matters for the Exam: Human oversight supports the governance training requirements named in Domain 5, Task Statement 5.2, and the broader responsible AI accountability themes in Domain 4.

Example: A company requires periodic team training and a quarterly governance review of its AI applications, in addition to point-in-time human review of individual flagged outputs.

Common Misunderstanding: Learners sometimes treat human-in-the-loop and human oversight as exactly the same thing. Human-in-the-loop is a specific operational workflow step; human oversight is the broader organizational practice of governance and accountability.

Related AWS Service (if applicable): Amazon Augmented AI (A2I); AWS Audit Manager.

Hyperparameter

Domain(s): 1 **Priority:** Medium **Category:** AI/ML Concept

Simple Definition: A hyperparameter is a setting chosen before training begins that controls how a model learns, rather than something the model learns on its own from the data.

Detailed Explanation: Examples of hyperparameters include the learning rate, batch size, and number of epochs. These are configured by a data scientist or automatically tuned through a process called hyperparameter tuning, which is explicitly listed as an out-of-scope task for the AIF-C01 target candidate to perform themselves. Despite being out of scope to perform, learners are still expected to recognize the term and understand that hyperparameters are external settings, distinct from the model's internal learned parameters (such as the weights in a neural network).

Everyday Analogy: It is like the settings you choose on an oven before you start baking, such as temperature and bake time. You decide these in advance; the oven does not learn or adjust them on its own during baking.

Why It Matters for the Exam: Hyperparameter is named as foundational vocabulary in the term list and supports your understanding of why hyperparameter tuning is explicitly out of scope to perform on this exam.

Example: A data science team sets the learning rate hyperparameter to a small value to ensure the model's training updates happen gradually and stably.

Common Misunderstanding: Learners sometimes confuse hyperparameters with model parameters (or model weights). Hyperparameters are configured before training starts; parameters/weights are learned automatically by the model during training.

Related AWS Service (if applicable): Amazon SageMaker AI.

I

IAM (Identity and Access Management)

Domain(s): 4, 5 **Priority:** High **Category:** AWS Service

Simple Definition: AWS IAM is the foundational AWS service for controlling exactly who, or what system, can access specific AWS resources and what actions they are allowed to take.

Detailed Explanation: IAM lets organizations create users, groups, and roles, and attach policies that define precisely which AWS services and actions each identity is permitted to use. For AI workloads, IAM controls who can access training data, who can deploy or modify a foundation model application, and which applications or agents can call services like Amazon Bedrock. IAM is built around the principle of least privilege, meaning identities should only be granted the minimum permissions necessary to do their job, reducing the risk of accidental or malicious misuse.

Everyday Analogy: It is like a building's keycard access system, where each employee's keycard is programmed to open only the specific doors relevant to their job, rather than every employee being given a master key to every room.

Why It Matters for the Exam: IAM is one of the highest-priority AWS services on the exam, named directly in Domain 5, Task Statement 5.1, as a core method for securing AI systems. Expect it in nearly every security-related scenario question.

Example: A company uses IAM roles to ensure that only its data science team can access the raw training data stored in Amazon S3, while application developers only have access to the already-deployed model endpoint.

Common Misunderstanding: Learners sometimes confuse IAM with AWS Secrets Manager. IAM controls access to AWS services and resources through users, roles, and policies; Secrets Manager securely stores application credentials, such as database passwords and API keys.

Related AWS Service (if applicable): AWS Secrets Manager; AWS KMS.

IAM Role

Domain(s): 5 **Priority:** High **Category:** Security/Governance

Simple Definition: An IAM role is a set of permissions that can be temporarily assumed by a user, application, or AWS service to perform specific actions, without using long-term credentials.

Detailed Explanation: Unlike an IAM user, which typically has long-term credentials tied to a specific person, a role is meant to be assumed temporarily by whatever entity needs it at the moment, whether that is an EC2 instance, a Lambda function, an AI agent, or even a different AWS account. This makes roles especially well-suited to AI applications and agents, which often need to call multiple AWS services dynamically and should not be hard-coded with permanent credentials. Amazon Bedrock AgentCore Identity, for example, relies on roles and policies to securely manage what an autonomous agent is permitted to do.

Everyday Analogy: It is like a temporary visitor badge that grants access to specific areas of a building only for the duration of a scheduled visit, rather than a permanent employee badge that someone keeps indefinitely.

Why It Matters for the Exam: IAM roles are named specifically (alongside policies and permissions) in Task Statement 5.1. Know that roles are about temporary, assumable permissions, especially relevant to securing AI agents.

Example: An AI agent built with Amazon Bedrock AgentCore assumes an IAM role that grants it permission to query a specific database, but only for the duration of completing its assigned task.

Common Misunderstanding: Learners sometimes think IAM roles and IAM users are interchangeable. Users are typically tied to a specific person with longer-term credentials; roles are designed to be temporarily assumed by whichever entity needs the permissions at a given moment.

Related AWS Service (if applicable): Amazon Bedrock AgentCore Identity.

Inference

Domain(s): 1 **Priority:** High **Category:** AI/ML Concept

Simple Definition: Inference is the process of using an already-trained model to make a prediction or generate an output on new data.

Detailed Explanation: Inference is distinct from training: training is when a model learns patterns from historical data, while inference is when that already-trained model is applied to new, real-world data to produce a useful prediction or response. There are multiple types of inferencing relevant to the exam: batch inferencing (processing a large group of inputs together, often on a schedule), real-time inferencing (responding immediately to a single request, such as a live chatbot), asynchronous inferencing (handling large individual requests that may take longer to process, with results delivered later), and serverless inferencing (running predictions without managing dedicated infrastructure, scaling automatically with demand).

Everyday Analogy: Training is like a doctor going through years of medical school; inference is that same doctor diagnosing a brand-new patient using the knowledge and judgment they already built up during their training.

Why It Matters for the Exam: Task Statement 1.1 explicitly tests your ability to distinguish batch, real-time, asynchronous, and serverless inferencing and to recognize which type fits a given business scenario.

Example: An e-commerce recommendation engine uses real-time inferencing to instantly suggest products as a customer browses the website.

Common Misunderstanding: Learners sometimes use "inference" and "training" interchangeably. Training builds the model's knowledge from historical data; inference is the separate, later step of applying that already-built knowledge to new data.

Related AWS Service (if applicable): Amazon SageMaker AI; Amazon Bedrock.

Inference Parameters

Domain(s): 3 **Priority:** Medium **Category:** Foundation Model Application

Simple Definition: Inference parameters are the configurable settings that control how a foundation model generates its response at the moment a request is made.

Detailed Explanation: Common inference parameters include temperature (which controls how creative or conservative the output is), maximum output length (which limits how long a response can be), and sampling settings like top-P and top-K (which control how the model selects from possible next words). Adjusting these parameters lets developers tune the same underlying foundation model for different use cases without retraining it; for example, a creative writing application might use a higher temperature than a factual customer support application. Inference parameters are a key design consideration in Task Statement 3.1.

Everyday Analogy: It is like adjusting the settings on a camera, such as exposure and focus, before each individual shot, rather than buying an entirely new camera every time you want a different kind of photo.

Why It Matters for the Exam: Task Statement 3.1 explicitly names inference parameters, especially temperature and output length, as a topic you must understand the effect of. Expect a scenario question asking what setting to adjust for more creative versus more predictable output.

Example: A developer lowers the temperature setting for a customer service application to make responses more consistent and predictable, while a separate creative writing tool uses a higher temperature for more varied, surprising output.

Common Misunderstanding: Learners sometimes think inference parameters permanently change the underlying model. They only affect the behavior of a specific request or session; the foundation model's underlying trained parameters remain unchanged.

Related AWS Service (if applicable): Amazon Bedrock.

Instruction Tuning

Domain(s): 3 **Priority:** Medium **Category:** Foundation Model Application

Simple Definition: Instruction tuning is a fine-tuning method that trains a model using examples of instructions paired with the correct way to respond to them, so the model becomes better at following directions.

Detailed Explanation: Instruction tuning specifically focuses on teaching a model to understand and correctly execute a wide variety of task instructions, such as "summarize this," "translate this," or "answer this question," using a dataset built around instruction-and-response pairs. This differs from domain-specific fine-tuning, which focuses on teaching deep expertise in a particular subject area, and from continuous pre-training, which expands general knowledge rather than improving instruction-following behavior. Instruction tuning is one of the named fine-tuning methods in Task Statement 3.3.

Everyday Analogy: It is like training a new assistant by giving them many practice examples of specific requests paired with the exact correct way to respond, so they get better and better at correctly interpreting and following instructions in general.

Why It Matters for the Exam: Task Statement 3.3 explicitly names instruction tuning as one of the methods for fine-tuning an FM. Know that it focuses on improving how well a model follows

instructions, distinct from domain-specific adaptation.

Example: A foundation model provider applies instruction tuning using thousands of example prompts and ideal responses so the model becomes better at correctly following multi-step user instructions.

Common Misunderstanding: Learners sometimes confuse instruction tuning with domain-specific fine-tuning. Instruction tuning improves general instruction-following ability across many task types; domain-specific fine-tuning builds deep expertise in one particular subject area or industry.

Related AWS Service (if applicable): Amazon Bedrock; Amazon SageMaker AI.

Interpretability

Domain(s): 4 **Priority:** High **Category:** Responsible AI

Simple Definition: Interpretability is how easily a human can understand the internal reasoning or mechanics of how a model arrives at its outputs.

Detailed Explanation: Interpretability and explainability are closely related but slightly different: interpretability is a property of the model itself, referring to how transparent its internal logic naturally is, while explainability often refers to the broader practice and tools used to produce understandable explanations, even for models that are not inherently interpretable. Simple models, like linear regression or decision trees, are naturally highly interpretable because a human can directly trace the math behind a decision. Complex deep learning models and large language models are far less interpretable because their internal reasoning happens across millions or billions of parameters in ways that are difficult for humans to directly trace.

Everyday Analogy: A simple recipe with five clearly labeled steps is highly interpretable; a complex, intuition-driven cooking process where a chef cannot quite explain why they made each adjustment is low in interpretability, even if the final dish tastes great.

Why It Matters for the Exam: Task Statement 4.2 expects you to know the tradeoff between model performance and interpretability: more powerful deep learning models tend to be less interpretable than simpler models.

Example: A bank chooses a simpler, more interpretable model for a regulated loan decision process, even though a more complex deep learning model might achieve slightly higher raw accuracy.

Common Misunderstanding: Learners sometimes treat interpretability and explainability as fully identical terms. Interpretability is about how inherently transparent the model's mechanics are; explainability covers the broader set of methods used to produce understandable explanations for any model, interpretable or not.

Related AWS Service (if applicable): Amazon SageMaker Clarify.

L

Label

Domain(s): 1 **Priority:** High **Category:** AI/ML Concept

Simple Definition: A label is the correct answer or outcome assigned to a piece of data, used to teach a model what it should learn to predict.

Detailed Explanation: Labels are the defining feature of supervised learning: each example in a labeled dataset is paired with the correct outcome, such as "spam" or "not spam," allowing the model to learn the relationship between input features and the correct output. The accuracy and consistency of labels (sometimes called label quality) directly affects how well a model can learn and how fair its outcomes are, which is why label quality analysis is one of the tools used to detect bias in Domain 4. Data without labels, called unlabeled data, is instead used in unsupervised learning approaches like clustering.

Everyday Analogy: A label is like the answer written on the back of a flashcard, which a student uses to check whether their guess on the front of the card was correct.

Why It Matters for the Exam: Labeled versus unlabeled data is explicitly named in Task Statement 1.1 as a key data-type distinction you must understand.

Example: In a dataset used to train a spam filter, each email is labeled as either "spam" or "not spam" based on how it was historically classified by users.

Common Misunderstanding: Learners sometimes think all machine learning requires labels. Only supervised learning requires labeled data; unsupervised learning techniques like clustering work directly with unlabeled data.

Related AWS Service (if applicable): Amazon SageMaker AI (Ground Truth labeling capabilities).

Large Language Model (LLM)

Domain(s): 1, 2 **Priority:** High **Category:** GenAI Concept

Simple Definition: A large language model is a type of foundation model trained on enormous amounts of text that can understand and generate human-like language.

Detailed Explanation: LLMs are typically built using a transformer architecture, which allows the model to weigh the relevance of different words in a piece of text relative to each other, enabling it to generate coherent, contextually relevant responses. LLMs power most modern chatbots, writing assistants, summarization tools, and code generation tools. While "foundation model" is the broader category that can include image and multi-modal models, "LLM" specifically refers to the text-focused subset of foundation models.

Everyday Analogy: An LLM is like an extraordinarily well-read writer who has absorbed an enormous portion of all human writing and can draw on that knowledge to produce new, coherent text on almost any topic you ask about.

Why It Matters for the Exam: LLM is named directly in Task Statement 1.1 as a basic AI term, and it underpins the GenAI vocabulary in Domain 2. Know the relationship: LLM is a type

of foundation model focused on language.

Example: A company uses an LLM accessed through Amazon Bedrock to automatically draft responses to customer support emails.

Common Misunderstanding: Learners sometimes assume every foundation model is an LLM. Foundation models can also be built for images, audio, or multiple modalities at once; LLM specifically describes the text-focused category.

Related AWS Service (if applicable): Amazon Bedrock; Amazon Nova.

Latent Space

Domain(s): 2 **Priority:** Low **Category:** GenAI Concept

Simple Definition: Latent space is the mathematical space where a model represents the underlying meaning or characteristics of data in a compressed, numeric form.

Detailed Explanation: Latent space is closely related to embeddings and vectors: when data is converted into a numeric representation, it is placed somewhere within this multi-dimensional mathematical space, where similar concepts end up positioned near each other. Generative models, including diffusion models, often work by manipulating points within a latent space to produce new outputs that share characteristics with the surrounding data. This is a more conceptual, lower-priority technical term on the exam compared to embedding and vector, which are tested more directly.

Everyday Analogy: It is like an enormous, invisible map where every possible idea or image has a specific location, and ideas that are conceptually related end up located near each other on that map, even though no one drew the map by hand.

Why It Matters for the Exam: Expect this as a low-priority, recognition-level supporting term under the broader embedding and vector concepts in Domain 2.

Example: An image generation model navigates its latent space to gradually produce a new image that blends characteristics learned from its training data.

Common Misunderstanding: Learners sometimes think latent space is the same thing as a vector database. Latent space is the abstract mathematical space where representations exist conceptually; a vector database is the actual storage system used to save and search numeric embeddings.

Related AWS Service (if applicable): Amazon Bedrock.

Learning Rate

Domain(s): 1 **Priority:** Low **Category:** AI/ML Concept

Simple Definition: The learning rate is a hyperparameter that controls how big of an adjustment a model makes to its internal parameters each time it learns from a training example.

Detailed Explanation: A learning rate that is too high can cause a model to overshoot the optimal pattern and fail to settle into an accurate solution; a learning rate that is too low can make training extremely slow or get the model stuck before it has fully learned the patterns in the data. Choosing the right learning rate is part of hyperparameter tuning, which is explicitly out of scope for the AIF-C01 target candidate to perform, but the term itself is still useful background vocabulary.

Everyday Analogy: It is like adjusting the size of your steps while trying to find the lowest point in a valley while blindfolded: huge steps might cause you to overshoot the lowest point repeatedly, while tiny steps will get you there accurately but very slowly.

Why It Matters for the Exam: This is a low-priority, definitional-only term related to the broader hyperparameter concept.

Example: A data science team reduces the learning rate partway through training to allow the model to make smaller, more precise adjustments as it gets closer to an optimal solution.

Common Misunderstanding: Learners sometimes think a higher learning rate is always better because it sounds faster. A learning rate that is too high can actually prevent a model from ever converging on an accurate solution.

Related AWS Service (if applicable): Amazon SageMaker AI.

LIME (Local Interpretable Model-Agnostic Explanations)

Domain(s): 4 **Priority:** Low **Category:** Responsible AI

Simple Definition: LIME is a technique that helps explain an individual prediction from any machine learning model by approximating it with a simpler, easier-to-understand model just for that one prediction.

Detailed Explanation: LIME works by making small changes to a single input and observing how the model's prediction changes in response, then using those observations to build a simplified, locally accurate explanation of why the model made that specific decision. It is described as "model-agnostic" because it can be applied to explain predictions from virtually any type of model, regardless of how that model works internally. LIME and SHAP are both supporting, lower-priority explainability techniques mentioned in the broader Domain 4 conversation about transparency and explainability tools.

Everyday Analogy: It is like zooming in on just one small section of a complicated, tangled map to draw a simple, clear local diagram that explains just that one part, without needing to fully explain the entire complex map.

Why It Matters for the Exam: LIME is a low-priority, supporting term. You should recognize it as an explainability technique, but deep technical knowledge of how it works is not required.

Example: A data science team uses LIME to explain why a complex model denied a single specific loan application, identifying that low income and high existing debt were the most influential factors for that particular case.

Common Misunderstanding: Learners sometimes think LIME explains a model's overall behavior across all predictions. LIME specifically explains individual, local predictions rather than

the model's general behavior as a whole.

Related AWS Service (if applicable): Amazon SageMaker Clarify.

M

Machine Learning (ML)

Domain(s): 1 **Priority:** High **Category:** AI/ML Concept

Simple Definition: Machine learning is a subset of AI in which computer systems learn patterns from data and improve their performance on a task without being explicitly programmed with step-by-step rules for every situation.

Detailed Explanation: Instead of a developer writing detailed rules for every possible scenario, an ML algorithm is exposed to data and automatically identifies the patterns and relationships within it, producing a trained model that can then make predictions on new data. ML includes multiple learning approaches, including supervised learning, unsupervised learning, and reinforcement learning, and it forms the technical foundation for more advanced techniques like deep learning and generative AI. ML is the central, broad subset of AI that the entire AIF-C01 exam builds outward from.

Everyday Analogy: Instead of memorizing a fixed rulebook for identifying every breed of dog, machine learning is like learning to recognize dog breeds by looking at thousands of labeled photos until you naturally pick up on the patterns yourself.

Why It Matters for the Exam: ML is the most foundational term on the entire exam. Task Statement 1.1 expects you to clearly define it and explain its relationship to AI, deep learning, and GenAI.

Example: A retailer uses machine learning to automatically predict which customers are likely to stop shopping with them, based on patterns learned from past customer behavior data.

Common Misunderstanding: Learners sometimes think ML and AI are synonyms. ML is one major approach used to build AI systems; AI is the broader field and goal that ML, among other approaches, works toward.

Related AWS Service (if applicable): Amazon SageMaker AI.

Model

Domain(s): 1 **Priority:** High **Category:** AI/ML Concept

Simple Definition: A model is the specific, trained result produced after an algorithm has learned patterns from a particular dataset.

Detailed Explanation: A model encapsulates everything the training process learned and can then be used during inference to make predictions or generate output on new data. The same algorithm trained on different datasets will produce different models, since each model reflects the unique patterns found in the specific data it learned from. Foundation models are an especially

large and general-purpose category of model, pre-trained on massive datasets and adaptable to many tasks.

Everyday Analogy: If the algorithm is a recipe, the model is the specific batch of cookies you actually baked using that recipe with your particular ingredients; a different baker using the same recipe with different ingredients would end up with a slightly different batch.

Why It Matters for the Exam: Model is one of the most fundamental terms tested in Task Statement 1.1. Be ready to clearly distinguish it from "algorithm" and "training."

Example: A company trains a model using historical sales data so it can predict next month's demand for a particular product.

Common Misunderstanding: Learners frequently use "model" and "algorithm" interchangeably. The algorithm is the general method used; the model is the specific trained output of applying that method to a particular dataset.

Related AWS Service (if applicable): Amazon SageMaker AI; Amazon Bedrock.

Model Card

Domain(s): 4, 5 **Priority:** High **Category:** AWS Service

Simple Definition: A model card is a structured document that records key information about an AI model, including its intended use, performance, limitations, and training data.

Detailed Explanation: On AWS, Amazon SageMaker Model Cards provide a standardized format for documenting a model's purpose, the data it was trained on, its evaluation metrics, known limitations, and ethical considerations. This documentation supports both transparency (helping stakeholders understand how a model works and where it may fall short) and governance and compliance (providing evidence of due diligence and data provenance for auditors and regulators). Model cards are referenced in both Domain 4 (transparency and explainability tools) and Domain 5 (documenting data origins for governance purposes).

Everyday Analogy: It is like the nutrition label on a packaged food product: a quick, standardized summary of exactly what is inside, where key ingredients came from, and any warnings a consumer should be aware of before using it.

Why It Matters for the Exam: Model cards appear in both Domain 4 and Domain 5 task statements. Know Amazon SageMaker Model Cards as the specific AWS service name.

Example: Before deploying a new fraud detection model, a company creates a Model Card documenting the training data sources, expected accuracy, and known limitations for use by both technical staff and auditors.

Common Misunderstanding: Learners sometimes think a model card is only relevant to data scientists. Model cards are designed to be useful to a broad audience, including auditors, business stakeholders, and end users who need to understand a model's appropriate use and limitations.

Related AWS Service (if applicable): Amazon SageMaker Model Cards.

Model Customization

Domain(s): 3 **Priority:** Medium **Category:** Foundation Model Application

Simple Definition: Model customization is the umbrella term for any technique used to adapt a general-purpose foundation model to perform better on a specific task or for a specific organization.

Detailed Explanation: Model customization spans a wide spectrum of approaches with very different cost and complexity profiles: prompt engineering and in-context learning (cheapest, requires no model changes), RAG (moderate cost, supplies external knowledge without changing the model), fine-tuning (more expensive, changes the model's internal parameters for a specific task), and full pre-training (most expensive, builds a brand-new foundation model from scratch). Task Statement 3.1 explicitly expects you to understand the cost tradeoffs across this entire spectrum, and to select the appropriate approach given a business scenario's requirements and constraints.

Everyday Analogy: It is like the spectrum of options for getting a custom-fit suit: buying off the rack and just adding a belt (prompt engineering), getting minor alterations from a tailor (fine-tuning), or having an entirely new suit made from scratch by a custom tailor (full pre-training).

Why It Matters for the Exam: This is a core exam concept. Expect a scenario question asking which customization approach best fits a given cost, time, or data constraint.

Example: A startup with limited budget and a small amount of proprietary data chooses RAG over fine-tuning, since RAG is significantly less expensive and does not require retraining the underlying model.

Common Misunderstanding: Learners sometimes think fine-tuning is always the "best" customization option because it changes the model directly. In many real business cases, RAG or even well-designed prompt engineering can achieve the needed result at a fraction of the cost and complexity.

Related AWS Service (if applicable): Amazon Bedrock; Amazon SageMaker AI.

Model Deployment

Domain(s): 1 **Priority:** Medium **Category:** AI/ML Concept

Simple Definition: Model deployment is the process of making a trained model available so it can actually be used to generate predictions in a real application.

Detailed Explanation: After a model has been trained and evaluated, it must be deployed before it can provide any business value. Task Statement 1.3 names two primary deployment methods: using a managed API service, where the cloud provider (such as AWS through Amazon Bedrock or SageMaker AI) handles the underlying infrastructure, and self-hosting an API, where the customer manages their own hosting environment and infrastructure. Deployment is followed by ongoing model monitoring to ensure the model continues to perform well once it is live.

Everyday Analogy: It is like the difference between finishing culinary school (training) and actually opening a restaurant where customers can order and eat your food (deployment); all the learning in the world does not provide value until it is put into actual practice.

Why It Matters for the Exam: Task Statement 1.3 explicitly names managed API services and self-hosted APIs as the two deployment methods you must be able to describe.

Example: A company deploys its fraud detection model behind a managed API endpoint in Amazon SageMaker AI so that the company's transaction processing system can request real-time predictions.

Common Misunderstanding: Learners sometimes think deployment is the final stage of the AI/ML lifecycle. In reality, deployment is followed by ongoing monitoring and potential retraining, since model performance can degrade over time due to data drift.

Related AWS Service (if applicable): Amazon SageMaker AI; Amazon Bedrock.

Model Drift

Domain(s): 1, 4 **Priority:** Medium **Category:** AI/ML Concept

Simple Definition: Model drift is the gradual decline in a deployed model's accuracy or performance over time.

Detailed Explanation: Model drift is often the downstream effect of data drift: as the real-world data a model encounters changes from what it was originally trained on, the model's predictions become progressively less accurate. Model drift can also result from changes in the relationship between inputs and outputs themselves, even if the raw data distribution does not change. Detecting model drift early through ongoing model monitoring (such as with Amazon SageMaker Model Monitor) is essential for knowing when a model needs to be retrained.

Everyday Analogy: It is like a GPS app that was extremely accurate when it launched but gradually becomes less reliable over the years as new roads are built and old ones are closed, even though the app itself never changed.

Why It Matters for the Exam: Model drift connects MLOps monitoring practices (Task Statement 1.3) with the trustworthiness monitoring tools named in Domain 4.

Example: A customer churn prediction model that performed well at launch gradually becomes less accurate over 18 months as customer behavior patterns shift, signaling model drift.

Common Misunderstanding: Learners sometimes use "model drift" and "data drift" interchangeably. Data drift describes a change in the input data itself; model drift describes the resulting decline in the model's predictive performance, which data drift commonly causes.

Related AWS Service (if applicable): Amazon SageMaker Model Monitor.

Model Evaluation

Domain(s): 3 **Priority:** High **Category:** AWS Service

Simple Definition: Model evaluation is the process of systematically testing a model's quality and performance, and Amazon Bedrock Model Evaluation is AWS's managed service for doing this for foundation models.

Detailed Explanation: Amazon Bedrock Model Evaluation allows organizations to assess and compare foundation models using automated metrics (like ROUGE, BLEU, and BERTScore), human evaluation workflows, or even another LLM acting as a judge (LLM-as-a-judge). This managed evaluation capability removes much of the manual effort otherwise required to systematically test FM quality, helping organizations confidently choose between candidate models or confirm that a customized model meets business requirements before deployment. Model evaluation is one of the named evaluation approaches in Task Statement 3.4, alongside human-in-the-loop evaluation and benchmark datasets.

Everyday Analogy: It is like a standardized test review board that can score multiple candidates' essay answers using a consistent rubric, freeing teachers from having to grade every single submission manually from scratch.

Why It Matters for the Exam: Task Statement 3.4 explicitly names Amazon Bedrock Model Evaluation as a key approach for evaluating FM performance. It is one of the most heavily tested service names in Domain 3.

Example: A company uses Amazon Bedrock Model Evaluation to automatically compare three candidate foundation models' summarization quality using ROUGE scores before selecting one for production.

Common Misunderstanding: Learners sometimes think Bedrock Model Evaluation only supports automated metrics. It also supports human evaluation workflows, allowing organizations to combine automated scoring with human judgment.

Related AWS Service (if applicable): Amazon Bedrock Model Evaluation.

Model Governance

Domain(s): 5 **Priority:** Medium **Category:** Security/Governance

Simple Definition: Model governance is the set of policies and processes an organization uses specifically to track, control, and oversee its AI/ML models throughout their lifecycle.

Detailed Explanation: Model governance covers questions like who approved a model for production use, what version of the model is currently deployed, what data it was trained on, and when it is scheduled for review or retraining. It is a more model-specific subset of the broader AI governance umbrella, which also covers organizational policy, training, and review cadence beyond just individual models. Tools like a model registry support model governance by providing a centralized, trackable inventory of model versions.

Everyday Analogy: It is like a hospital's detailed record-keeping for every piece of medical equipment: which device is currently in use, who approved it, when it was last serviced, and when it is due for its next inspection.

Why It Matters for the Exam: Model governance is a supporting concept under the broader governance and compliance objectives in Domain 5, Task Statement 5.2.

Example: A bank maintains model governance records showing exactly which version of its credit risk model is currently live, who approved its deployment, and when its next scheduled review is due.

Common Misunderstanding: Learners sometimes treat model governance and AI governance as identical. AI governance is the broader organizational umbrella covering policy and oversight across all AI use; model governance specifically focuses on tracking and controlling individual models and their versions.

Related AWS Service (if applicable): Amazon SageMaker Model Cards.

Model Inversion

Domain(s): 4, 5 **Priority:** Low **Category:** Security/Governance

Simple Definition: Model inversion is a type of attack where someone tries to reconstruct sensitive details about the original training data just by studying how a model responds to different inputs.

Detailed Explanation: In a model inversion attack, an attacker repeatedly queries a model and analyzes its outputs to infer characteristics of the data the model was trained on, potentially exposing private or sensitive information that should never have been recoverable from the deployed model alone. This is a privacy and security concern especially relevant to models trained on sensitive personal data, such as health records or biometric data. Model inversion is a lower-priority, supporting security concept on the exam compared to more heavily tested threats like prompt injection.

Everyday Analogy: It is like trying to reconstruct a person's home address just by asking them a long series of seemingly unrelated, indirect questions and piecing together their answers, even though they never directly told you where they live.

Why It Matters for the Exam: This is a low-priority, recognition-level security term that supports the broader data leakage and privacy themes in Domain 5.

Example: A research team tests their facial recognition model for vulnerability to model inversion attacks that might reconstruct recognizable images of individuals from the training dataset.

Common Misunderstanding: Learners sometimes confuse model inversion with simple data leakage. Data leakage usually refers to sensitive information accidentally appearing directly in an output; model inversion is a more deliberate, indirect attack technique aimed at reconstructing training data through repeated probing.

Related AWS Service (if applicable): Amazon Bedrock Guardrails; AWS KMS.

Model Latency

Domain(s): 3 **Priority:** Medium **Category:** Foundation Model Application

Simple Definition: Model latency is the amount of time it takes for a model to generate a response after receiving a request.

Detailed Explanation: Latency is one of the FM selection criteria named in Task Statement 3.1, since different business use cases have very different latency tolerance: a live customer-facing chatbot needs very low latency for a good user experience, while a batch document summarization task can tolerate much higher latency. Latency is influenced by factors like model size, output

length, and infrastructure choices, including whether provisioned throughput is used to guarantee more consistent response times.

Everyday Analogy: It is like the difference between asking a question to someone standing right next to you (low latency) versus mailing a letter and waiting for a reply (high latency); both eventually answer your question, but the experience is very different.

Why It Matters for the Exam: Task Statement 3.1 names latency directly as an FM selection criterion. Expect scenario questions where latency requirements help determine the right model or deployment choice.

Example: A real-time voice assistant application prioritizes a smaller, faster foundation model to minimize latency, even if a larger model might produce marginally higher-quality responses.

Common Misunderstanding: Learners sometimes assume the largest, most capable model is always the best choice. For latency-sensitive applications, a smaller or more optimized model may be the better business decision, even with a slight tradeoff in raw output quality.

Related AWS Service (if applicable): Amazon Bedrock.

Model Parameter

Domain(s): 2 **Priority:** Medium **Category:** GenAI Concept

Simple Definition: A model parameter is one of the internal numeric values that a model adjusts and learns during training in order to make accurate predictions.

Detailed Explanation: Foundation models can have billions of parameters, and the total parameter count is often used as a rough (though imperfect) indicator of a model's capacity and complexity. Parameters are sometimes referred to as weights, since each parameter represents the "weight" or importance given to a particular connection or pattern within the model. Parameter count is one factor among many (including cost, latency, and capability) that influences model selection decisions in Domain 3.

Everyday Analogy: Think of parameters like the countless small adjustments a sound engineer makes to dozens of mixing board dials to get a song's final sound exactly right; each individual dial setting is like one parameter, and there can be an enormous number of them working together.

Why It Matters for the Exam: Model parameter and model size are referenced as part of FM selection criteria in Domain 3. You are not expected to calculate parameter counts, only to recognize the term.

Example: A company chooses a smaller foundation model with fewer parameters for a simple, cost-sensitive classification task, rather than a much larger model intended for complex reasoning.

Common Misunderstanding: Learners sometimes assume more parameters always means a better model for every use case. A larger parameter count generally increases capability but also increases cost and latency, so the right size depends on the specific task.

Related AWS Service (if applicable): Amazon Bedrock.

Model Provider

Domain(s): 2 **Priority:** Low **Category:** GenAI Concept

Simple Definition: A model provider is the company or organization that develops and makes a particular foundation model available for use.

Detailed Explanation: Amazon Bedrock is specifically designed to give customers access to foundation models from multiple different model providers through a single, unified API, rather than locking an organization into only one company's models. This includes Amazon's own Nova models as well as models from various third-party AI companies. Understanding that Bedrock is provider-agnostic, offering choice across providers, is a useful supporting detail when comparing Bedrock to a platform tied to a single model source.

Everyday Analogy: It is like a single department store that carries multiple competing brands of appliances under one roof, letting a shopper compare and choose between brands rather than being limited to only one manufacturer's products.

Why It Matters for the Exam: This is a low-priority, supporting detail under the broader discussion of Amazon Bedrock's multi-provider model access in Domain 2.

Example: A company evaluates foundation models from several different model providers, all accessible through Amazon Bedrock, before selecting the one that best fits its cost and performance needs.

Common Misunderstanding: Learners sometimes think Amazon Bedrock is itself a model provider in the same sense as the companies whose models it hosts. Bedrock is the platform; the model providers are the separate organizations that built the individual foundation models available through that platform (including Amazon itself, for models like Nova).

Related AWS Service (if applicable): Amazon Bedrock; Amazon Nova.

Model Registry

Domain(s): 5 **Priority:** Low **Category:** Security/Governance

Simple Definition: A model registry is a centralized system for tracking and managing different versions of trained models throughout their lifecycle.

Detailed Explanation: A model registry typically records metadata such as which version of a model is currently in production, when each version was trained, what data and parameters were used, and its evaluation results. This supports model governance and model risk management by making it possible to quickly answer questions like "which model version made this prediction" or "can we roll back to a previous version if a problem is discovered." A model registry is a supporting tool for the broader governance objectives in Domain 5.

Everyday Analogy: It is like a library's card catalog system that tracks every edition of every book, including which edition is currently checked out and where older editions are stored, so nothing gets lost or confused.

Why It Matters for the Exam: This is a low-priority, supporting term under the broader model governance and MLOps themes.

Example: A company uses a model registry to confirm exactly which version of its recommendation model is currently live in production before approving a new version for deployment.

Common Misunderstanding: Learners sometimes think a model registry only matters for very large organizations. Even smaller teams benefit from tracking model versions to support troubleshooting, rollback, and compliance needs.

Related AWS Service (if applicable): Amazon SageMaker AI (model registry capability).

Model Risk Management (MRM)

Domain(s): 5 **Priority:** Low **Category:** Security/Governance

Simple Definition: Model risk management is the organizational practice of identifying, assessing, and controlling the risks that could arise from a model's errors, misuse, or unintended consequences.

Detailed Explanation: MRM is especially important in heavily regulated industries like banking and insurance, where models influence high-stakes financial decisions, and regulators expect documented evidence that an organization understands and actively manages model-related risks. MRM ties together several other concepts on this exam, including model validation, ongoing performance monitoring, bias detection, and governance documentation like model cards, into a single coordinated risk-management discipline. This is a lower-priority, supporting term that connects governance themes from Domain 5 to broader enterprise risk practices.

Everyday Analogy: It is like a building inspector's ongoing program for an entire city, not just checking one building once, but continuously assessing structural risk, tracking which buildings need repairs, and maintaining records to demonstrate due diligence to regulators.

Why It Matters for the Exam: This is a lower-priority supporting concept; recognize it conceptually as the organizational practice of managing AI/ML-related risk rather than expecting deep technical detail.

Example: A bank's model risk management team requires every new AI-driven lending model to pass an independent validation review and ongoing monitoring before and after deployment.

Common Misunderstanding: Learners sometimes think MRM only applies to traditional statistical models. It increasingly applies to generative AI and foundation model applications as well, given their growing use in business-critical decisions.

Related AWS Service (if applicable): AWS Audit Manager; Amazon SageMaker Model Cards.

Model Throughput

Domain(s): 3 **Priority:** Low **Category:** Foundation Model Application

Simple Definition: Model throughput is the volume of requests or amount of data a model can process within a given period of time.

Detailed Explanation: Throughput matters for applications that need to handle a high volume of simultaneous requests, such as a customer service platform serving thousands of users at

once. AWS offers provisioned throughput as an option for foundation models, which reserves dedicated processing capacity to guarantee a consistent throughput level, generally at a different cost structure than on-demand, token-based pricing. Throughput and latency are related but distinct: latency measures the speed of a single response, while throughput measures overall request-handling capacity over time.

Everyday Analogy: Latency is how quickly one customer gets served at a coffee shop counter; throughput is how many total customers the shop can serve across an entire busy morning rush.

Why It Matters for the Exam: Throughput supports the cost tradeoff discussion in Task Statement 2.3, particularly around provisioned throughput as a pricing and performance option.

Example: A large enterprise selects provisioned throughput for its customer-facing GenAI application to guarantee consistent performance during periods of extremely high demand.

Common Misunderstanding: Learners sometimes confuse throughput with latency. Latency is about the speed of an individual response; throughput is about the total capacity to handle many requests over a period of time.

Related AWS Service (if applicable): Amazon Bedrock (provisioned throughput).

Model Weight

Domain(s): 2 **Priority:** Low **Category:** GenAI Concept

Simple Definition: A model weight is another name for a model parameter: an internal numeric value that determines how much influence a particular input or pattern has on the model's output.

Detailed Explanation: "Weight" and "parameter" are largely used interchangeably in casual discussion of foundation models, referring to the learned numeric values adjusted during training. Open-source foundation models are sometimes described by whether their weights are publicly released, which affects how much customization and self-hosting flexibility an organization has compared to a closed, API-only model. This is a low-priority, recognition-level term that supports the broader discussion of model parameters and FM sourcing.

Everyday Analogy: It is like the individual settings on a complex audio equalizer; each weight is like one small dial setting that, combined with thousands or billions of others, shapes the overall final output.

Why It Matters for the Exam: This is a low-priority supporting term. Recognize that "weight" and "parameter" generally describe the same underlying concept.

Example: A research team chooses an open-source foundation model specifically because its weights are publicly available, allowing for more extensive self-hosted customization.

Common Misunderstanding: Learners sometimes think "weight" refers to the overall size or file size of a model. It actually refers to the specific learned numeric values inside the model, not the storage size of the model file.

Related AWS Service (if applicable): Amazon Bedrock; Amazon SageMaker JumpStart.

Multimodal Model

Domain(s): 2, 3 **Priority:** High **Category:** GenAI Concept

Simple Definition: A multimodal model is a foundation model that can understand and/or generate more than one type of data, such as text, images, and audio, all within the same model.

Detailed Explanation: A multimodal model might accept an image and a text question together and produce a text answer describing the image, or generate an image directly from a text description. This capability expands the range of business use cases a single foundation model can support, since separate single-purpose models are not required for each data type. Modality is explicitly named as one of the FM selection criteria in Task Statement 3.1, since some business use cases specifically require multimodal capability while others only need a single modality, such as text-only.

Everyday Analogy: It is like a single translator who is fluent not just in two spoken languages, but also in sign language and written text, able to seamlessly work across all of those different forms of communication at once.

Why It Matters for the Exam: Multimodal capability is named directly in Task Statement 3.1 as a selection criterion, and it is foundational vocabulary in Task Statement 2.1. Recognize use cases that specifically require multimodal support.

Example: A retail company uses a multimodal foundation model that can analyze a customer-submitted photo of a damaged product alongside a written complaint to generate an appropriate response.

Common Misunderstanding: Learners sometimes assume every foundation model is automatically multimodal. Many foundation models, including many LLMs, are text-only; multimodal capability is a specific, distinguishing feature that not every model has.

Related AWS Service (if applicable): Amazon Bedrock; Amazon Nova.

N

Natural Language Processing (NLP)

Domain(s): 1 **Priority:** High **Category:** AI/ML Concept

Simple Definition: Natural language processing is the field of AI focused on enabling computers to understand, interpret, and generate human language.

Detailed Explanation: NLP covers a wide range of tasks, including sentiment analysis, entity recognition, language translation, text summarization, and speech-related tasks when combined with audio processing. Modern NLP is largely powered by large language models built using deep learning, which can handle far more nuanced and flexible language tasks than older, rule-based language processing approaches. On AWS, core NLP capability is delivered through Amazon Comprehend (analyzing existing text) and Amazon Translate (translating between languages), while more advanced generative language tasks rely on foundation models accessed through Amazon Bedrock.

Everyday Analogy: It is like teaching a computer to read, understand the tone of, and even respond to a letter the way a thoughtful human reader would, rather than just scanning it for specific keywords.

Why It Matters for the Exam: NLP is named directly in Task Statement 1.1 as a basic AI term to define, and again in Task Statement 1.2 as a real-world application category. Know Amazon Comprehend as the primary NLP analysis service.

Example: A company uses Amazon Comprehend to automatically analyze the sentiment of thousands of customer reviews, sorting them into positive, negative, and neutral categories.

Common Misunderstanding: Learners sometimes think NLP is the same thing as generative AI. NLP is the broader field covering both traditional analysis tasks (like sentiment detection) and generative tasks (like text generation); generative AI is a more recent, specific approach to handling some NLP tasks.

Related AWS Service (if applicable): Amazon Comprehend; Amazon Translate.

Neural Network

Domain(s): 1 **Priority:** High **Category:** AI/ML Concept

Simple Definition: A neural network is a machine learning model structure loosely inspired by the brain's neurons, made up of layers of interconnected nodes that process and pass along information.

Detailed Explanation: A neural network typically has an input layer (where data enters), one or more hidden layers (where patterns are processed), and an output layer (where the final prediction or result emerges). Each connection between nodes has an associated weight, and each node applies an activation function to decide how strongly to pass information forward. Deep learning specifically refers to neural networks with many hidden layers, allowing them to learn increasingly complex and abstract patterns. Neural networks are the foundational architecture behind nearly all modern deep learning and generative AI systems.

Everyday Analogy: It is like an enormous assembly line of inspectors, where each inspector looks at the work passed to them from the previous one, makes a small judgment call, and passes their own refined version forward, until the final inspector produces the finished result.

Why It Matters for the Exam: Neural network is named directly in Task Statement 1.1 as a basic term to define, and it forms the conceptual foundation for understanding deep learning and foundation models.

Example: A computer vision system uses a neural network with many layers to progressively recognize edges, shapes, and eventually full objects within an image.

Common Misunderstanding: Learners sometimes think neural networks work exactly like a biological brain. The "neuron" terminology is a loose, simplified inspiration; the actual mathematics involved is quite different from how biological brains truly function.

Related AWS Service (if applicable): Amazon SageMaker AI.

NIST AI RMF (NIST AI Risk Management Framework)

Domain(s): 5 **Priority:** Low **Category:** Security/Governance

Simple Definition: The NIST AI Risk Management Framework is a voluntary set of guidelines from the U.S. National Institute of Standards and Technology that helps organizations identify, assess, and manage risks associated with AI systems.

Detailed Explanation: The framework organizes AI risk management around four core functions: govern, map, measure, and manage, providing a structured way for organizations to think through AI risk at every stage, from initial design through deployment and ongoing monitoring. While the AIF-C01 exam more directly emphasizes the Generative AI Security Scoping Matrix as its named governance framework, the NIST AI RMF is a broadly recognized reference framework relevant to the overall responsible AI and governance conversation in Domains 4 and 5.

Everyday Analogy: It is like a general safety inspection checklist used across many different industries, providing a consistent structure for identifying and managing risk, even though each specific organization applies it slightly differently to their own situation.

Why It Matters for the Exam: This is a low-priority, supporting governance framework. The exam places more direct emphasis on the Generative AI Security Scoping Matrix by name, but recognize NIST AI RMF as another relevant framework if it appears as an answer choice.

Example: A government contractor aligns its internal AI governance program with the NIST AI Risk Management Framework to demonstrate a structured approach to managing AI-related risk.

Common Misunderstanding: Learners sometimes think the NIST AI RMF is a legally binding regulation. It is a voluntary framework providing guidance and structure, not a law with direct enforcement power.

Related AWS Service (if applicable): AWS Audit Manager.

O

One-Shot Prompting (Single-Shot Prompting)

Domain(s): 3 **Priority:** High **Category:** Foundation Model Application

Simple Definition: One-shot prompting (also called single-shot prompting) is a prompt engineering technique where you provide exactly one example of the desired input-output pattern before asking the model to handle a new case.

Detailed Explanation: One-shot prompting sits between zero-shot prompting (no examples at all) and few-shot prompting (multiple examples) on the spectrum of example-based prompting techniques. Providing a single example can meaningfully improve a model's understanding of the expected format or style compared to zero-shot prompting, while using fewer tokens and less context window space than few-shot prompting. The official AIF-C01 exam guide refers to this technique as "single-shot prompting," while it is also commonly called "one-shot prompting" in the broader industry.

Everyday Analogy: It is like showing a new employee exactly one completed example of a report before asking them to write their own, giving them a helpful reference point without requiring an

extensive training session.

Why It Matters for the Exam: Task Statement 3.2 names single-shot prompting directly as one of the prompt engineering techniques you must define and differentiate from zero-shot, few-shot, and chain-of-thought prompting.

Example: A prompt that includes exactly one example of a correctly formatted product description before asking the model to write a new one for a different product is using one-shot prompting.

Common Misunderstanding: Learners sometimes think one-shot prompting and few-shot prompting are the same since both involve examples. The key distinction is quantity: one-shot uses exactly one example, while few-shot uses multiple examples.

Related AWS Service (if applicable): Amazon Bedrock; Amazon Bedrock Prompt Management.

Orchestration

Domain(s): 2, 3 **Priority:** Medium **Category:** GenAI Concept

Simple Definition: Orchestration is the coordination of multiple steps, tools, or AI agents to work together in the correct order to complete a complex task.

Detailed Explanation: In agentic AI systems, orchestration determines how an agent breaks a goal into smaller steps, decides which tool or data source to use at each step, and manages the overall sequence and flow of a multi-step task. In multi-agent systems, orchestration also covers how different specialized agents communicate and hand off work to each other. Workflow orchestration is named directly as a foundational agentic AI concept in Task Statement 2.1, and is operationally supported on AWS through services like Amazon Bedrock Agents and Amazon Bedrock AgentCore.

Everyday Analogy: It is like a film director who does not personally act, film, or edit every scene, but coordinates the actors, camera crew, and editors so the entire production comes together correctly and in the right order.

Why It Matters for the Exam: Workflow orchestration is named in Task Statement 2.1 as a core agentic AI concept. Recognize it as the coordination layer that enables multi-step, multi-tool agent behavior.

Example: An AI agent uses orchestration to first retrieve a customer's account details, then check inventory availability, and finally generate a personalized response, all in the correct sequence.

Common Misunderstanding: Learners sometimes think orchestration only matters for multi-agent systems. Even a single AI agent performing several sequential steps relies on orchestration logic to manage that sequence correctly.

Related AWS Service (if applicable): Amazon Bedrock AgentCore; Amazon Bedrock Agents.

Overfitting

Domain(s): 1, 4 **Priority:** High **Category:** AI/ML Concept

Simple Definition: Overfitting happens when a model learns the training data so closely, including its noise and quirks, that it performs poorly on new, unseen data.

Detailed Explanation: An overfit model essentially memorizes the specific examples it was trained on rather than learning the more general, underlying patterns that would transfer well to new situations. This often shows up as a model that performs extremely well on its own training data but noticeably worse on a separate validation or test dataset. Overfitting is closely related to the bias-variance tradeoff: overfit models typically have high variance, meaning their predictions change significantly with small changes in the training data. Techniques like cross-validation, regularization, and dropout help reduce the risk of overfitting.

Everyday Analogy: It is like a student who memorizes the exact answers to specific practice questions instead of actually understanding the underlying concepts, and then performs poorly when the real exam asks similar questions phrased differently.

Why It Matters for the Exam: Overfitting is named directly in Task Statement 1.1 and again in Domain 4's discussion of bias and variance effects on model performance. Know that overfitting is associated with high variance.

Example: A model trained on a small dataset of only 50 customer examples performs perfectly on those 50 examples but fails badly when applied to thousands of new customers.

Common Misunderstanding: Learners sometimes think more training is always better and will fix performance problems. Training a model for too long or on too little varied data can actually cause it to overfit, making its real-world performance worse rather than better.

Related AWS Service (if applicable): Amazon SageMaker AI; Amazon SageMaker Model Monitor.

P

Perplexity

Domain(s): 3 **Priority:** Low **Category:** Foundation Model Application

Simple Definition: Perplexity is a metric that measures how well a language model predicts the next word in a sequence of text, with lower perplexity generally indicating better predictive performance.

Detailed Explanation: Perplexity is a more technical, statistical evaluation metric than the business-facing metrics like task completion rate or user satisfaction, and it is most often used by model developers during training and research rather than by business users evaluating an application's real-world success. A model with low perplexity is, on average, less "surprised" by the actual next word in real text, suggesting it has learned the underlying patterns of language well. This is a lower-priority, supporting metric on the exam compared to ROUGE, BLEU, BERTScore, and LLM-as-a-judge.

Everyday Analogy: It is like measuring how often a very good guesser correctly predicts the next word in a sentence someone is reading aloud to them; a great guesser who is rarely surprised has low perplexity.

Why It Matters for the Exam: This is a low-priority, recognition-level metric. The exam more heavily emphasizes ROUGE, BLEU, BERTScore, and LLM-as-a-judge as the named FM evaluation metrics in Task Statement 3.4.

Example: A model research team monitors perplexity scores during the pre-training process to track how well the model is learning to predict language patterns over time.

Common Misunderstanding: Learners sometimes think perplexity directly measures whether a model's output is factually correct or helpful for business purposes. It only measures how well the model predicts likely word sequences, which is not the same as factual accuracy or business usefulness.

Related AWS Service (if applicable): Amazon Bedrock Model Evaluation.

PII (Personally Identifiable Information)

Domain(s): 4, 5 **Priority:** High **Category:** Security/Governance

Simple Definition: PII is any piece of information that could be used, alone or combined with other data, to identify a specific individual.

Detailed Explanation: Common examples of PII include full names, social security numbers, email addresses, phone numbers, and account numbers. AI systems pose a particular PII risk in two directions: training data may contain PII that should be protected or removed, and generated outputs might accidentally reveal PII that was present in training data or retrieved knowledge base content. Amazon Macie is specifically designed to automatically discover and classify PII and other sensitive data stored in Amazon S3, while Amazon Bedrock Guardrails can detect and redact PII appearing in real-time model inputs or outputs.

Everyday Analogy: It is like the specific combination of details on a person's driver's license: individually some details might seem harmless, but combined together they uniquely identify exactly one specific person.

Why It Matters for the Exam: PII detection and protection is referenced in both Domain 4 (responsible data characteristics) and Domain 5 (security and privacy considerations, especially data leakage prevention). Know Amazon Macie and Amazon Bedrock Guardrails as the relevant AWS tools.

Example: A company uses Amazon Bedrock Guardrails to automatically detect and redact any customer phone numbers or email addresses that might accidentally appear in a chatbot's generated response.

Common Misunderstanding: Learners sometimes think PII only refers to obviously sensitive data like social security numbers. PII can also include combinations of seemingly minor details, like birth date plus zip code plus gender, that together can uniquely identify a person.

Related AWS Service (if applicable): Amazon Macie; Amazon Bedrock Guardrails.

Precision

Domain(s): 1, 3 **Priority:** High **Category:** AI/ML Concept

Simple Definition: Precision measures, out of all the times a model predicted a positive result, how often that prediction was actually correct.

Detailed Explanation: Precision is calculated as true positives divided by the sum of true positives and false positives. High precision means that when the model says "yes, this is a positive case," it is usually right; it does not necessarily mean the model catches every actual positive case, which is what recall measures instead. Precision is especially important in scenarios where a false positive is costly or disruptive, such as flagging legitimate transactions as fraudulent and unnecessarily blocking a customer's purchase.

Everyday Analogy: If a smoke detector only goes off for genuine fires and almost never for burnt toast, it has high precision; every alarm it does sound is highly likely to indicate a real fire.

Why It Matters for the Exam: Precision is named directly alongside accuracy, recall, and F1 score in Task Statement 1.3. Memorize the precision-versus-recall distinction: precision cares about the accuracy of positive predictions, while recall cares about catching all actual positives.

Example: A spam filter with high precision rarely marks a legitimate email as spam, even if it occasionally misses some actual spam messages.

Common Misunderstanding: Learners frequently confuse precision and recall. A simple way to remember the difference: precision asks "of what I flagged as positive, how much was actually correct?" while recall asks "of all the actual positives out there, how many did I successfully catch?"

Related AWS Service (if applicable): Amazon SageMaker AI; Amazon Bedrock Model Evaluation.

Pre-Training

Domain(s): 2, 3 **Priority:** High **Category:** GenAI Concept

Simple Definition: Pre-training is the initial, large-scale phase of training a foundation model on massive amounts of general data before it is adapted for any specific task.

Detailed Explanation: During pre-training, a model is exposed to enormous datasets, often scraped from a huge portion of publicly available text, images, or other content, and learns broad, general-purpose patterns of language, vision, or reasoning. This is the most resource-intensive and expensive stage of the foundation model lifecycle, requiring massive computing infrastructure and significant time. Once pre-training is complete, the resulting foundation model can then be adapted to specific use cases through far less expensive methods like fine-tuning, RAG, or simple prompt engineering, rather than requiring every application to repeat this expensive pre-training process from scratch.

Everyday Analogy: Pre-training is like a person completing a broad, general university education covering many subjects, before later specializing through a more focused graduate program (fine-tuning) tailored to a specific career.

Why It Matters for the Exam: Pre-training is named directly in both the FM lifecycle (Task Statement 2.1) and FM training stages (Task Statement 3.3). Know that it is the most expensive customization approach on the cost spectrum.

Example: A major AI company spends enormous computing resources pre-training a new foundation model on a massive, diverse dataset before making it available through a platform like Amazon Bedrock.

Common Misunderstanding: Learners sometimes think most organizations using AI need to perform pre-training themselves. The vast majority of organizations instead use already-pre-trained foundation models and apply much less expensive customization techniques like RAG or fine-tuning.

Related AWS Service (if applicable): Amazon Bedrock; Amazon SageMaker AI.

Principle of Least Privilege

Domain(s): 5 **Priority:** High **Category:** Security/Governance

Simple Definition: The principle of least privilege means giving a user, application, or system only the minimum permissions it needs to do its job, and nothing more.

Detailed Explanation: Applying least privilege to AI systems means an AI agent, application, or pipeline should only have access to the specific data sources, AWS services, and actions strictly required for its task, reducing the potential damage if that system is compromised or misused. This is especially important for autonomous AI agents, since an agent with overly broad permissions could take unintended or harmful actions across systems it never actually needed to access. Least privilege is implemented through carefully scoped IAM policies and roles.

Everyday Analogy: It is like giving a hotel housekeeper a key that only opens the specific rooms they are assigned to clean that day, rather than a master key that opens every room in the entire hotel.

Why It Matters for the Exam: This principle underlies the broader IAM and access control discussion in Domain 5, Task Statement 5.1. Expect scenario questions where the "best practice" answer involves restricting permissions to only what is necessary.

Example: A company configures an AI agent's IAM role to allow it to only read from a specific product catalog database, rather than granting broad access to the company's entire data infrastructure.

Common Misunderstanding: Learners sometimes think least privilege only applies to human users. It applies equally, and arguably more critically, to AI agents, applications, and automated systems that can take autonomous actions.

Related AWS Service (if applicable): AWS IAM.

Prompt

Domain(s): 2, 3 **Priority:** High **Category:** GenAI Concept

Simple Definition: A prompt is the input, usually text, that you give to a foundation model to tell it what task to perform or what response to generate.

Detailed Explanation: A prompt can be as simple as a single question or as complex as a detailed set of instructions including context, examples, and formatting requirements. The structure and quality of a prompt directly affects the quality of a model's response, which is the entire premise behind the discipline of prompt engineering. Prompts can also be categorized by role: a system prompt sets overall behavior and constraints for the model, while a user prompt is the specific request from an individual user within that established context.

Everyday Analogy: A prompt is like the specific instructions you give a freelance writer for an assignment; a vague, one-line instruction will get a much less useful result than a clear, detailed brief.

Why It Matters for the Exam: Prompt is foundational vocabulary across Domains 2 and 3. Understand its core components (context, instruction, negative prompts) as named in Task Statement 3.2.

Example: A customer asks a chatbot, "Summarize this contract in three bullet points," which serves as the prompt guiding the model's response.

Common Misunderstanding: Learners sometimes think a longer prompt is always better. Effective prompts balance specificity and concision; an overly long, unfocused prompt can confuse a model just as much as one that is too vague.

Related AWS Service (if applicable): Amazon Bedrock; Amazon Bedrock Prompt Management.

Prompt Engineering

Domain(s): 2, 3 **Priority:** High **Category:** Foundation Model Application

Simple Definition: Prompt engineering is the practice of carefully designing and refining the instructions you give a foundation model in order to get the most accurate, useful, and consistent response.

Detailed Explanation: Prompt engineering includes techniques such as zero-shot, single-shot, few-shot, and chain-of-thought prompting, as well as best practices like providing clear context, being appropriately specific and concise, using negative prompts to specify what to avoid, and using guardrails to constrain output. It is the cheapest and fastest method of customizing a foundation model's behavior for a particular use case, requiring no changes to the model itself, unlike fine-tuning or pre-training. Prompt versioning and management, supported on AWS through Amazon Bedrock Prompt Management, helps teams systematically test, track, and improve prompts over time as part of an ongoing prompt engineering practice.

Everyday Analogy: It is like learning to ask a brilliant but very literal expert exactly the right questions, in exactly the right way, to get the most useful and complete answer out of them.

Why It Matters for the Exam: Prompt engineering is one of the most heavily tested topics in Domain 3 (28% of the exam). Be thoroughly comfortable with all of its named techniques, benefits, best practices, and risks.

Example: A team experiments with several different prompt phrasings and structures to find the version that most reliably produces correctly formatted, accurate summaries from their foundation model.

Common Misunderstanding: Learners sometimes think prompt engineering is a one-time setup step. In practice, it is an ongoing, iterative discipline, including ongoing experimentation, versioning, and refinement as business needs and models evolve.

Related AWS Service (if applicable): Amazon Bedrock Prompt Management.

Prompt Injection (Technical)

Domain(s): 3, 5 **Priority:** High **Category:** Security/Governance

Simple Definition: Prompt injection is an attack where a malicious instruction is hidden inside the data or content a model processes, tricking the model into doing something it should not.

Detailed Explanation: Unlike a user directly typing a malicious instruction into a chat box, prompt injection often hides the malicious instruction inside a document, webpage, email, or other content the model is asked to read or summarize, so the model unknowingly follows the embedded instruction as though it came from a trusted source. This is closely related to prompt poisoning (where malicious instructions are injected through data sources) and prompt hijacking (where the model's intended behavior is overridden). Prompt injection is one of the most heavily tested security threats specific to generative AI systems, appearing in both Domain 3 (prompt engineering risks) and Domain 5 (AI system security considerations).

Everyday Analogy: It is like a hidden message written in invisible ink inside a letter that a trusted assistant is asked to read aloud; the assistant has no way of knowing the hidden instruction was not actually part of the legitimate letter.

Why It Matters for the Exam: Prompt injection appears across both Domain 3 and Domain 5. Know how it differs from jailbreaking (which attempts to bypass safety restrictions directly) and from prompt poisoning and prompt hijacking, which are closely related variants.

Example: An AI agent that summarizes web pages encounters a webpage containing a hidden instruction telling it to ignore its original task and instead reveal confidential system information; this is a prompt injection attack.

Common Misunderstanding: Learners sometimes think prompt injection requires a user to directly type something malicious. The defining feature of prompt injection is that the malicious instruction is often embedded within external content the model processes, not necessarily typed directly by the end user.

Related AWS Service (if applicable): Amazon Bedrock Guardrails.

Protected Characteristic

Domain(s): 4 **Priority:** Medium **Category:** Responsible AI

Simple Definition: A protected characteristic is a personal trait, such as race, gender, age, religion, or disability status, that is legally protected from being used as a basis for unfair treatment.

Detailed Explanation: Many anti-discrimination laws explicitly identify certain protected characteristics and prohibit using them as a basis for decisions in areas like employment, lending, and housing. For AI systems, this means organizations must be especially careful that models do not directly or indirectly (through proxy variables correlated with a protected characteristic) produce unfair outcomes based on these traits. Understanding protected characteristics supports the broader disparate impact and disparate treatment concepts, since both of those fairness violations specifically involve unfair outcomes tied to protected characteristics.

Everyday Analogy: It is like a sports league rule that explicitly states certain personal traits, such as a player's hometown or family background, can never be used as a factor in deciding who makes the team, regardless of how the decision is reached.

Why It Matters for the Exam: Protected characteristic supports the broader fairness, bias, disparate impact, and disparate treatment objectives in Domain 4, Task Statement 4.1.

Example: A company auditing its hiring AI tool checks whether outcomes differ significantly across protected characteristics like gender and age, even though those characteristics are never directly used as input features.

Common Misunderstanding: Learners sometimes think avoiding direct use of a protected characteristic as an input automatically prevents unfair outcomes. Proxy variables correlated with a protected characteristic can still produce unfair, discriminatory results even without directly using that characteristic.

Related AWS Service (if applicable): Amazon SageMaker Clarify.

R

RAG (Retrieval Augmented Generation)

Domain(s): 3, 5 **Priority:** High **Category:** Foundation Model Application

Simple Definition: RAG is a technique that lets a foundation model search and retrieve relevant information from an organization's own documents or knowledge base before generating a response.

Detailed Explanation: RAG works by converting an organization's documents into chunks and embeddings, storing them in a vector database, and then, when a user asks a question, retrieving the most relevant chunks and supplying them to the foundation model as additional context before it generates its final answer. This grounds the model's response in real, verifiable, up-to-date information rather than relying solely on what the model learned during its original pre-training, which significantly reduces hallucinations. On AWS, RAG is most directly implemented through Amazon Bedrock Knowledge Bases, which automates the underlying chunking, embedding, storage, and retrieval steps.

Everyday Analogy: It is like giving an open-book exam to a very smart student instead of a closed-book exam: rather than relying purely on memory, the student can look up specific, accurate facts from approved source material before answering.

Why It Matters for the Exam: RAG is one of the most heavily tested concepts in Domain 3, named directly in Task Statement 3.1, and it connects to grounding and hallucination-reduction discussions in Domain 5. Expect multiple questions referencing RAG, knowledge bases, vector databases, and chunking together.

Example: A company builds a RAG-powered internal assistant using Amazon Bedrock Knowledge Bases so employees can ask natural-language questions and receive answers grounded in the company's actual current policy documents.

Common Misunderstanding: Learners frequently confuse RAG with fine-tuning. RAG retrieves and supplies external information at the moment of the request without changing the underlying model; fine-tuning permanently changes the model's internal parameters through additional training.

Related AWS Service (if applicable): Amazon Bedrock Knowledge Bases; Amazon OpenSearch Service; Amazon Aurora; Amazon Neptune; Amazon RDS for PostgreSQL.

Recall

Domain(s): 1, 3 **Priority:** High **Category:** AI/ML Concept

Simple Definition: Recall measures, out of all the actual positive cases that exist, how many of them the model successfully identified.

Detailed Explanation: Recall is calculated as true positives divided by the sum of true positives and false negatives. High recall means the model successfully catches most of the actual positive cases, even if it occasionally also flags some false positives along the way; this differs from precision, which focuses on the accuracy of the positive predictions actually made. Recall is especially critical in high-stakes scenarios where missing an actual positive case is very costly, such as failing to detect actual fraud or a serious medical condition.

Everyday Analogy: If a smoke detector goes off for nearly every real fire, even if it also occasionally goes off for burnt toast, it has high recall; it rarely misses a genuine emergency, even though some of its alarms turn out to be false alarms.

Why It Matters for the Exam: Recall is named directly alongside accuracy, precision, and F1 score in Task Statement 1.3. Memorize the distinction: recall is about catching all actual positives; precision is about the accuracy of the positive predictions made.

Example: A medical screening tool with high recall successfully flags nearly all patients who actually have a particular condition, even if it occasionally also flags some healthy patients who do not have it.

Common Misunderstanding: Learners often mix up precision and recall under exam pressure. A simple memory trick: recall is about "recalling," or catching, all the real positives that exist; precision is about how "precisely" correct each positive flag turns out to be.

Related AWS Service (if applicable): Amazon SageMaker AI; Amazon Bedrock Model Evaluation.

Regression

Domain(s): 1 **Priority:** High **Category:** AI/ML Concept

Simple Definition: Regression is a machine learning technique used to predict a continuous numerical value, rather than assigning an item to a category.

Detailed Explanation: Regression models learn the relationship between input features and a numeric outcome, allowing them to predict values such as prices, temperatures, demand levels, or sales figures. This differs from classification, which predicts a category label, and clustering, which discovers groupings without predefined labels. Regression is one of the three core technique types named in Task Statement 1.2, and the exam frequently tests whether learners can correctly identify a regression scenario based on whether the desired output is a continuous number.

Everyday Analogy: It is like predicting exactly how many inches of rain will fall tomorrow, rather than simply predicting whether it will be a "rainy day" or a "dry day."

Why It Matters for the Exam: Recognize regression scenarios: anything where the business question is "how much" or "how many," producing a specific numeric prediction rather than a category.

Example: A real estate company uses a regression model to predict the exact sale price of a home based on its size, location, and condition.

Common Misunderstanding: Learners sometimes confuse regression with classification. If the model's output is a specific number along a continuous scale (price, temperature, demand), it is regression; if the output is a category label, it is classification.

Related AWS Service (if applicable): Amazon SageMaker AI.

Regularization

Domain(s): 1 **Priority:** Low **Category:** AI/ML Concept

Simple Definition: Regularization is a set of techniques used during training to prevent a model from becoming too complex and overfitting the training data.

Detailed Explanation: Regularization works by discouraging the model from relying too heavily on any single feature or pattern, often by adding a penalty for excessive complexity into the training process. Dropout is one specific example of a regularization technique used in neural networks. Regularization helps a model generalize better to new, unseen data rather than simply memorizing the quirks of its training set, directly addressing the overfitting problem.

Everyday Analogy: It is like a teacher who deliberately limits how much extra credit a student can earn from any single assignment, encouraging the student to develop genuinely well-rounded skills rather than over-relying on one particular strength.

Why It Matters for the Exam: This is a low-priority, supporting term under the broader overfitting discussion. Recognize it conceptually as a technique to reduce overfitting.

Example: A data science team applies regularization to their model to prevent it from assigning excessive importance to a single noisy feature in their training dataset.

Common Misunderstanding: Learners sometimes think regularization always improves accuracy. Its primary purpose is to improve generalization to new data, which can sometimes mean slightly lower accuracy on the training data itself in exchange for better real-world performance.

Related AWS Service (if applicable): Amazon SageMaker AI.

Reinforcement Learning

Domain(s): 1 **Priority:** High **Category:** AI/ML Concept

Simple Definition: Reinforcement learning is a type of machine learning where an agent learns the best actions to take by receiving rewards or penalties based on the outcomes of its actions.

Detailed Explanation: Unlike supervised learning, which learns from a fixed set of labeled examples, reinforcement learning learns through trial and error within an environment, gradually improving its strategy (called a policy) as it discovers which actions lead to better rewards over time. This approach is well-suited to problems involving sequential decision-making, such as game-playing, robotics, and resource optimization, where the best action often depends on a long sequence of prior decisions rather than a single isolated input. Reinforcement learning is also the conceptual foundation behind RLHF, where human feedback is used as the reward signal to improve a foundation model's behavior.

Everyday Analogy: It is like training a dog with treats and corrections: the dog gradually learns which behaviors lead to a reward and which do not, refining its actions over many repeated attempts rather than being given an explicit instruction manual upfront.

Why It Matters for the Exam: Reinforcement learning is named directly in Task Statement 1.1 as one of the three core types of AI/ML learning, alongside supervised and unsupervised learning. Know its basic mechanism: learning through rewards and penalties rather than labeled examples.

Example: A robotics company uses reinforcement learning to train a robotic arm to improve its precision over thousands of trial-and-error attempts at a manufacturing task.

Common Misunderstanding: Learners sometimes think reinforcement learning requires labeled data like supervised learning does. Reinforcement learning instead learns from reward and penalty signals generated through interaction with an environment, not from a fixed dataset of correct answers.

Related AWS Service (if applicable): Amazon SageMaker AI.

Reinforcement Learning from Human Feedback (RLHF)

Domain(s): 3 **Priority:** High **Category:** Foundation Model Application

Simple Definition: RLHF is a training technique that uses human ratings of a model's outputs as the reward signal to teach the model to produce responses people find more helpful, accurate, or appropriate.

Detailed Explanation: In RLHF, human reviewers rate or rank different model outputs based on quality, helpfulness, or alignment with desired behavior, and these ratings are used to train

the model to favor producing similar high-quality outputs in the future. This technique has been instrumental in making modern foundation models more conversational, helpful, and aligned with human expectations, beyond what raw pre-training on text data alone can achieve. RLHF is named directly in Task Statement 3.3 as part of preparing data and incorporating human quality judgments into the fine-tuning process.

Everyday Analogy: It is like a writing student who improves their essays over time specifically because a teacher reviews drafts and provides feedback on which versions are better, gradually shaping the student's instincts toward what good writing looks like.

Why It Matters for the Exam: RLHF is explicitly named in Task Statement 3.3 as part of fine-tuning data preparation. Know that it uses human feedback as a reward signal within a reinforcement learning framework.

Example: A foundation model provider uses RLHF, having human reviewers rank several candidate responses to the same prompt, to train the model to consistently produce the kind of response humans rated most favorably.

Common Misunderstanding: Learners sometimes think RLHF is the same as simple human-in-the-loop review. Human-in-the-loop review checks individual outputs after the fact; RLHF actually uses human feedback as part of the training process itself to permanently shape the model's future behavior.

Related AWS Service (if applicable): Amazon Bedrock; Amazon SageMaker AI.

Responsible AI

Domain(s): 4 **Priority:** High **Category:** Responsible AI

Simple Definition: Responsible AI is the practice of designing, building, and deploying AI systems in a way that is ethical, fair, safe, and trustworthy.

Detailed Explanation: Responsible AI is the overarching umbrella concept for all of Domain 4, encompassing six named core features: bias, fairness, inclusivity, robustness, safety, and veracity. It requires attention throughout the entire AI/ML lifecycle, from how training data is collected and curated, through how a model is evaluated for bias and explainability, to how it is monitored and governed once deployed. Responsible AI is not a single tool or checklist but an ongoing organizational commitment supported by specific AWS tools, including Amazon SageMaker Clarify, SageMaker Model Cards, Amazon Bedrock Guardrails, and Amazon A2I.

Everyday Analogy: It is like responsible parenting: it is not a single action you take once, but an ongoing, multi-faceted commitment that touches every decision you make, from how you set rules to how you handle mistakes and unexpected situations.

Why It Matters for the Exam: Responsible AI is the title and core theme of all of Domain 4 (14% of the exam). Be ready to define the term broadly and connect it to its six named core features.

Example: A company building a hiring AI tool applies responsible AI principles by testing for bias across demographic groups, documenting the model's limitations in a model card, and providing human review for borderline decisions.

Common Misunderstanding: Learners sometimes think responsible AI is only about avoiding bias. Bias is just one of six named core features; fairness, inclusivity, robustness, safety, and veracity are equally important parts of the broader responsible AI umbrella.

Related AWS Service (if applicable): Amazon SageMaker Clarify; Amazon Bedrock Guardrails; Amazon SageMaker Model Cards.

Robustness

Domain(s): 4 **Priority:** Medium **Category:** Responsible AI

Simple Definition: Robustness is an AI system's ability to keep performing reliably even under unexpected, unusual, or challenging conditions.

Detailed Explanation: A robust model continues to function correctly when faced with noisy input, edge cases, slightly unusual phrasing, or even deliberate adversarial inputs designed to confuse it. Robustness is one of the six named core features of responsible AI, and it connects directly to broader concerns about model reliability and resistance to manipulation. Building robust AI systems often involves testing against a wide range of unusual or edge-case scenarios during development, not just the typical, expected inputs.

Everyday Analogy: It is like a well-built bridge that can handle not just normal everyday traffic, but also unexpected stress like a sudden storm or an unusually heavy vehicle, without failing.

Why It Matters for the Exam: Robustness is named directly in Task Statement 4.1 as one of the six core features of responsible AI. Be ready to recognize scenarios describing reliability under unusual conditions as testing this concept.

Example: A company tests its customer service chatbot against deliberately unusual, oddly phrased, or even adversarial customer messages to confirm it still responds appropriately.

Common Misunderstanding: Learners sometimes confuse robustness with accuracy. Accuracy measures correctness on typical, expected data; robustness specifically measures reliable performance under atypical, unusual, or challenging conditions.

Related AWS Service (if applicable): Amazon Bedrock Guardrails.

Role Prompting

Domain(s): 3 **Priority:** Medium **Category:** Foundation Model Application

Simple Definition: Role prompting is a prompt engineering technique where you instruct a model to respond as if it were a particular persona or expert, such as "act as an experienced tax accountant."

Detailed Explanation: Assigning a role helps shape the tone, vocabulary, and perspective of a model's response, often improving the relevance and appropriateness of the output for a specific audience or use case. Role prompting is frequently included as part of a system prompt, which establishes the overall context and behavior expectations for the model before any individual user

prompts are processed. This is a supporting technique within the broader prompt engineering discipline tested in Domain 3.

Everyday Analogy: It is like asking an actor to step into a specific character before performing a scene, which shapes how they speak and behave for the rest of that performance.

Why It Matters for the Exam: Role prompting is a supporting technique under the broader prompt engineering objectives in Task Statement 3.2. Recognize it as a way to shape tone and perspective through the prompt's framing.

Example: A prompt that begins with "You are a friendly, patient customer support agent for a software company" is using role prompting to shape the tone of the model's subsequent responses.

Common Misunderstanding: Learners sometimes think role prompting changes the model's actual underlying knowledge or capabilities. It only shapes the style, tone, and framing of the response; it does not grant the model new factual knowledge it did not already have.

Related AWS Service (if applicable): Amazon Bedrock; Amazon Bedrock Prompt Management.

ROUGE Score

Domain(s): 3 **Priority:** High **Category:** Foundation Model Application

Simple Definition: ROUGE is an automated metric, most commonly used to evaluate summarization quality, that measures how much overlap exists between a model's generated summary and a human-written reference summary.

Detailed Explanation: ROUGE stands for Recall-Oriented Understudy for Gisting Evaluation, and as the name suggests, it leans toward measuring how much of the important content from a reference summary was successfully captured (recall-oriented) in the generated summary. It compares overlapping words and phrases between the generated and reference text, similar in spirit to BLEU but applied primarily to summarization tasks rather than translation. ROUGE is one of the named automated FM evaluation metrics in Task Statement 3.4, alongside BLEU, BERTScore, and LLM-as-a-judge.

Everyday Analogy: It is like checking how many of the key points from an official meeting summary also appear in a student's own summary of that same meeting, to judge whether they captured the essential gist correctly.

Why It Matters for the Exam: Memorize the pairing: ROUGE = summarization quality, BLEU = translation quality. This pairing is one of the most reliably tested details in Domain 3's evaluation metrics section.

Example: A news organization evaluating an AI-generated article summarization tool uses ROUGE scores to measure how well the generated summaries capture the key content of full human-written reference summaries.

Common Misunderstanding: Learners often forget which metric pairs with which task. The memory aid "ROUGE for the Gist" (since the official name includes "Gisting Evaluation") can help you remember it is associated with summarization.

Related AWS Service (if applicable): Amazon Bedrock Model Evaluation.

S

Semantic Search

Domain(s): 3 **Priority:** Medium **Category:** Foundation Model Application

Simple Definition: Semantic search is a search technique that finds results based on the meaning of a query, rather than requiring an exact keyword match.

Detailed Explanation: Semantic search relies on embeddings: both the search query and the searchable content are converted into vectors, and the search system finds content whose vector is closest in meaning to the query's vector, even if no exact words match. This is a significant improvement over traditional keyword search, which can miss relevant results simply because they use different wording than the query. Semantic search is the underlying retrieval mechanism behind RAG systems, allowing a knowledge base to find the most relevant document chunks for a user's natural-language question.

Everyday Analogy: It is like a librarian who understands what you are actually trying to find and points you to the perfect book, even if the words you used to describe your interest never literally appear anywhere in that book's title or description.

Why It Matters for the Exam: Semantic search is named directly in Task Statement 2.1 as a GenAI use case category and underpins the retrieval step in RAG, tested heavily in Domain 3.

Example: A company's internal search tool uses semantic search to correctly surface a relevant HR policy document even when an employee's search query uses completely different wording than the document itself.

Common Misunderstanding: Learners sometimes think semantic search and traditional keyword search work the same way, just better. Semantic search relies on a fundamentally different mechanism, comparing meaning through vector embeddings rather than matching literal words and phrases.

Related AWS Service (if applicable): Amazon OpenSearch Service; Amazon Kendra; Amazon Bedrock Knowledge Bases.

Semi-Supervised Learning

Domain(s): 1 **Priority:** Low **Category:** AI/ML Concept

Simple Definition: Semi-supervised learning is a machine learning approach that uses a combination of a small amount of labeled data and a much larger amount of unlabeled data to train a model.

Detailed Explanation: Semi-supervised learning is useful in situations where obtaining a large amount of labeled data is expensive or time-consuming, but unlabeled data is abundant and relatively cheap to collect. The small labeled portion helps guide the model's learning process, while the larger unlabeled portion helps the model learn broader underlying structure and patterns in

the data. This approach sits conceptually between fully supervised learning (all labeled data) and fully unsupervised learning (no labeled data at all).

Everyday Analogy: It is like learning a new subject mostly by reading a huge stack of unmarked reference material on your own, but occasionally checking your understanding against a small handful of pages that do have an answer key.

Why It Matters for the Exam: This is a lower-priority, supporting term under the broader supervised versus unsupervised learning discussion in Task Statement 1.1.

Example: A company with only a small number of manually labeled fraud examples but millions of unlabeled transaction records uses semi-supervised learning to build a more effective fraud detection model than labeled data alone would allow.

Common Misunderstanding: Learners sometimes think semi-supervised learning is simply "less accurate" supervised learning. It is a deliberate strategy to make the most of limited labeled data combined with abundant unlabeled data, often producing strong results when fully labeling a dataset would be impractical.

Related AWS Service (if applicable): Amazon SageMaker AI.

SHAP (SHapley Additive exPlanations)

Domain(s): 4 **Priority:** Low **Category:** Responsible AI

Simple Definition: SHAP is a technique that explains a model's prediction by calculating exactly how much each individual input feature contributed to that specific prediction.

Detailed Explanation: SHAP is based on a concept from game theory and provides a mathematically grounded way of fairly distributing "credit" for a prediction among all of the input features that contributed to it. This produces a clear breakdown showing, for example, that a particular loan denial was driven 40% by credit score, 30% by income, and 30% by other factors. SHAP is one of the supporting explainability techniques, alongside LIME, referenced in Domain 4's discussion of transparency and explainability tools, and it is incorporated into Amazon SageMaker Clarify's feature importance capabilities.

Everyday Analogy: It is like a detailed receipt that breaks down exactly how much each ingredient contributed to the final price of a meal, rather than just showing one lump total at the bottom.

Why It Matters for the Exam: This is a low-priority, supporting explainability technique. Recognize it as a feature-importance method, and know that it is incorporated into Amazon SageMaker Clarify.

Example: A data science team uses SHAP values to confirm that a customer's payment history was the single largest contributing factor in their credit risk model's prediction.

Common Misunderstanding: Learners sometimes think SHAP and LIME are identical techniques. Both produce feature-importance explanations, but they use different mathematical approaches; the exam only requires you to recognize both as explainability tools, not to distinguish their internal mathematics.

Related AWS Service (if applicable): Amazon SageMaker Clarify.

Shared Responsibility Model

Domain(s): 5 **Priority:** High **Category:** Security/Governance

Simple Definition: The shared responsibility model is the division of security and compliance duties between AWS and the customer, where AWS secures the underlying cloud infrastructure and the customer secures what they build and configure on top of it.

Detailed Explanation: AWS is responsible for "security of the cloud," meaning the physical data centers, hardware, networking, and the virtualization layer beneath every AWS service. The customer is responsible for "security in the cloud," meaning how they configure access controls, encrypt their data, manage their applications, and use AWS services securely. For AI workloads, this means AWS secures the underlying infrastructure running services like Amazon Bedrock and Amazon SageMaker AI, while the customer remains fully responsible for properly configuring IAM permissions, managing their training data securely, and using guardrails appropriately. Exactly where the line falls shifts depending on the specific service used; for fully managed services, AWS takes on more of the operational burden than for more customer-managed infrastructure.

Everyday Analogy: It is like renting an apartment in a secure building: the landlord is responsible for the building's structural security, like locks on the main entrance and the integrity of the walls, while the tenant is responsible for locking their own apartment door and deciding who they let inside.

Why It Matters for the Exam: The shared responsibility model is one of the most heavily emphasized concepts in Domain 5 and is explicitly listed as recommended prior AWS knowledge in the exam guide. Expect direct questions asking you to identify whether a given responsibility belongs to AWS or the customer.

Example: AWS is responsible for the physical security of the data centers running Amazon Bedrock, while the customer is responsible for properly configuring IAM policies that control who within their organization can access their specific Bedrock applications.

Common Misunderstanding: Learners sometimes think using a fully managed AWS service removes all of their security responsibility. Even with managed services, the customer still retains responsibility for configuration, access management, and how they use the data and outputs.

Related AWS Service (if applicable): Applies across all AWS services; documented in detail through AWS Artifact.

Speech Recognition

Domain(s): 1 **Priority:** High **Category:** AI/ML Concept

Simple Definition: Speech recognition is the AI capability that converts spoken audio into written text.

Detailed Explanation: Speech recognition systems analyze audio waveforms to identify spoken words and phrases, converting them into accurate written transcripts. This capability is also

referred to as automatic speech recognition (ASR), and on AWS it is delivered through Amazon Transcribe. Speech recognition is the opposite direction of text-to-speech conversion, which is delivered through Amazon Polly; the exam frequently tests this directional pairing.

Everyday Analogy: It is like a court stenographer who listens to spoken testimony and types out an accurate written transcript in real time.

Why It Matters for the Exam: Speech recognition is named directly in Task Statement 1.2 as a real-world AI application category. Know Amazon Transcribe as the AWS service, and remember that it works in the opposite direction from Amazon Polly.

Example: A call center uses Amazon Transcribe to automatically generate written transcripts of customer service calls for later analysis and quality review.

Common Misunderstanding: Learners frequently mix up Amazon Transcribe and Amazon Polly. Transcribe converts speech into text; Polly converts text into speech. They are functional opposites.

Related AWS Service (if applicable): Amazon Transcribe.

SSE-KMS (Server-Side Encryption with AWS KMS)

Domain(s): 5 **Priority:** Low **Category:** Security/Governance

Simple Definition: SSE-KMS is a method of encrypting data stored in Amazon S3 where the encryption keys are created and managed using AWS KMS, giving the customer more control over key policies and auditing.

Detailed Explanation: SSE-KMS provides additional benefits compared to default S3 encryption, including the ability to use a customer managed key, detailed audit logging of key usage through AWS CloudTrail, and more granular control over who can use a specific key to decrypt data. For AI workloads handling sensitive training data or knowledge base documents, SSE-KMS offers stronger governance and auditability than the more basic SSE-S3 option. This is a lower-priority, technical supporting detail under the broader encryption at rest topic in Domain 5.

Everyday Analogy: It is like storing valuables in a safe deposit box where the bank can provide a detailed log of exactly who accessed the key and when, rather than a simple home safe with no such tracking.

Why It Matters for the Exam: This is a low-priority, supporting detail. Recognize SSE-KMS as a more controllable, auditable encryption option compared to SSE-S3.

Example: A healthcare company uses SSE-KMS to encrypt its Amazon S3-based training data, allowing for detailed audit logging of every time the encryption key is used.

Common Misunderstanding: Learners sometimes think SSE-KMS and SSE-S3 provide identical capabilities. SSE-KMS provides more granular access control and detailed audit logging through AWS KMS, while SSE-S3 uses simpler, AWS-managed encryption with less customer control.

Related AWS Service (if applicable): AWS KMS; Amazon S3.

SSE-S3 (Server-Side Encryption with Amazon S3-Managed Keys)

Domain(s): 5 **Priority:** Low **Category:** Security/Governance

Simple Definition: SSE-S3 is the default, simplest method of encrypting data stored in Amazon S3, using encryption keys that Amazon S3 itself manages automatically.

Detailed Explanation: SSE-S3 provides automatic encryption at rest for objects stored in Amazon S3 without requiring the customer to manage any encryption keys themselves, making it a low-effort baseline option for protecting stored data, including AI training datasets and model artifacts. While simple and effective, SSE-S3 offers less granular control and auditability than SSE-KMS, which uses AWS KMS-managed keys instead. This is a lower-priority, supporting detail under the broader encryption at rest topic.

Everyday Analogy: It is like a hotel room safe that automatically locks itself and manages its own combination without requiring the guest to do anything extra, compared to a personal lockbox where the guest manages their own specific key.

Why It Matters for the Exam: This is a low-priority, supporting detail. Recognize SSE-S3 as the simpler, default encryption option compared to the more controllable SSE-KMS.

Example: A small team enables SSE-S3 to quickly and easily encrypt the Amazon S3 bucket storing their AI application's log files, without needing to set up and manage their own encryption keys.

Common Misunderstanding: Learners sometimes think SSE-S3 means data is unencrypted unless a customer takes extra steps. SSE-S3 still provides genuine encryption at rest; it simply uses AWS-managed keys rather than customer managed keys, trading some control for simplicity.

Related AWS Service (if applicable): Amazon S3.

Supervised Learning

Domain(s): 1 **Priority:** High **Category:** AI/ML Concept

Simple Definition: Supervised learning is a machine learning approach where a model learns from labeled data, meaning each training example includes the correct answer.

Detailed Explanation: During training, a supervised learning algorithm compares its predictions against the known correct labels and adjusts itself to reduce the gap between its predictions and the actual ground truth. This approach is well-suited to problems like classification and regression, where historical examples with known correct outcomes are available. Supervised learning is one of the three core learning types named in Task Statement 1.1, alongside unsupervised and reinforcement learning, and it is the approach most commonly associated with traditional predictive business applications.

Everyday Analogy: It is like studying for a test using flashcards that have the correct answer printed on the back, so you can immediately check and correct your understanding after each guess.

Why It Matters for the Exam: Supervised learning is foundational vocabulary and is the learning type most closely associated with classification and regression. Be ready to recognize supervised learning scenarios based on the presence of labeled historical data.

Example: A bank trains a supervised learning model using historical loan data labeled with whether each loan was repaid or defaulted.

Common Misunderstanding: Learners sometimes assume all machine learning is supervised learning. Unsupervised learning (working with unlabeled data) and reinforcement learning (learning through rewards and penalties) are equally valid, distinct approaches.

Related AWS Service (if applicable): Amazon SageMaker AI.

Synthetic Data

Domain(s): 2 **Priority:** Medium **Category:** GenAI Concept

Simple Definition: Synthetic data is artificially generated data created to resemble real data, often used to supplement or replace real datasets for training or testing AI models.

Detailed Explanation: Synthetic data can be useful when real data is scarce, expensive to collect, or too sensitive to use directly due to privacy concerns, since it can mimic the statistical patterns of real data without exposing any actual personal or confidential information. Generative AI models themselves can be used to create synthetic data, since they are specifically designed to generate new content that follows learned patterns. Synthetic data must be carefully evaluated to ensure it accurately represents real-world diversity and does not introduce its own unintended biases or quality issues.

Everyday Analogy: It is like using realistic, computer-generated crash test dummies and simulated scenarios to test car safety, rather than relying exclusively on real human test subjects in every single scenario.

Why It Matters for the Exam: Synthetic data is a supporting concept under the broader GenAI use case discussion in Domain 2, and it connects to privacy-enhancing approaches discussed in Domain 5.

Example: A healthcare AI team uses synthetic patient data, generated to mirror real statistical patterns without containing any actual patient information, to test a new diagnostic model without privacy risk.

Common Misunderstanding: Learners sometimes think synthetic data is automatically free of bias just because it is artificially generated. If the generative process used to create synthetic data was itself trained on biased real-world data, the resulting synthetic data can still reflect and even amplify those same biases.

Related AWS Service (if applicable): Amazon Bedrock.

System Prompt

Domain(s): 3 **Priority:** Medium **Category:** Foundation Model Application

Simple Definition: A system prompt is a set of instructions given to a foundation model that establishes its overall behavior, tone, and constraints before any individual user begins interacting with it.

Detailed Explanation: A system prompt typically defines the model's role (sometimes through role prompting), the boundaries of what it should and should not discuss, formatting expectations, and any other persistent context that should apply across an entire conversation or application. This differs from a user prompt, which is the specific, individual request a person makes within the context already established by the system prompt. Protecting the contents of a system prompt from being revealed to end users is one specific concern under prompt exposure risk, named in Task Statement 3.2.

Everyday Analogy: A system prompt is like a job description and code of conduct given to a new customer service representative before their shift begins, while a user prompt is each individual customer's specific question once the representative is already on duty.

Why It Matters for the Exam: System prompt and user prompt are supporting concepts under the broader prompt structure and engineering discussion in Task Statement 3.2.

Example: A company sets a system prompt instructing its chatbot to always respond in a friendly, professional tone and to never discuss competitor products, which then applies to every individual user prompt that follows.

Common Misunderstanding: Learners sometimes think a system prompt is visible to end users. System prompts are typically hidden from the end user and only seen by developers configuring the application, which is part of why prompt exposure (accidentally revealing the system prompt) is considered a security risk.

Related AWS Service (if applicable): Amazon Bedrock; Amazon Bedrock Prompt Management.

T

Temperature (Inference Parameter)

Domain(s): 3 **Priority:** High **Category:** Foundation Model Application

Simple Definition: Temperature is an inference parameter that controls how creative or random a foundation model's output is, compared to how predictable or conservative it is.

Detailed Explanation: A low temperature setting makes the model favor the most statistically likely next word at each step, producing more consistent, predictable, and focused output. A high temperature setting allows the model to occasionally choose less likely words, producing more varied, creative, and sometimes surprising output, but also increasing the risk of less coherent or accurate responses. Choosing the right temperature depends entirely on the use case: factual, customer-facing applications typically benefit from lower temperature, while creative writing or brainstorming applications often benefit from higher temperature.

Everyday Analogy: It is like the difference between a cautious chef who always follows a recipe exactly the same way every time (low temperature) versus an adventurous chef who likes to experiment with unexpected ingredient combinations each time they cook (high temperature).

Why It Matters for the Exam: Temperature is explicitly named in Task Statement 3.1 as a key inference parameter you must understand the effect of. Expect a scenario question asking whether to raise or lower temperature for a given business need.

Example: A legal document summarization tool uses a low temperature setting to ensure consistent, predictable, fact-focused summaries every time.

Common Misunderstanding: Learners sometimes think temperature affects the model's underlying knowledge or accuracy in a simple, linear way. It actually affects the randomness of word selection during generation; very high temperature can reduce coherence and factual reliability rather than simply making output "more creative" in a purely positive sense.

Related AWS Service (if applicable): Amazon Bedrock.

Text-to-Code

Domain(s): 2 **Priority:** Medium **Category:** GenAI Concept

Simple Definition: Text-to-code is a generative AI capability where a model produces working program code based on a natural-language description of what the code should do.

Detailed Explanation: Text-to-code models are trained on large amounts of code paired with descriptions, comments, and documentation, allowing them to translate a plain-language request, such as "write a function that sorts a list of numbers," into functioning code in a requested programming language. This capability powers tools like AI coding assistants, which can accelerate development by generating boilerplate code, suggesting fixes, or even drafting entire functions from a description. On AWS, this capability is most closely associated with Amazon Q and Kiro.

Everyday Analogy: It is like describing exactly what you want a custom piece of furniture to look like and do, and having a skilled craftsperson translate that description directly into a finished, functional product.

Why It Matters for the Exam: Text-to-code is named as a GenAI use case category in Task Statement 2.1. Know that it connects to AWS developer tools like Amazon Q and Kiro.

Example: A developer describes a desired data validation function in plain English, and a text-to-code tool generates the corresponding working code automatically.

Common Misunderstanding: Learners sometimes think text-to-code tools always produce perfect, bug-free code. Like any generative AI output, code generation can contain errors or hallucinated logic and should be reviewed before being used in production.

Related AWS Service (if applicable): Amazon Q; Kiro.

Text-to-Image

Domain(s): 2 **Priority:** Medium **Category:** GenAI Concept

Simple Definition: Text-to-image is a generative AI capability where a model creates a new image based on a written text description.

Detailed Explanation: Text-to-image models, frequently built using diffusion model architectures, learn the relationship between descriptive language and visual concepts from massive paired datasets of images and their captions. A user provides a text prompt describing the desired image, and the model generates a corresponding original image that matches the description as closely as

it can. This is one of the most well-known and widely demonstrated generative AI capabilities, alongside text-to-text generation.

Everyday Analogy: It is like describing a scene in vivid detail to a skilled illustrator and having them paint exactly what you described, without you ever having to pick up a brush yourself.

Why It Matters for the Exam: Text-to-image is named as a use case category in Task Statement 2.1, and it connects to the broader diffusion model and multimodal model concepts.

Example: A marketing team generates several custom promotional images for a new product using a text-to-image foundation model accessed through Amazon Bedrock.

Common Misunderstanding: Learners sometimes think text-to-image models can perfectly reproduce any specific, highly detailed request on the first attempt. In practice, multiple prompt iterations are often needed to achieve a desired result, similar to prompt engineering for text generation.

Related AWS Service (if applicable): Amazon Bedrock.

Text-to-Text

Domain(s): 2 **Priority:** Medium **Category:** GenAI Concept

Simple Definition: Text-to-text is a generative AI capability where a model takes written text as input and produces new written text as output, such as summarizing, translating, or answering a question.

Detailed Explanation: Text-to-text covers the broad core capability of most LLMs: taking a prompt as input and generating a relevant text response as output. This includes tasks like summarization, translation, question answering, and content drafting. Text-to-text is the most common and widely applicable generative AI capability category and underlies the majority of chatbot and AI assistant applications.

Everyday Analogy: It is like handing a skilled writer a topic or a source document and asking them to produce a new piece of writing, whether that is a summary, a translation, or an entirely new piece of content based on what they were given.

Why It Matters for the Exam: Text-to-text is named as a use case category in Task Statement 2.1, representing the core, most common type of generative AI application.

Example: A customer service chatbot uses text-to-text generation to read a customer's written question and produce a relevant, helpful written response.

Common Misunderstanding: Learners sometimes think text-to-text only refers to simple text transformation tasks like translation. It also covers more complex generative tasks like open-ended content drafting and conversational responses.

Related AWS Service (if applicable): Amazon Bedrock.

Token

Domain(s): 2, 3 **Priority:** High **Category:** GenAI Concept

Simple Definition: A token is a small chunk of text, such as a word or part of a word, that a foundation model processes as a single unit when reading input or generating output.

Detailed Explanation: Before a model can process text, the text must be broken down into tokens through a process called tokenization. Tokens are not always whole words; common words might be a single token, while longer or less common words might be split into multiple tokens. The number of tokens in a prompt and a response directly determines both the cost of an interaction, since most foundation models use token-based pricing, and whether the input fits within the model's context window.

Everyday Analogy: Think of tokens like puzzle pieces that text gets broken into before a model can process it; some pieces represent a whole common word, while others represent just a fragment of a longer or unusual word.

Why It Matters for the Exam: Token is one of the most foundational vocabulary terms in Domain 2, directly connecting to token-based pricing, context window limits, and overall cost management for GenAI applications.

Example: A 500-word customer email might be broken down into roughly 650-700 tokens once submitted to a foundation model, since some words get split into multiple token pieces.

Common Misunderstanding: Learners sometimes assume one token always equals exactly one word. In practice, the relationship between words and tokens varies, and longer or unusual words are often split into multiple tokens.

Related AWS Service (if applicable): Amazon Bedrock.

Tokenization

Domain(s): 2 **Priority:** Medium **Category:** GenAI Concept

Simple Definition: Tokenization is the process of breaking raw text into individual tokens before a foundation model can process it.

Detailed Explanation: Tokenization is the first technical step in how a foundation model handles any input: the raw text a user submits is converted into a sequence of tokens using a tokenizer specific to that model, and these tokens are what the model actually processes and generates, rather than working with raw characters or whole words directly. Different foundation models may use different tokenization schemes, which means the exact same piece of text could be broken into a different number of tokens depending on which model is being used. This process directly determines token-based pricing and context window consumption.

Everyday Analogy: It is like a translator first breaking a sentence down into individual recognizable building blocks before they can begin translating it piece by piece into another language.

Why It Matters for the Exam: Tokenization is named directly in Task Statement 2.1 as foundational GenAI vocabulary, supporting your understanding of tokens, context windows, and pricing.

Example: Two different foundation models might tokenize the exact same sentence into slightly different numbers of tokens, depending on each model's specific tokenization scheme.

Common Misunderstanding: Learners sometimes think tokenization is identical across every foundation model. Tokenization schemes can vary between models and providers, meaning token counts for the same text are not necessarily consistent across different models.

Related AWS Service (if applicable): Amazon Bedrock.

Tool Use (Function Calling)

Domain(s): 2, 3 **Priority:** Medium **Category:** GenAI Concept

Simple Definition: Tool use, also called function calling, is the ability of an AI agent or model to call external tools, APIs, or functions to gather information or take action beyond simply generating text.

Detailed Explanation: Tool use is what transforms a model from something that can only generate plausible-sounding text into something that can actually look up real-time information, perform calculations, query a database, or trigger an action in another system. The model decides when a tool is needed, determines which tool to call and with what input, and then incorporates the tool's result into its final response. Tool usage is named directly as a foundational agentic AI concept in Task Statement 2.1, and it is a core capability that enables AI agents to complete multi-step, real-world tasks.

Everyday Analogy: It is like an assistant who, instead of just guessing an answer from memory, picks up the phone, calls the right department, gets the actual current information, and then gives you an accurate, up-to-date answer.

Why It Matters for the Exam: Tool usage is explicitly named in Task Statement 2.1 as a foundational agentic AI concept. Connect it to the broader discussion of AI agents and orchestration in Domain 3.

Example: A travel-booking AI agent uses tool calling to check real-time flight availability through an airline's API before recommending specific flight options to a user.

Common Misunderstanding: Learners sometimes think any AI response that includes specific, accurate-seeming data automatically used tool calling. Without explicit tool use, a model can only draw on patterns learned during training, which can result in outdated or hallucinated information rather than genuinely real-time, verified data.

Related AWS Service (if applicable): Amazon Bedrock Agents; Amazon Bedrock AgentCore; Strands Agents.

Top-K Sampling

Domain(s): 2 **Priority:** Low **Category:** GenAI Concept

Simple Definition: Top-K sampling is a technique that limits a model's word choices at each step to only the K most likely next words, before randomly selecting among them.

Detailed Explanation: Top-K sampling works alongside temperature to control output randomness: rather than considering every possible next word, the model narrows its options down to a fixed number (K) of the most probable candidates and then samples from that smaller set. A smaller K value produces more focused, predictable output, while a larger K value allows for more variety. This is a low-priority, technical inference parameter on the exam, secondary in importance to temperature.

Everyday Analogy: It is like a multiple-choice test that only shows you the top five most plausible answer choices instead of an open-ended blank, narrowing your options before you make your final pick.

Why It Matters for the Exam: This is a low-priority, supporting inference parameter. The exam more heavily emphasizes temperature; recognize top-K only at a definitional level.

Example: A developer sets a relatively small top-K value to keep a customer-facing chatbot's responses focused and predictable.

Common Misunderstanding: Learners sometimes confuse top-K sampling with temperature. Top-K limits how many candidate words are even considered at each step; temperature affects how randomly the model chooses among whichever candidates are being considered.

Related AWS Service (if applicable): Amazon Bedrock.

Top-P Sampling

Domain(s): 2 **Priority:** Low **Category:** GenAI Concept

Simple Definition: Top-P sampling, also called nucleus sampling, is a technique that limits a model's word choices to a dynamically sized group of the most likely next words whose combined probability reaches a chosen threshold, before randomly selecting among them.

Detailed Explanation: Unlike top-K sampling, which always considers a fixed number of candidate words, top-P sampling adjusts how many candidates are considered based on the actual probability distribution at each specific step; if one word is overwhelmingly likely, very few alternatives might be considered, while in a more uncertain situation, many more candidates might be included. This makes top-P a more adaptive method of controlling output randomness compared to the fixed-size approach of top-K sampling. This is a low-priority, technical supporting term on the exam.

Everyday Analogy: Rather than always offering exactly five multiple-choice options like top-K, top-P is like offering just enough plausible options to cover most of the realistic possibilities, sometimes two options, sometimes ten, depending on how clear-cut the situation is.

Why It Matters for the Exam: This is a low-priority, supporting inference parameter, secondary in exam emphasis to temperature.

Example: A creative writing tool uses a higher top-P value to allow for a wider, more varied range of plausible word choices at each step.

Common Misunderstanding: Learners sometimes think top-P and top-K work identically. Top-K uses a fixed count of candidates regardless of the situation; top-P dynamically adjusts the candidate pool size based on the actual probability distribution at each step.

Related AWS Service (if applicable): Amazon Bedrock.

Toxicity (AI Outputs)

Domain(s): 4, 5 **Priority:** High **Category:** Security/Governance

Simple Definition: Toxicity refers to harmful, offensive, hateful, or otherwise inappropriate content that an AI model might generate.

Detailed Explanation: Toxic outputs can include hate speech, harassment, violent content, or other material that could harm users, damage an organization's reputation, or create legal liability. Toxicity detection is one of the named security and privacy considerations in Domain 5, Task Statement 5.1, and it is operationally addressed through tools like Amazon Bedrock Guardrails, which can automatically detect and filter toxic content from both user inputs and model outputs before it reaches an end user. Managing toxicity is closely connected to the broader safety feature of responsible AI in Domain 4.

Everyday Analogy: It is like a moderator at a public event who steps in immediately if a speaker starts using offensive or harmful language, preventing it from reaching the audience.

Why It Matters for the Exam: Toxicity detection is explicitly named in Domain 5, Task Statement 5.1, and connects directly to Amazon Bedrock Guardrails' content filtering capability.

Example: A social media content generation tool uses toxicity detection through Bedrock Guardrails to automatically block any generated post containing hateful or harassing language.

Common Misunderstanding: Learners sometimes think toxicity detection only matters for obviously offensive content like profanity. It also covers more subtle forms of harm, including hate speech directed at specific groups and content that could incite violence or harassment.

Related AWS Service (if applicable): Amazon Bedrock Guardrails.

Training

Domain(s): 1 **Priority:** High **Category:** AI/ML Concept

Simple Definition: Training is the process of teaching a machine learning model to recognize patterns by exposing it to data and adjusting its internal parameters to improve its predictions.

Detailed Explanation: During training, an algorithm processes a dataset, makes predictions, compares those predictions to known correct outcomes (in supervised learning) or evaluates patterns on its own (in unsupervised learning), and adjusts its internal parameters to reduce errors or improve performance. Training typically happens over multiple passes through the dataset, called epochs, and is influenced by hyperparameters like learning rate and batch size. Once training is complete, the resulting trained model can be used during inference to make predictions on new, unseen data.

Everyday Analogy: Training is like a student studying and practicing extensively with example problems before an exam, gradually improving their understanding and skill with each round of practice.

Why It Matters for the Exam: Training is one of the most fundamental terms tested in Task Statement 1.1. Be ready to clearly distinguish training (learning from data) from inference (applying that learning to new data).

Example: A company trains a new model on five years of historical sales data before deploying it to predict future demand.

Common Misunderstanding: Learners sometimes think training and inference are interchangeable terms for "using" a model. Training is specifically the learning phase using historical data; inference is the later phase of applying that already-learned knowledge to new data.

Related AWS Service (if applicable): Amazon SageMaker AI.

Transparency

Domain(s): 4 **Priority:** High **Category:** Responsible AI

Simple Definition: Transparency is the openness with which an organization shares how an AI system works, what data it was built on, and how it is used.

Detailed Explanation: Transparency operates at a broader, organizational level than explainability, which focuses specifically on explaining individual decisions; transparency instead covers general openness about a system's purpose, capabilities, limitations, and governance. AWS tools like Amazon SageMaker Model Cards directly support transparency by providing clear, accessible documentation about a model's intended use, training data, and known limitations. Transparency also connects to AI decision transparency, a human-centered design principle ensuring that people understand what role AI played in a decision that affects them.

Everyday Analogy: It is like a restaurant that openly posts its ingredient sourcing and nutritional information for every dish, rather than keeping that information hidden and only revealing it if specifically pressed.

Why It Matters for the Exam: Transparency is one of the six core responsible AI features named in Task Statement 4.1, and it is the core theme of Task Statement 4.2 alongside explainability. Know that transparency and explainability are related but distinct.

Example: A company publishes a model card for its AI-powered loan decision tool, openly documenting the model's training data, evaluation results, and known limitations for both regulators and customers to review.

Common Misunderstanding: Learners sometimes treat transparency and explainability as exact synonyms. Transparency is about general openness regarding how a system works overall; explainability is specifically about being able to explain individual outputs or decisions.

Related AWS Service (if applicable): Amazon SageMaker Model Cards.

U

Underfitting

Domain(s): 1, 4 **Priority:** High **Category:** AI/ML Concept

Simple Definition: Underfitting happens when a model is too simple to capture the real patterns in the data, resulting in poor performance on both the training data and new data.

Detailed Explanation: An underfit model fails to learn the meaningful relationships present in the training data, often because it is too simple, was not trained long enough, or was not given enough relevant features to work with. Unlike overfitting, where a model performs well on training data but poorly on new data, an underfit model performs poorly across the board, including on the very data it was trained on. Underfitting is associated with high bias, meaning the model makes overly simplistic assumptions that do not match the true complexity of the underlying patterns.

Everyday Analogy: It is like a student who barely studied at all and performs poorly not just on the real exam, but even on simple practice questions drawn directly from the same material they were supposed to learn.

Why It Matters for the Exam: Underfitting is named directly in Task Statement 1.1 and again in Domain 4's bias and variance discussion. Know that underfitting is associated with high bias, the opposite pairing from overfitting's association with high variance.

Example: A overly simplistic linear model fails to capture the true, more complex relationship between advertising spend and sales, resulting in poor predictions even on the historical data it was trained on.

Common Misunderstanding: Learners sometimes think underfitting and overfitting are simply two names for the same general problem of "bad" model performance. They are opposite failure modes: underfitting means the model is too simple and misses real patterns (high bias); overfitting means the model is too complex and memorizes noise (high variance).

Related AWS Service (if applicable): Amazon SageMaker AI.

Unsupervised Learning

Domain(s): 1 **Priority:** High **Category:** AI/ML Concept

Simple Definition: Unsupervised learning is a machine learning approach where a model finds patterns and structure in data that has no labels or predefined correct answers.

Detailed Explanation: Since there is no labeled "correct answer" to learn from, unsupervised learning algorithms instead look for natural patterns, similarities, or structures within the data itself, such as grouping similar items together (clustering) or identifying unusual outliers (anomaly detection). This makes unsupervised learning especially valuable when labeled data is scarce, expensive, or simply does not exist for a particular problem, such as discovering previously unknown customer segments. Unsupervised learning is one of the three core learning types named in Task Statement 1.1, alongside supervised and reinforcement learning.

Everyday Analogy: It is like being given a giant box of mixed photographs with no labels or captions and being asked to sort them into sensible groups based purely on what you notice they have in common.

Why It Matters for the Exam: Recognize unsupervised learning scenarios: anything involving unlabeled data where the goal is to discover hidden structure, such as customer segmentation or anomaly detection.

Example: A retailer uses unsupervised learning to discover previously unknown natural groupings within their customer base, without having predefined any segments in advance.

Common Misunderstanding: Learners sometimes think unsupervised learning cannot be useful without "the right answer" already defined. Discovering previously unknown structure or patterns, without any predefined correct answer, is precisely the strength and purpose of unsupervised learning.

Related AWS Service (if applicable): Amazon SageMaker AI.

User Prompt

Domain(s): 3 **Priority:** Low **Category:** Foundation Model Application

Simple Definition: A user prompt is the specific request or question a person submits to a foundation model during an individual interaction.

Detailed Explanation: A user prompt operates within the broader context already established by the system prompt, and it represents the actual, specific task or question for that particular turn of the conversation. In a multi-turn conversation, multiple user prompts can build on each other along with the model's previous responses, all within the limits of the context window. Understanding the relationship between system prompts and user prompts is a supporting detail within the broader prompt structure discussion in Task Statement 3.2.

Everyday Analogy: If the system prompt is the job description given to a customer service representative before their shift, the user prompt is each individual customer's specific question once that representative is already on duty.

Why It Matters for the Exam: This is a lower-priority, supporting term that complements the more heavily tested system prompt concept.

Example: Within a chatbot already configured with a system prompt establishing a friendly, professional tone, a customer's specific question, "Can I return this item after 30 days?", is the user prompt.

Common Misunderstanding: Learners sometimes think user prompts can override anything set in a system prompt. Well-designed applications typically prioritize the constraints and behavior set in the system prompt, even when a user's specific request might otherwise push toward different behavior.

Related AWS Service (if applicable): Amazon Bedrock.

V

Variance (ML)

Domain(s): 1, 4 **Priority:** High **Category:** AI/ML Concept

Simple Definition: Variance measures how much a model's predictions would change if it were trained on a different sample of training data.

Detailed Explanation: A model with high variance is very sensitive to the specific data it was trained on, meaning small changes in the training set could lead to noticeably different predictions; this is closely associated with overfitting, since an overfit model has essentially memorized the quirks of one particular training set. A model with low variance produces more consistent predictions regardless of small changes to the training data, but if combined with high bias, it may be consistently wrong in a systematic way (underfitting). Balancing bias and variance is a foundational concept in understanding model performance and fairness.

Everyday Analogy: A high-variance model is like a student whose test scores swing wildly depending on which exact practice questions they happened to study, while a low-variance student performs consistently regardless of which specific practice questions they used.

Why It Matters for the Exam: Variance is named directly in the term list and pairs with bias in Domain 4's discussion of effects on demographic groups and model fairness. Memorize: high variance is associated with overfitting; high bias is associated with underfitting.

Example: A complex model trained on a small dataset shows high variance, producing noticeably different predictions when retrained on a slightly different sample of the same underlying population.

Common Misunderstanding: Learners sometimes think reducing variance always improves a model. Reducing variance without addressing bias can simply shift the problem toward underfitting; the goal is to find an appropriate balance between bias and variance for the specific problem.

Related AWS Service (if applicable): Amazon SageMaker AI; Amazon SageMaker Clarify.

Vector

Domain(s): 2, 3 **Priority:** High **Category:** GenAI Concept

Simple Definition: A vector is a list of numbers that represents a piece of data, such as a word, sentence, or image, in a way that captures its meaning or characteristics.

Detailed Explanation: A vector is the actual numeric output produced when something is converted into an embedding; each number in the vector corresponds to a position along one of many dimensions in a mathematical space, and the overall combination of numbers captures the underlying meaning of the original content. Vectors that are mathematically close together (using measures like cosine similarity) represent content with similar meaning, which is the foundation of semantic search and RAG retrieval. Vectors must be stored somewhere efficient enough to search quickly across potentially millions of entries, which is the role of a vector database.

Everyday Analogy: A vector is like a precise set of GPS coordinates for an idea: just as coordinates pinpoint an exact physical location, a vector pinpoints an exact "location" of meaning within a mathematical space of concepts.

Why It Matters for the Exam: Vector is named directly in Task Statement 2.1 as foundational GenAI vocabulary and underpins the vector database and RAG concepts in Domain 3.

Example: The sentence "The cat sat on the mat" might be converted into a vector of several hundred numbers that captures its overall meaning for comparison against other sentences.

Common Misunderstanding: Learners sometimes think "vector" and "embedding" are entirely

separate concepts. An embedding is the broader process and concept of representing data numerically; a vector is the specific list of numbers that results from that process.

Related AWS Service (if applicable): Amazon OpenSearch Service; Amazon Aurora; Amazon Neptune; Amazon RDS for PostgreSQL.

Vector Database

Domain(s): 3 **Priority:** High **Category:** GenAI Concept

Simple Definition: A vector database is a specialized database designed to efficiently store and search large numbers of vectors, allowing fast retrieval of the most similar items to a given query.

Detailed Explanation: Vector databases are purpose-built to perform similarity searches across millions or billions of vectors quickly, which is essential for RAG applications that need to instantly find the most relevant document chunks for a user's question. Task Statement 3.1 specifically names the AWS services capable of storing vector embeddings for RAG: Amazon OpenSearch Service, Amazon Aurora, Amazon Neptune, and Amazon RDS for PostgreSQL (using the pgvector extension). Each of these offers a different balance of features, with OpenSearch Service being a dedicated search and analytics engine, while Aurora, Neptune, and RDS for PostgreSQL add vector capability on top of their existing relational or graph database functions.

Everyday Analogy: It is like a highly specialized library catalog system designed specifically to instantly find every book most similar in theme to a description you provide, rather than a general filing system that only searches by exact title or author name.

Why It Matters for the Exam: Task Statement 3.1 explicitly names the four AWS vector storage services. This is one of the most reliably tested specific service-name lists on the exam; memorize all four.

Example: A company building a RAG-powered customer support tool stores its document embeddings in Amazon OpenSearch Service so it can quickly retrieve the most relevant content for each customer question.

Common Misunderstanding: Learners sometimes think only Amazon OpenSearch Service can store vectors for RAG. Amazon Aurora, Amazon Neptune, and Amazon RDS for PostgreSQL (with pgvector) are also explicitly named as valid vector storage options.

Related AWS Service (if applicable): Amazon OpenSearch Service; Amazon Aurora; Amazon Neptune; Amazon RDS for PostgreSQL.

Veracity

Domain(s): 4 **Priority:** Medium **Category:** Responsible AI

Simple Definition: Veracity is the responsible AI principle that AI systems should produce truthful, accurate outputs and not present false information as if it were fact.

Detailed Explanation: Veracity is one of the six named core features of responsible AI in Task Statement 4.1, and it is closely tied to the problem of hallucination, since a hallucinating model

directly violates the principle of veracity by confidently presenting fabricated information as though it were true. Supporting veracity in practice involves techniques like grounding (anchoring responses in verified sources), output validation (checking outputs against defined criteria), and confidence scoring (signaling how certain a model is about its answer).

Everyday Analogy: It is like a news anchor who has a professional and ethical obligation to only report stories that have been fact-checked and verified, rather than reporting unverified rumors with the same tone of confidence as established facts.

Why It Matters for the Exam: Veracity is named directly in Task Statement 4.1 as one of the six core responsible AI features. Connect it conceptually to hallucination and grounding, which are tested across multiple domains.

Example: A company evaluates its AI research assistant's veracity by checking how often it correctly cites real, verifiable sources versus inventing plausible-sounding but false citations.

Common Misunderstanding: Learners sometimes think veracity and accuracy are exactly the same concept. Accuracy is a specific, measurable model performance metric; veracity is the broader responsible AI principle concerned with whether outputs are truthful and not misleadingly presented as fact.

Related AWS Service (if applicable): Amazon Bedrock Knowledge Bases (RAG grounding); Amazon Bedrock Guardrails.

VPC (Virtual Private Cloud)

Domain(s): 5 **Priority:** Low **Category:** AWS Service

Simple Definition: Amazon VPC is a logically isolated private network within AWS where customers can deploy their resources with full control over networking and security settings.

Detailed Explanation: A VPC lets an organization define its own private network environment within AWS, including subnets, routing rules, and security groups, keeping resources isolated from the public internet unless explicitly configured otherwise. For AI workloads, deploying resources within a VPC helps keep sensitive training data, model endpoints, and applications off the public internet, supporting both security and compliance objectives. AWS PrivateLink works alongside a VPC to provide private connectivity specifically to AWS services without that traffic ever needing to leave the private network environment.

Everyday Analogy: A VPC is like a private, gated business park where each company has full control over its own internal roads, gates, and security checkpoints, separate from the public streets outside.

Why It Matters for the Exam: VPC supports the broader network security discussion in Domain 5. Know the relationship between VPC (the overall private network) and PrivateLink (a specific private connectivity feature within it).

Example: A healthcare company deploys its AI inference endpoints within a VPC to keep patient-related traffic isolated from the public internet.

Common Misunderstanding: Learners sometimes think a VPC alone guarantees private connectivity to AWS services. Without an additional feature like AWS PrivateLink, traffic to AWS

services from within a VPC may still need to traverse the public internet unless specifically configured otherwise.

Related AWS Service (if applicable): AWS PrivateLink.

VPC Endpoint

Domain(s): 5 **Priority:** Low **Category:** Security/Governance

Simple Definition: A VPC endpoint is a connection point that allows resources inside a VPC to privately access supported AWS services without going through the public internet.

Detailed Explanation: VPC endpoints are the underlying technical mechanism that AWS PrivateLink uses to provide private connectivity. There are different types of VPC endpoints depending on the target service, but the core concept is consistent: traffic stays within the AWS network rather than routing out to the public internet and back. For AI applications handling sensitive data, using VPC endpoints to connect to services like Amazon Bedrock or Amazon S3 reduces exposure and supports stricter security and compliance requirements.

Everyday Analogy: It is like a private, direct hallway connecting two specific offices within the same secure building, rather than requiring someone to exit the building and walk around the block to get between them.

Why It Matters for the Exam: This is a lower-priority, technical supporting detail under the broader VPC and PrivateLink discussion in Domain 5.

Example: A company configures a VPC endpoint so that its application can privately call Amazon S3 to retrieve training data without that traffic crossing the public internet.

Common Misunderstanding: Learners sometimes think a VPC endpoint and AWS PrivateLink are two completely separate, unrelated features. VPC endpoints are the specific underlying mechanism that enables the private connectivity that AWS PrivateLink provides.

Related AWS Service (if applicable): AWS PrivateLink.

Z

Zero-Shot Learning

Domain(s): 2 **Priority:** Medium **Category:** GenAI Concept

Simple Definition: Zero-shot learning is a model's ability to correctly perform a task it was never explicitly trained on or given examples of, by generalizing from its broader pre-trained knowledge.

Detailed Explanation: Foundation models often demonstrate zero-shot learning because their pre-training exposed them to such a vast and diverse range of data and tasks that they can often generalize reasonably well to a brand-new task description without needing any task-specific examples. This is conceptually related to, but distinct from, zero-shot prompting: zero-shot learning refers to the model's underlying capability to generalize, while zero-shot prompting refers to the specific technique of simply asking the model to perform a task without providing any examples in the prompt.

Everyday Analogy: It is like an experienced, well-traveled translator who, despite never having specifically studied a particular regional dialect, can often still understand and respond reasonably well based on their broad overall language expertise.

Why It Matters for the Exam: Zero-shot learning is foundational vocabulary under Domain 2's GenAI concepts. Be ready to distinguish it conceptually from the more specific, applied technique of zero-shot prompting tested in Domain 3.

Example: A foundation model correctly classifies a customer review's sentiment into a custom category it was never specifically trained on, simply based on the category description provided in the prompt.

Common Misunderstanding: Learners often treat "zero-shot learning" and "zero-shot prompting" as fully interchangeable. Zero-shot learning describes the model's underlying generalization capability; zero-shot prompting describes the specific technique of using that capability by asking without providing any examples.

Related AWS Service (if applicable): Amazon Bedrock.

Zero-Shot Prompting

Domain(s): 3 **Priority:** High **Category:** Foundation Model Application

Simple Definition: Zero-shot prompting is a prompt engineering technique where you ask a model to perform a task without providing any examples of the desired output.

Detailed Explanation: Zero-shot prompting relies entirely on the model's pre-trained general knowledge and its ability to understand and follow a clearly written instruction, without any demonstration examples to guide it. It is the simplest and most token-efficient prompting technique, sitting at one end of the spectrum that also includes single-shot prompting (one example) and few-shot prompting (multiple examples). Zero-shot prompting tends to work best for simpler, well-understood tasks, while more complex or unusual tasks often benefit from the additional guidance that examples provide.

Everyday Analogy: It is like asking a knowledgeable new employee to complete a task using only a clear written instruction, without showing them any sample of a previously completed version first.

Why It Matters for the Exam: Task Statement 3.2 explicitly names zero-shot prompting as one of the core prompt engineering techniques you must define and differentiate from single-shot, few-shot, and chain-of-thought prompting.

Example: A prompt that simply says "Translate this sentence into French" with no example translations provided is using zero-shot prompting.

Common Misunderstanding: Learners sometimes think zero-shot prompting means the model has no relevant knowledge to draw on. The model is still drawing on everything it learned during pre-training; it simply is not given any task-specific examples within the current prompt itself.

Related AWS Service (if applicable): Amazon Bedrock; Amazon Bedrock Prompt Management.

VERSION B: DOMAIN-ORGANIZED GLOSSARY

This version groups every term under its primary exam domain for fast, domain-by-domain review. Each entry shows only the Simple Definition and Why It Matters for the Exam fields. For full detail on any term, including analogies, examples, and common misunderstandings, refer back to Version A above.

Domain 1: Fundamentals of AI and ML

Accuracy

Simple Definition: The percentage of predictions a model got right out of all predictions made. **Why It Matters:** One of four core performance metrics (with precision, recall, F1); can mislead on imbalanced data.

Activation Function

Simple Definition: A rule inside a neural network that decides how strongly information passes to the next layer. **Why It Matters:** Low-priority; recognize it as part of how neural networks process information.

Algorithm

Simple Definition: A step-by-step method a computer follows to solve a problem. **Why It Matters:** Foundational vocabulary; distinguish clearly from "model" and "training."

Anomaly Detection

Simple Definition: Using AI/ML to automatically spot data points that deviate from a normal pattern. **Why It Matters:** The technique underlying fraud detection, a named real-world AI application category.

Artificial Intelligence (AI)

Simple Definition: The broad field of building systems that perform tasks requiring human-like thinking. **Why It Matters:** The umbrella term containing ML, deep learning, and GenAI; know the nested relationship.

Batch Size

Simple Definition: The number of training examples processed together before a model updates. **Why It Matters:** Low-priority hyperparameter; recognition only, no calculation required.

Classification

Simple Definition: Assigning an input to one of a set of predefined categories. **Why It Matters:** One of three core techniques (with regression, clustering); recognize "which category" scenarios.

Clustering

Simple Definition: Grouping similar items together without predefined categories. **Why It Matters:** Unsupervised technique; recognize "discover natural groupings" scenarios like segmentation.

Computer Vision

Simple Definition: The AI field enabling computers to interpret images and video. **Why It Matters:** Named basic term and application category; know Rekognition vs. Textract.

Confusion Matrix

Simple Definition: A table showing correct and incorrect classification predictions by category. **Why It Matters:** Source of accuracy, precision, recall, and F1 score calculations.

Cross-Validation

Simple Definition: Repeatedly splitting training data to more reliably estimate model performance. **Why It Matters:** Low-priority; recognize its purpose in detecting overfitting.

Data Drift

Simple Definition: When real-world production data starts looking different from the original training data. **Why It Matters:** Supports MLOps monitoring and retraining concepts; distinguish from model drift.

Data Preprocessing

Simple Definition: Cleaning and preparing raw data before it is used to train a model. **Why It Matters:** A named AI/ML pipeline stage, positioned before feature engineering.

Dataset

Simple Definition: The collection of data used to train, validate, or test a model. **Why It Matters:** Foundational vocabulary; know labeled/unlabeled and structured/unstructured distinctions.

Deep Learning

Simple Definition: Machine learning using multi-layer neural networks to learn complex patterns automatically. **Why It Matters:** A named basic term; know it as a subset of ML underlying foundation models.

Dropout

Simple Definition: Randomly ignoring some neural network connections during training to reduce overfitting. **Why It Matters:** Low-priority regularization technique; recognition only.

Epoch

Simple Definition: One complete pass through the entire training dataset. **Why It Matters:** Low-priority hyperparameter related to overfitting/underfitting.

F1 Score

Simple Definition: A single metric combining precision and recall into a balanced score. **Why It Matters:** One of four named performance metrics; best for imbalanced datasets.

Feature

Simple Definition: An individual measurable property of the data used to make predictions. **Why It Matters:** Supports the pipeline discussion, especially feature engineering.

Feature Engineering

Simple Definition: Creating or transforming input variables to help a model learn more effectively. **Why It Matters:** A named pipeline stage; explicitly out of scope to perform, in scope to recognize.

Ground Truth

Simple Definition: The verified, correct label used to train and evaluate a model. **Why It Matters:** Supports labeled data and label quality concepts (the latter tested in Domain 4).

Hyperparameter

Simple Definition: A setting chosen before training that controls how a model learns. **Why It Matters:** Recognize the term; hyperparameter tuning itself is explicitly out of scope to perform.

Inference

Simple Definition: Using an already-trained model to make a prediction on new data. **Why It Matters:** Know the four inferencing types: batch, real-time, asynchronous, serverless.

Label

Simple Definition: The correct answer assigned to a piece of data for supervised learning. **Why It Matters:** Labeled vs. unlabeled data is a named exam distinction.

Learning Rate

Simple Definition: A hyperparameter controlling how big each training adjustment is. **Why It Matters:** Low-priority; recognize its role in training stability.

Machine Learning (ML)

Simple Definition: A subset of AI where systems learn patterns from data rather than explicit rules. **Why It Matters:** The most foundational term on the exam; know its relation to AI, deep learning, GenAI.

Model

Simple Definition: The trained result of applying an algorithm to a specific dataset. **Why It Matters:** Distinguish clearly from "algorithm" (the general method).

Model Deployment

Simple Definition: Making a trained model available for use in a real application. **Why It Matters:** Know the two named methods: managed API service and self-hosted API.

Model Drift

Simple Definition: The gradual decline in a deployed model's performance over time. **Why It Matters:** Often caused by data drift; connects to MLOps monitoring and retraining.

Natural Language Processing (NLP)

Simple Definition: The AI field for understanding and generating human language. **Why It Matters:** Named basic term and application category; know Amazon Comprehend.

Neural Network

Simple Definition: A model structure of interconnected layers loosely inspired by the brain. **Why It Matters:** Foundational vocabulary underlying deep learning and foundation models.

Overfitting

Simple Definition: When a model learns training data too closely and performs poorly on new data. **Why It Matters:** Associated with high variance; tested alongside underfitting and bias/variance tradeoff.

Precision

Simple Definition: Of all positive predictions made, how many were actually correct. **Why It Matters:** Named metric; distinguish from recall using the "accuracy of positive flags" framing.

Recall

Simple Definition: Of all actual positive cases, how many the model successfully identified. **Why It Matters:** Named metric; distinguish from precision using the "catching all real positives" framing.

Regression

Simple Definition: Predicting a continuous numerical value rather than a category. **Why It Matters:** One of three core techniques; recognize "how much/how many" business scenarios.

Regularization

Simple Definition: Techniques during training that prevent a model from becoming too complex and overfitting. **Why It Matters:** Low-priority; supports the overfitting discussion.

Reinforcement Learning

Simple Definition: Learning through rewards and penalties from interacting with an environment. **Why It Matters:** One of three core learning types; basis for RLHF in Domain 3.

Semi-Supervised Learning

Simple Definition: Learning using a small amount of labeled data plus a large amount of unlabeled data. **Why It Matters:** Low-priority; supports the supervised/unsupervised learning discussion.

Speech Recognition

Simple Definition: Converting spoken audio into written text. **Why It Matters:** Named application category; know Amazon Transcribe versus Amazon Polly (opposite directions).

Supervised Learning

Simple Definition: Learning from labeled data where the correct answer is known in advance. **Why It Matters:** One of three core learning types; basis for classification and regression.

Training

Simple Definition: Teaching a model to recognize patterns by exposing it to data. **Why It Matters:** Distinguish clearly from inference (applying the learned model to new data).

Underfitting

Simple Definition: When a model is too simple to capture real patterns, performing poorly even on training data. **Why It Matters:** Associated with high bias; the opposite failure mode from overfitting.

Unsupervised Learning

Simple Definition: Learning patterns from unlabeled data with no predefined correct answers. **Why It Matters:** One of three core learning types; basis for clustering and anomaly detection.

Variance (ML)

Simple Definition: How much a model's predictions change with different training data samples. **Why It Matters:** Pairs with bias; high variance is associated with overfitting.

Domain 2: Fundamentals of GenAI

Agent (AI Agent)

Simple Definition: A system that autonomously plans and carries out multi-step tasks using tools. **Why It Matters:** Core agentic AI concept; know Strands Agents vs. Amazon Bedrock AgentCore.

Chunking

Simple Definition: Breaking large documents into smaller pieces for retrieval and embedding. **Why It Matters:** Named foundational GenAI concept; underpins RAG and vector databases in Domain 3.

Context Window

Simple Definition: The maximum amount of text (in tokens) a model can consider at once. **Why It Matters:** Limits how much information a model can use; a key FM selection criterion.

Continued Pre-Training (Continuous Pre-Training)

Simple Definition: Updating a foundation model with new general data without full retraining. **Why It Matters:** Distinguish from fine-tuning (task-specific) and full pre-training (from scratch).

Diffusion Model

Simple Definition: A generative model that creates images by refining random noise step by step. **Why It Matters:** Named foundational vocabulary; most associated with image generation.

Embedding

Simple Definition: Converting data into numbers that capture its meaning for comparison. **Why It Matters:** Core building block of RAG and semantic search, heavily tested in Domain 3.

Few-Shot Learning

Simple Definition: A model's ability to learn a new task from just a few examples. **Why It Matters:** Distinguish the underlying capability from the applied technique, few-shot prompting.

Foundation Model (FM)

Simple Definition: A large, general-purpose, pre-trained model adaptable to many tasks. **Why It Matters:** Central vocabulary across Domains 2 and 3; know its relation to LLM.

Generative AI (GenAI)

Simple Definition: AI that creates new content rather than just predicting or classifying. **Why It Matters:** Core exam theme; distinguish from traditional predictive ML.

Hallucination

Simple Definition: A confident but false or fabricated AI-generated response. **Why It Matters:** Tested across Domains 2, 4, and 5; connects to grounding and legal risk.

Large Language Model (LLM)

Simple Definition: A foundation model trained on text that understands and generates language. **Why It Matters:** Named basic term; know LLM is a text-focused subset of foundation models.

Latent Space

Simple Definition: The mathematical space where models represent meaning numerically. **Why It Matters:** Low-priority; supports the embedding/vector discussion.

Model Parameter

Simple Definition: An internal numeric value a model learns during training. **Why It Matters:** Parameter count is one factor in FM selection (capability vs. cost/latency).

Model Provider

Simple Definition: The company that develops and offers a particular foundation model. **Why It Matters:** Know that Bedrock provides multi-provider access, including Amazon's own Nova models.

Model Weight

Simple Definition: Another name for a model parameter. **Why It Matters:** Low-priority; recognize "weight" and "parameter" as largely interchangeable.

Multimodal Model

Simple Definition: A foundation model that handles more than one data type (text, image, audio). **Why It Matters:** Named FM selection criterion (modality); know not all FMs are multimodal.

Orchestration

Simple Definition: Coordinating multiple steps, tools, or agents to complete a complex task. **Why It Matters:** Named foundational agentic AI concept supporting multi-step agent behavior.

Pre-Training

Simple Definition: The initial large-scale training phase of a foundation model on general data. **Why It Matters:** The most expensive customization approach; named in the FM lifecycle.

Prompt

Simple Definition: The input given to a foundation model to guide its response. **Why It Matters:** Foundational vocabulary; know its core components (context, instruction, negative prompts).

Synthetic Data

Simple Definition: Artificially generated data resembling real data. **Why It Matters:** Supports GenAI use cases and privacy-preserving data strategies.

Text-to-Code

Simple Definition: Generating program code from a natural-language description. **Why It Matters:** Named use case category; connects to Amazon Q and Kiro.

Text-to-Image

Simple Definition: Generating a new image from a text description. **Why It Matters:** Named use case category; connects to diffusion models.

Text-to-Text

Simple Definition: Generating new text output from text input. **Why It Matters:** Named use case category; the most common GenAI capability.

Token

Simple Definition: A small chunk of text a model processes as a single unit. **Why It Matters:** Foundational vocabulary directly tied to token-based pricing and context window limits.

Tokenization

Simple Definition: The process of breaking text into tokens. **Why It Matters:** Supports token, pricing, and context window concepts.

Tool Use (Function Calling)

Simple Definition: An AI agent calling external tools or APIs to gather information or take action. **Why It Matters:** Named foundational agentic AI concept enabling real-world agent actions.

Top-K Sampling

Simple Definition: Limiting word choices to a fixed number of top candidates before sampling. **Why It Matters:** Low-priority inference parameter, secondary to temperature.

Top-P Sampling

Simple Definition: Limiting word choices to a dynamically sized group based on probability threshold. **Why It Matters:** Low-priority inference parameter, secondary to temperature.

Zero-Shot Learning

Simple Definition: A model's ability to perform a task it was never explicitly trained on. **Why It Matters:** Distinguish the underlying capability from the applied technique, zero-shot prompting.

Domain 3: Applications of Foundation Models

Amazon Bedrock Agents

Simple Definition: A Bedrock feature for building AI agents that complete multi-step tasks. **Why It Matters:** Distinguish from Amazon Bedrock AgentCore (the production deployment/management layer).

Amazon Bedrock Knowledge Bases

Simple Definition: A managed feature connecting FMs to an organization's documents for RAG. **Why It Matters:** The named AWS implementation of RAG; near-certain exam topic.

BERTScore

Simple Definition: An automated metric measuring semantic similarity between generated and reference text. **Why It Matters:** One of four named FM evaluation metrics; more forgiving of paraphrasing than ROUGE/BLEU.

BLEU Score

Simple Definition: An automated metric comparing generated translation to a reference translation. **Why It Matters:** Memorize the pairing: BLEU = translation quality.

Chain-of-Thought Prompting

Simple Definition: Asking a model to reason step by step before answering. **Why It Matters:** A named prompt engineering technique; improves accuracy on multi-step reasoning tasks.

Few-Shot Prompting

Simple Definition: Providing several examples in a prompt before asking the model to handle a new case. **Why It Matters:** A named prompt engineering technique on the example-quantity spectrum.

Fine-Tuning

Simple Definition: Further training a foundation model on a specific, smaller dataset. **Why It Matters:** A key FM customization approach; know its cost position relative to RAG and pre-training.

Grounding

Simple Definition: Anchoring a model's response in verified source material. **Why It Matters:** Connects RAG (Domain 3) to hallucination prevention (Domain 5).

Human-in-the-Loop

Simple Definition: A person reviewing or approving AI output before it is finalized. **Why It Matters:** A named FM evaluation approach and responsible AI safeguard; know Amazon A2I.

Inference Parameters

Simple Definition: Configurable settings (like temperature) controlling how a model generates output. **Why It Matters:** A named FM design consideration; expect scenario questions on adjusting them.

Instruction Tuning

Simple Definition: Fine-tuning a model using instruction-and-response example pairs. **Why It Matters:** A named fine-tuning method; improves general instruction-following.

Model Customization

Simple Definition: Any technique used to adapt a foundation model to a specific need. **Why It Matters:** Know the full cost/complexity spectrum: prompting, RAG, fine-tuning, pre-training, distillation.

Model Evaluation

Simple Definition: Systematically testing a model's quality; Amazon Bedrock Model Evaluation is AWS's service. **Why It Matters:** A named, heavily tested evaluation approach in Domain 3.

Model Latency

Simple Definition: The time it takes a model to generate a response. **Why It Matters:** A named FM selection criterion; tradeoff with model size and capability.

Model Throughput

Simple Definition: The volume of requests a model can process over time. **Why It Matters:** Connects to provisioned throughput as a cost and performance option.

One-Shot Prompting (Single-Shot Prompting)

Simple Definition: Providing exactly one example before asking the model to handle a new case. **Why It Matters:** A named prompt engineering technique between zero-shot and few-shot.

Perplexity

Simple Definition: A metric measuring how well a model predicts the next word in text. **Why It Matters:** Low-priority; secondary to ROUGE, BLEU, BERTScore, and LLM-as-a-judge.

Prompt Engineering

Simple Definition: Designing instructions to get the best response from a foundation model. **Why It Matters:** One of the most heavily tested Domain 3 topics; know all named techniques and risks.

RAG (Retrieval Augmented Generation)

Simple Definition: Letting a model search an organization's own data before responding. **Why It Matters:** One of the most heavily tested Domain 3 concepts; know Bedrock Knowledge Bases.

Reinforcement Learning from Human Feedback (RLHF)

Simple Definition: Using human ratings of outputs as a reward signal to improve a model. **Why It Matters:** Named in fine-tuning data preparation (Task Statement 3.3).

Role Prompting

Simple Definition: Instructing a model to respond as a specific persona or expert. **Why It Matters:** A supporting prompt engineering technique shaping tone and perspective.

ROUGE Score

Simple Definition: An automated metric comparing generated summaries to reference summaries. **Why It Matters:** Memorize the pairing: ROUGE = summarization quality.

Semantic Search

Simple Definition: Searching by meaning rather than exact keyword match. **Why It Matters:** The retrieval mechanism behind RAG, powered by embeddings.

System Prompt

Simple Definition: Instructions establishing a model's overall behavior before user interaction. **Why It Matters:** Distinguish from user prompt; relevant to prompt exposure risk.

Temperature (Inference Parameter)

Simple Definition: Controls how creative versus predictable a model's output is. **Why It Matters:** The most heavily tested inference parameter; know low vs. high temperature use cases.

User Prompt

Simple Definition: The specific request a person submits within a single interaction. **Why It Matters:** Distinguish from system prompt as the supporting concept.

Vector Database

Simple Definition: A database optimized for storing and searching vector embeddings. **Why It Matters:** Memorize the four named AWS options: OpenSearch Service, Aurora, Neptune, RDS for PostgreSQL.

Zero-Shot Prompting

Simple Definition: Asking a model to perform a task with no examples provided. **Why It Matters:** A named prompt engineering technique; the simplest, most token-efficient approach.

Domain 4: Guidelines for Responsible AI

Adversarial Input

Simple Definition: Data deliberately crafted to trick an AI model into a wrong decision. **Why It Matters:** Supports robustness and security threat awareness.

AI Bias

Simple Definition: Unfair model outputs that favor or disadvantage certain groups. **Why It Matters:** A core responsible AI feature; know Amazon SageMaker Clarify for detection.

AI Safety

Simple Definition: Designing AI to avoid causing harm to people. **Why It Matters:** A named core responsible AI feature; connects to Amazon Bedrock Guardrails.

Algorithmic Bias

Simple Definition: Unfairness arising from an algorithm's own design choices, not just data. **Why It Matters:** Distinguishes the source of bias (algorithm design vs. dataset).

Amazon Bedrock Guardrails

Simple Definition: A feature for setting safety limits on what a model can say or do. **Why It Matters:** One of the most heavily tested service names across Domains 4 and 5.

Controllability

Simple Definition: The degree to which humans can monitor and override AI behavior. **Why It Matters:** Especially important for autonomous agentic AI systems.

Data Minimization

Simple Definition: Collecting only the data actually necessary for a specific purpose. **Why It Matters:** Supports responsible dataset characteristics and privacy regulations like GDPR.

Disparate Impact

Simple Definition: A neutral-seeming system that still disproportionately harms a group. **Why It Matters:** Distinguish from disparate treatment (unequal outcomes vs. intentional differential treatment).

Disparate Treatment

Simple Definition: Intentionally treating people differently based on a protected characteristic. **Why It Matters:** Distinguish from disparate impact.

Equity

Simple Definition: Fair outcomes that account for different starting circumstances. **Why It Matters:** Distinguish from equality (identical treatment).

Explainability (XAI)

Simple Definition: The ability to explain why a model produced a particular output. **Why It Matters:** Core theme of Task Statement 4.2; know the explainability/performance tradeoff.

Fairness

Simple Definition: Treating all individuals and groups justly, without systematic disadvantage. **Why It Matters:** One of the six named core responsible AI features; heavily tested.

Human Oversight

Simple Definition: Ongoing organizational practice of monitoring and remaining accountable for AI behavior. **Why It Matters:** Distinguish from human-in-the-loop (a specific workflow step vs. broader governance).

Interpretability

Simple Definition: How easily a human can understand a model's internal reasoning. **Why It Matters:** Know the tradeoff: more powerful models tend to be less interpretable.

LIME

Simple Definition: A technique explaining individual predictions with a simplified local model. **Why It Matters:** Low-priority; a supporting explainability tool alongside SHAP.

Model Card

Simple Definition: Structured documentation of a model's intended use, data, and limitations. **Why It Matters:** Named transparency and governance tool; know Amazon SageMaker Model Cards.

Model Inversion

Simple Definition: An attack that reconstructs sensitive training data details from model outputs. **Why It Matters:** Low-priority; a supporting privacy/security threat.

Protected Characteristic

Simple Definition: A legally protected personal trait (race, gender, age, etc.). **Why It Matters:** Underlies disparate impact and disparate treatment fairness violations.

Responsible AI

Simple Definition: Designing and deploying AI ethically, fairly, safely, and trustworthily. **Why It Matters:** The overarching theme of Domain 4; know its six named core features.

Robustness

Simple Definition: Reliable AI performance under unusual or challenging conditions. **Why It Matters:** A named core responsible AI feature.

SHAP

Simple Definition: A technique calculating each feature's contribution to a specific prediction. **Why It Matters:** Low-priority; a supporting explainability tool incorporated into SageMaker Clarify.

Transparency

Simple Definition: Openness about how an AI system works and what data it used. **Why It Matters:** A named core responsible AI feature; distinguish from explainability.

Veracity

Simple Definition: AI outputs should be truthful and accurate, not presented falsely as fact. **Why It Matters:** A named core responsible AI feature; connects directly to hallucination.

Domain 5: Security, Compliance, and Governance for AI Solutions

AI Governance

Simple Definition: Policies and processes ensuring responsible, compliant AI use. **Why It Matters:** Core theme of Task Statement 5.2; know the Generative AI Security Scoping Matrix.

Audit Trail

Simple Definition: A recorded history of who did what, and when, within a system. **Why It Matters:** Named security and logging requirement; know AWS CloudTrail.

AWS KMS (AWS Key Management Service)

Simple Definition: A managed service for creating and controlling encryption keys. **Why It Matters:** Core encryption service; distinguish from AWS Secrets Manager.

AWS PrivateLink

Simple Definition: Private connectivity to AWS services without using the public internet. **Why It Matters:** Named security service; distinguish from the broader VPC concept.

Compliance

Simple Definition: Meeting legal, regulatory, or industry standards for AI data use. **Why It Matters:** Connects to AWS Artifact and AWS Audit Manager.

Content Filtering

Simple Definition: Screening AI inputs/outputs to block harmful or unwanted material. **Why It Matters:** Implemented via Amazon Bedrock Guardrails; overlaps with toxicity detection.

Customer Managed Key (CMK)

Simple Definition: An AWS KMS encryption key that the customer directly creates and controls. **Why It Matters:** Low-priority; supports the broader encryption discussion.

Data Classification

Simple Definition: Labeling data by sensitivity level (public, confidential, etc.). **Why It Matters:** Supports data governance; connects to Amazon Macie.

Data Lineage

Simple Definition: The documented history of where data came from and how it changed. **Why It Matters:** Named source citation and data origin practice; distinguish from data cataloging.

Data Masking

Simple Definition: Hiding or replacing sensitive information in a dataset. **Why It Matters:** A named privacy-enhancing technology for secure data engineering.

Encryption at Rest

Simple Definition: Protecting stored data so it cannot be read without a decryption key. **Why It Matters:** Named security consideration; pairs with encryption in transit.

Encryption in Transit

Simple Definition: Protecting data as it moves between systems or across a network. **Why It Matters:** Named security consideration; pairs with encryption at rest.

GDPR

Simple Definition: An EU law setting strict rules for personal data privacy. **Why It Matters:** A named compliance regulation relevant to AI training data and automated decisions.

HIPAA

Simple Definition: A US law protecting the privacy and security of health information. **Why It Matters:** A named compliance regulation relevant to healthcare AI applications.

IAM (Identity and Access Management)

Simple Definition: The AWS service controlling who can access which resources and actions. **Why It Matters:** One of the highest-priority named security services in Domain 5.

IAM Role

Simple Definition: Temporary permissions assumable by a user, application, or service. **Why It Matters:** Especially relevant to securing autonomous AI agents.

Model Governance

Simple Definition: Policies and processes for tracking and controlling AI models over their lifecycle. **Why It Matters:** A subset of the broader AI governance umbrella.

Model Registry

Simple Definition: A centralized system for tracking model versions. **Why It Matters:** Low-priority; supports model governance and MLOps.

Model Risk Management (MRM)

Simple Definition: Organizational practice of identifying and controlling model-related risk. **Why It Matters:** Low-priority; especially relevant in regulated industries like banking.

NIST AI RMF

Simple Definition: A voluntary US framework for managing AI-related risk. **Why It Matters:** Low-priority; a supporting governance framework alongside the Generative AI Security Scoping Matrix.

PII (Personally Identifiable Information)

Simple Definition: Information that could identify a specific individual. **Why It Matters:** Connects to Amazon Macie (discovery) and Amazon Bedrock Guardrails (redaction).

Principle of Least Privilege

Simple Definition: Granting only the minimum permissions necessary to do a job. **Why It Matters:** A core IAM best practice, especially critical for AI agents.

Prompt Injection (Technical)

Simple Definition: An attack hiding malicious instructions inside data a model processes. **Why It Matters:** A heavily tested AI-specific security threat across Domains 3 and 5.

Shared Responsibility Model

Simple Definition: AWS secures the cloud infrastructure; the customer secures what they build on it. **Why It Matters:** One of the most heavily emphasized Domain 5 concepts; recommended prior AWS knowledge.

SSE-KMS

Simple Definition: S3 encryption using AWS KMS-managed keys for more granular control. **Why It Matters:** Low-priority; supports the encryption at rest discussion.

SSE-S3

Simple Definition: S3's default, automatic encryption using S3-managed keys. **Why It Matters:** Low-priority; the simpler counterpart to SSE-KMS.

Toxicity (AI Outputs)

Simple Definition: Harmful, offensive, or inappropriate content an AI model might generate. **Why It Matters:** Named security and privacy consideration; addressed via Amazon Bedrock Guardrails.

VPC (Virtual Private Cloud)

Simple Definition: A logically isolated private network within AWS. **Why It Matters:** Supports the broader network security discussion alongside PrivateLink.

VPC Endpoint

Simple Definition: A connection point for private access to AWS services without the public internet. **Why It Matters:** Low-priority; the underlying mechanism behind AWS PrivateLink.

End of Master Glossary - Task 9 of 15 Document version: 1.0 | Companion to the Master Project Blueprint and all five Domain Study Modules Next task: Task 10 - Visual Cheat Sheet and One-Page Summary